A Multi-Platform Configuration Compiler for Vendor-Neutral Network Device State

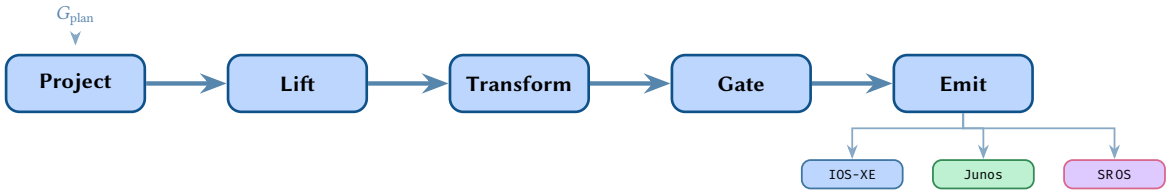
Structured Compilation Across Heterogeneous Vendor Platforms

Simon Knight, Ph.D.

Adelaide, Australia

March 2026 • Version 1.0.0

Last updated: March 18, 2026



Abstract. This report presents a compiler architecture for translating vendor-neutral network device state into platform-specific configurations. Building on the multi-layer graph model of ANK-TR-2026-03, we define a five-stage pipeline — Project, Lift, Transform, Gate, Emit — that mirrors classical compiler structure. The system introduces a typed Device Intermediate Representation (IR) as the central artefact, vendor-specific Abstract Syntax Tree (AST) transforms as the code-generation layer, a formal negation model for configuration attribute state, and a feature capability system that gates emission on platform and version support. Beyond theory, this report addresses critical operational realities for field deployment, demonstrating how the compiler handles Brownfield coexistence, hardware-specific ASIC constraints, Zero Touch Provisioning (ZTP) for Day 0 rollouts, transactional rollbacks, and configuration source-mapping. We establish 19 design rules that govern the compiler's correctness invariants and demonstrate the pipeline through a worked example producing Cisco IOS-XE, NX-OS, IOS-XR, Juniper Junos, Nokia SR OS, and Arista EOS output from a single plan topology.

Contents

AST	Abstract Syntax Tree	2
BGP	Border Gateway Protocol	2
DAG	Directed Acyclic Graph	59
EVPN	Ethernet Virtual Private Network	2
IR	Intermediate Representation	2
NTE	Network Topology Engine	44
PE	Provider Edge	45
QoS	Quality of Service	24
RPSL	Routing Policy Specification Language	56
VRF	Virtual Routing and Forwarding	22
ACL	Access Control List	20

1 Introduction and Motivation

Network configuration remains one of the most error-prone activities in network operations. Studies consistently report that between 40% and 80% of network outages trace to configuration errors [1, 2], and the complexity of configuration management grows super-linearly with the number of devices and vendors in a deployment [3, 4]. While Software-Defined Networking [5, 6] and intent-based networking [7] have shifted control-plane programming towards centralised abstractions, the vast majority of production networks still rely on per-device CLI configuration.

The root cause is a *semantic gap*: operators reason about network-wide intent — "Border Gateway Protocol (BGP) session between PE01 and PE02 in AS 65000 with Ethernet Virtual Private Network (EVPN) address family" — while devices consume vendor-specific syntax — Cisco IOS-XE `router bgp 65000`, NX-OS `router bgp 65000` (with `feature bgp` prerequisite), IOS-XR `router bgp 65000` (candidate-commit), Junos `set protocols bgp group OVERLAY`, Nokia `/configure router bgp`. Bridging this gap manually across six or more platform families is tedious, repetitive, and error-prone.

This report addresses the gap by treating network configuration generation as a *compilation problem*. We define a five-stage pipeline that accepts a vendor-neutral plan topology (the source language), constructs a typed per-device Intermediate Representation (IR), applies vendor-specific Abstract Syntax Tree (AST) transforms, gates emission on platform capabilities, and renders final configuration text (the target language).

Insight: *The semantic gap between operator intent and device syntax is the fundamental driver for configuration compilation.*

1.1 Relationship to ANK-TR-2026-03

The companion report ANK-TR-2026-03 [8] defines the *state model*: a multi-layer attributed graph $G = (V, E, \tau, \alpha)$ that captures the full protocol state of a network in a vendor-neutral manner. That model answers the question "what state exists?" This report answers the complementary question: "given the desired state, how do we produce the configuration commands that *realise* it on each device?"

The state model's plan topology G_{plan} is the input to our compiler. Each device $d \in V$ is projected out of the plan graph, its multi-layer attributes collapsed into a per-device IR tree, and that tree compiled through vendor-specific transforms into executable configuration.

1.2 Contributions

This report makes the following contributions:

1. A **five-stage compilation pipeline** (Project, Lift, Transform, Gate, Emit) with formal semantics.
2. A **typed Device IR schema** with dependency ordering, serialisation format, and validation invariants.
3. A **vendor AST transform system** based on pattern matching and rule registries, with a category-theoretic formalisation.
4. A **formal negation model** for configuration attribute states including vendor-specific default handling.
5. A **feature capability model** that gates transforms on platform, version, and feature support.
6. A **diff compilation** algorithm that produces ordered changesets from IR differences.
7. **19 design rules** governing the compiler's correctness invariants.
8. A **worked example** compiling a PE router's IS-IS, BGP, and EVPN configuration across six vendor platforms.
9. A **practical implementation path** showing how graph query languages and structured tooling can realise the formal model in production-grade systems.

1.3 Practical Motivation

This work is motivated by a concrete engineering problem: large-scale networks contain hundreds or thousands of devices across multiple vendor platforms, and manually producing correct configuration for each is unsustainable. The formal model we develop is not academic for its own sake — it provides the structural guarantees needed to build *reliable automation tooling* that operators can trust with production networks.

The category-theoretic framework, in particular, serves as a design-time "type system" for the compiler: it catches structural errors (missing transforms, non-commuting vendor paths, incomplete negation handling) during development rather than during a production deployment at 2am. section 42 describes how every formal construct maps to a concrete implementation technology — graph query languages for projection, pattern matching for transforms, property-based testing for verification.

1.4 Report Structure

The report is organised in eight parts. part I establishes the compiler analogy, surveys related work, and fixes mathematical notation. part II defines the Device IR and its schema. part III presents the AST transform system.

part IV addresses negation semantics. part V describes the feature capability model. part VI covers template rendering and diff compilation. part VII demonstrates the pipeline through a worked example and discusses implementation. part IX summarises contributions and future work.

2 Compiler Analogy and Architecture Overview

The configuration compilation pipeline mirrors classical compiler architecture [9–11]. The analogy extends beyond surface similarity: like LLVM's multi-target IR [12], our Device IR serves as a platform-independent intermediate representation from which multiple vendor-specific outputs are generated. The formal verification aspects draw inspiration from CompCert's certified compilation [13]. The five pipeline stages map directly to classical compiler phases:

Classical Compiler	Config Compiler	Stage Name
Source program	Plan topology G_{plan}	—
Lexing / Parsing	Device projection π_d	**Project**
Semantic analysis	Multi-layer attribute lifting	**Lift**
IR construction	IR tree construction	—
Optimisation passes	Vendor AST transforms	**Transform**
Target capability check	Feature gating	**Gate**
Code generation	Template rendering	**Emit**
Target program	Device configuration text	—

The key insight is that the IR serves the same role in both systems: it is a *canonical, target-independent representation* that decouples analysis from code generation. Just as LLVM's IR [12] enables multiple frontends to target multiple backends, our Device IR enables a single plan topology to produce configuration for any supported platform.

Insight: *The analogy is not superficial: both compilers and our system face the same fundamental challenge of meaning-preserving translation across abstraction levels.*

Design Rule 1: Separation of Concerns

The IR must not encode vendor-specific syntax or structural conventions. All vendor knowledge resides exclusively in the Transform and Emit stages.

Design Rule 2: Correctness-by-Construction

Every emitted configuration line must be traceable to an IR node. The compiler shall not emit configuration that has no representation in the IR.

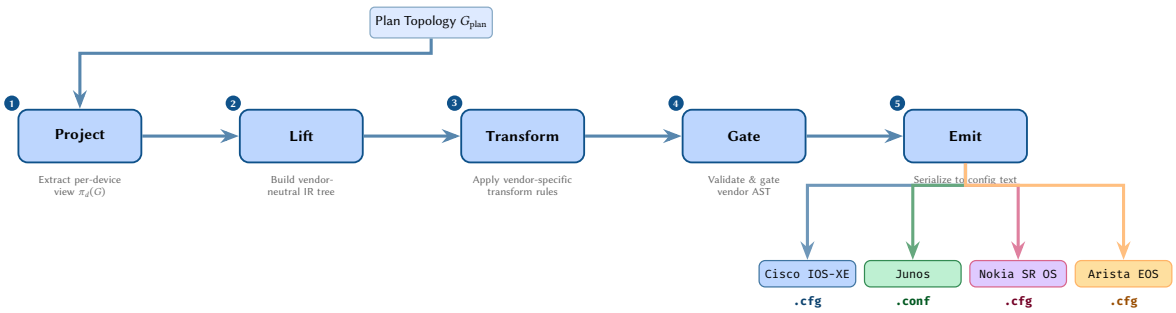


Figure 1. The five-stage network configuration compiler pipeline. A plan topology G_{plan} enters at the top; vendor-specific configuration files emerge at the bottom.

3 Related Work

The configuration compilation problem sits at the intersection of programming language theory, network verification, and systems automation. We survey work across three areas: domain-specific languages [14] for network configuration, verification and analysis tools that reason about correctness, and operational automation frameworks that manage device interaction. The broader context of model-driven architecture [15] and generative programming [16] provides the software engineering foundation.

3.1 Network Configuration Languages

Propane [17, 18] compiles high-level routing policy specifications into BGP configurations. It demonstrates that network-wide intent can be compiled to per-device configuration, but restricts itself to BGP routing policy and does not address multi-vendor output or the full device configuration space.

NetComplete [19] uses program synthesis to autocomplete partial network configurations, finding parameter values that satisfy network-wide properties. While complementary to our approach, it operates on existing vendor-specific configurations rather than compiling from a vendor-neutral IR.

NetKAT [20] provides a formal algebraic framework for specifying and reasoning about network packet-processing behaviour. Its Kleene algebra with tests gives a sound equational theory for network programs. Our work operates at the configuration level rather than the packet-processing level, but shares the goal of formal foundations.

Merlin [21] provides a language for expressing network-wide resource provisioning policies with bandwidth guarantees. Like Propane, it addresses a specific aspect of configuration (resource allocation) rather than full device configuration compilation.

Frenetic [22] introduced the concept of composable network programming abstractions, enabling modular specification of forwarding policies. El-Hassany et al. [23] extended configuration synthesis to network-wide properties using constraint solving, while Jin et al. [24] explored intent-based abstractions that bridge the gap between high-level objectives and device-level configuration.

Capirca [25] is an open-source tool developed by Google that pioneered the concept of a multi-vendor policy compiler. It takes a vendor-agnostic representation of firewall and ACL policies and compiles them into platform-specific syntax (Cisco, Juniper, iptables, etc.). Our work generalizes this "compile-to-vendor" concept beyond stateless ACLs into stateful routing protocols, VPNs, and complex hardware topologies.

3.2 Network Automation Frameworks

NAPALM [26] provides a vendor-neutral Python API for retrieving and deploying configuration. It uses "getters" to normalise retrieved state and "merge/replace" operations for deployment, but does not include a compilation layer — users must provide vendor-specific configuration text.

Nornir [27] is a pluggable automation framework that orchestrates tasks across network inventories. It provides concurrency, inventory management, and task composition but delegates configuration generation to user-written code or templates. **Netmiko** [28] and **Scrapli** [29] operate at a lower level, providing multi-vendor SSH/NETCONF transport libraries. These are building blocks for configuration delivery rather than generation.

Ansible [30] provides network modules for many platforms, abstracting common operations (interface configuration, VLAN management) behind a YAML-driven interface. However, its abstraction is per-task rather than per-device: there is no global IR and no cross-feature dependency management. **Terraform** [31] applies infrastructure-as-code principles but treats network devices as opaque resources rather than providing protocol-aware compilation.

Kubernetes Operators and GitOps [32]: The software engineering industry has heavily adopted declarative intent systems, primarily through Kubernetes Custom Resource Definitions (CRDs) and the GitOps pattern. While operators continuously reconcile desired state against current state, they typically rely on the underlying system (e.g., Kubelet) to handle the actual resource translation. Our compiler acts as the missing translation engine for legacy network environments, enabling modern GitOps pipelines to safely manage heterogeneous physical infrastructure.

3.3 Data Models and Protocols

YANG [33, 34] provides a data modelling language for network configuration, and **OpenConfig** [35] defines vendor-neutral YANG models for common network features (e.g. the OpenConfig BGP model [36]). NETCONF [37], RESTCONF [38], and gNMI [39] provide transport protocols for configuration delivery. The YANG module classification framework [40] and RFC 8345's topology model [41] provide complementary data models, while RFC 8309 [42] formalises the distinction between service and device configuration models. These are complementary to our work: YANG models could serve as an alternative IR schema, and NETCONF/gNMI as alternative output formats. However, OpenConfig adoption remains incomplete across vendors and features, motivating our approach of targeting native CLI/XML formats.

3.4 Network Verification

Batfish [43] parses vendor configurations into a vendor-neutral data model for offline analysis. It performs the *inverse* of our compilation: decompiling vendor configuration into a normalised form. Batfish's vendor-neutral model is conceptually similar to our IR, though designed for analysis rather than generation.

Minesweeper [44] uses SMT solving to verify network-wide properties against router configurations. Our future work considers using SMT-based equivalence checking to verify that compiled configurations preserve the semantics of the source IR.

Header Space Analysis [45] provides a geometric framework for static network checking. **VeriFlow** [46] verifies network-wide invariants in real time as forwarding rules change, while **Libra** [47] scales forward-

Table 1. Comparison of network configuration tools and systems. This work fills a gap in the landscape: it is the only system that performs full device configuration compilation (not just a single protocol or a data model) across multiple vendor platforms with formal foundations rooted in graph-theoretic and category-theoretic models. ✓ = yes, — = no, ~ = partial.

Tool / System	Scope	Multi-Vendor	Formal Model	Direction	Config Gen.
This work	Full device config	✓	Graph + Category theory	Compile	✓
Propane [17]	BGP policy	—	SMT / Datalog	Compile	✓
NetComplete [19]	Synthesis / autocomplete	~	SMT	Compile	✓
NetKAT [20]	Packet processing	—	Kleene algebra	Compile	—
Merlin [21]	Resource provisioning	—	Regular expressions	Compile	~
Frenetic [22]	Packet processing	—	Functional	Compile	—
Header Space [45]	Forwarding verification	✓	Geometric	Decompile	—
VeriFlow [46]	Real-time verification	✓	—	Decompile	—
NAPALM [26]	Vendor-neutral API	✓	—	Neither	—
Nornir [27]	Orchestration framework	✓	—	Neither	—
Ansible Network [30]	Per-task abstraction	✓	—	Both	~
Batfish [43]	Verification	✓	Datalog	Decompile	—
OpenConfig / YANG [35]	Data model	~	YANG schema	Neither	—

ing table verification to large networks through divide-and-conquer decomposition. **Plankton** [48] and **Tiramisu** [49] extend scalable verification to multi-layer network configurations. Weitz et al. [50] apply SMT solving specifically to BGP configuration verification, a technique that could complement our compilation pipeline. The complexity studies of Benson et al. [1] and the misconfiguration detection work of Le et al. [4] and Sung et al. [3] further motivate the need for automated configuration generation to reduce human error.

Table 1 provides a structured comparison across key dimensions: scope, multi-vendor support, formal foundations, compilation direction, and configuration generation capability.

4 Mathematical Preliminaries

This section establishes the notation and formal framework used throughout the report. We draw on category theory [51–53] to formalise the relationship between vendor-neutral and vendor-specific representations, on lattice theory [54] for the negation model, and on graph-theoretic foundations [55] to express dependency constraints in the IR. Standard algorithmic complexity results [56] apply to the transform pipeline.

4.1 Notation

We adopt the following notation throughout this report. Let D denote the set of all devices in a plan topology. For a device $d \in D$:

- $G_{\text{plan}} = (V, E, \tau, \alpha)$ is the plan topology graph from [8], where V is the vertex set, E the edge set, τ a type function, and α an attribute function.
- $\text{IR}_d = (N, E_t, \tau_n, \alpha_n)$ is the device IR tree for device d, where N is the set of IR nodes, $E_t \subseteq N \times N$ the tree edges, $\tau_n : N \to \mathcal{T}_{\text{IR}}$ the node type function, and $\alpha_n : N \to \mathcal{A}$ the attribute function.
- $\mathcal{T}_{\text{IR}} = \{\texttt{IRDevice}, \texttt{IRInterface}, \texttt{IRRoutingInstance}, \ldots\}$ is the set of IR node types.
- AST_v denotes the vendor-specific abstract syntax tree for vendor $v \in \mathcal{V}$.
- $\mathcal{V} = \{\text{cisco}, \text{junos}, \text{nokia}, \text{arista}\}$ is the set of supported vendors.

4.2 Category-Theoretic Sketch

We briefly sketch a categorical view [51, 57] that will be developed fully in section 12. The compilation pipeline can be viewed as a composition of functors between categories:

$$\mathbf{Plan} \xrightarrow{\pi} \mathbf{IR} \xrightarrow{F_v} \mathbf{AST}_v \xrightarrow{R_v} \mathbf{Config} \tag{1}$$

where **Plan** is the category of plan topologies and topology morphisms, **IR** the category of typed IR trees and structure-preserving maps, $\mathbf{AST}_v$ the category of vendor-specific ASTs, and **Config** the category of configuration texts. The key requirement is that the diagram commutes: for any two vendors v_1, v_2, the composed functors $R_{v_1} \circ F_{v_1} \circ \pi$ and $R_{v_2} \circ F_{v_2} \circ \pi$ produce *semantically equivalent* configurations — they realise the same network state on different platforms.

4.3 Partial Orders

The dependency ordering among IR node types forms a partial order $(\mathcal{T}_{\text{IR}}, \preceq)$ where $t_1 \preceq t_2$ means "type t_1 must be configured before type t_2." This is analogous to Lamport's causal ordering [58]: an event (configuration

statement) must not precede its dependencies. The partial order induces a topological sort [56] that determines the order of configuration emission. We use Hasse diagrams to visualise these orderings.

The configuration attribute state space forms a bounded lattice $(S, \sqsubseteq)$ with bottom element $\bot$ (undefined), default element $\delta_v(a)$ (vendor v's default for attribute a), and explicit values at the top. Negation is formalised as an involution on this lattice.

Device IR

5 Device Projection from Plan Topology

The first stage of compilation extracts a per-device view from the plan topology. Given the plan graph $G_{\text{plan}} = (V, E, \tau, \alpha)$, the *device projection* π_d extracts all information relevant to device d.

Definition 1: Device Projection

The device projection $\pi_d : G_{\text{plan}} \to \text{IR}_d$ is a function that, given a plan topology and a device $d \in V$, produces the device IR tree:

$$\pi_d(G_{\text{plan}}) = (N_d, E_d, \tau_d, \alpha_d) \tag{2}$$

where:

- N_d contains an `IRDevice` root node, one `IRInterface` node for each edge $e \in E$ incident on d, one `IRRoutingInstance` node for each routing context, and protocol session nodes for each protocol configured on d.
- E_d encodes the parent–child relationships forming a tree.
- τ_d assigns each node its IR type from $\mathcal{T}_{\text{IR}}$.
- α_d collects the attributes from α restricted to d and its incident edges.

Design Rule 3: Device IR Completeness

The projection π_d must extract *all and only* the state that device d needs. No information about other devices' internal state shall appear in IR_d.

Design Rule 4: No Cross-Device References

All cross-device references (e.g., BGP peer addresses, OSPF area memberships) must be resolved during projection. The resulting IR tree is self-contained.

The projection process involves three sub-steps:

1. **Vertex extraction**: Identify d and its directly connected neighbours in G_{plan}.
2. **Edge collection**: Gather all edges incident on d, including their multi-layer attributes.
3. **Protocol resolution**: For each protocol session involving d, resolve the remote endpoint's addressing and parameters into local IR attributes.

Insight: *Projection is where network-wide knowledge gets "compiled away" into per-device facts. After projection, each device's IR can be processed independently.*

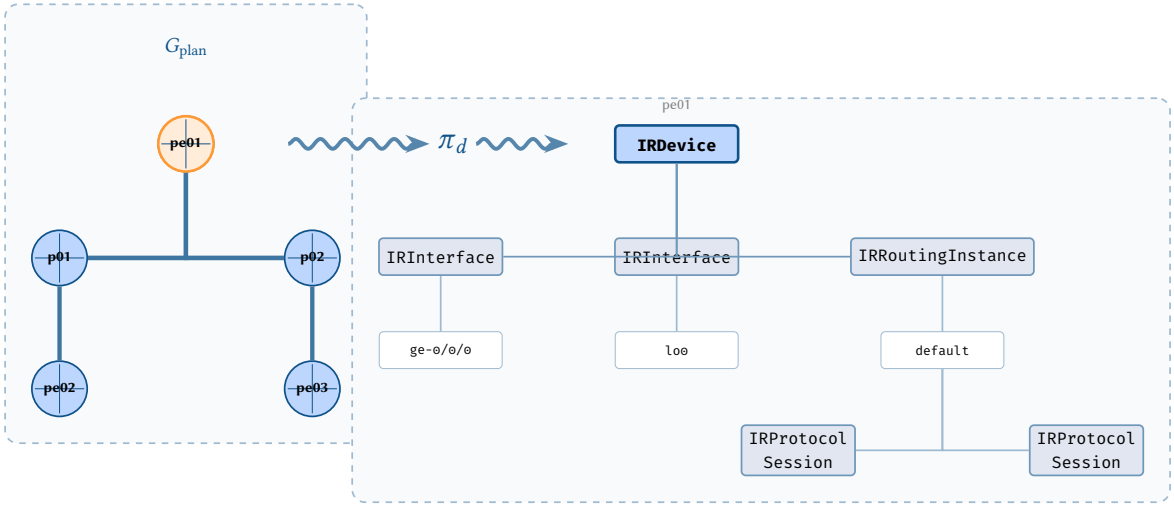


Figure 2. Device projection π_d: a single device (pe01, highlighted) is extracted from the plan topology G_{plan} and its neighbourhood is lowered into a vendor-neutral IR tree.

5.1 Multi-Layer Attribute Collapse

The plan topology assigns attributes at multiple layers (`phy_`, `switchport_`, `ipv4_`, `bgp_`, etc.) as defined in [8]. During projection, these multi-layer attributes must be *collapsed* onto the appropriate IR nodes. A single physical interface may carry L1 physical attributes, L2 VLAN assignments, L3 IP addressing, and protocol-specific parameters — all of which must be gathered and attached to the corresponding `IRInterface` node.

The collapse function $\gamma : \alpha(d) \to \alpha_n$ maps plan-level attributes to IR-level attributes:

$$\gamma(\alpha(d, e)) = \bigcup_{l \in \mathcal{L}} \alpha_l(d, e) \quad \text{where } \mathcal{L} = \{L1, L2, L2.5, L3, L3.5, \ldots\} \tag{3}$$

The result is a flat attribute map on each IR node that carries all relevant configuration state regardless of the protocol layer it originated from.

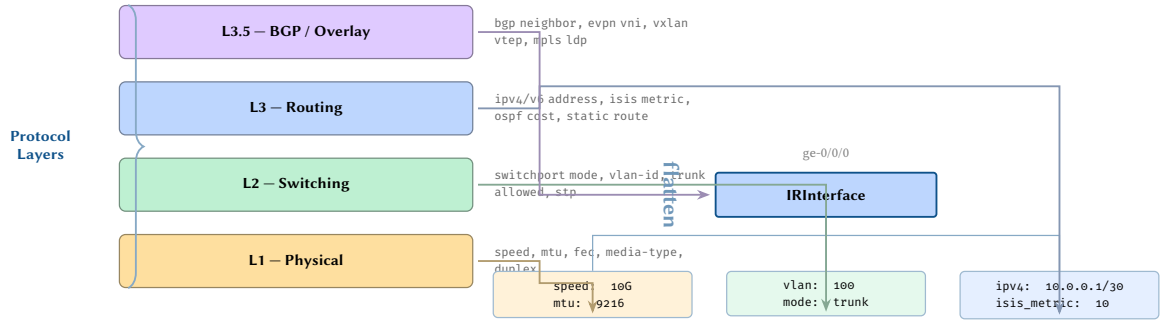


Figure 3. Layer collapse: properties from four protocol layers (L1 Physical through L3.5 BGP/Overlay) are flattened onto a single `IRInterface` node. Colour coding shows the provenance of each attribute group. The IR eliminates layer boundaries, presenting a unified view of all interface properties regardless of their protocol origin.

6 The IR Schema

The Device IR is organised as a *typed attributed tree* with a fixed set of node types forming a hierarchy.

Definition 2: Device IR

A Device IR is a typed attributed tree $\text{IR}_d = (N, E_t, \tau_n, \alpha_n)$ where:

- N is a finite set of nodes.
- $E_t \subseteq N \times N$ forms a rooted tree.
- $\tau_n : N \to \mathcal{T}_{\text{IR}}$ assigns each node a type from the IR type hierarchy.
- $\alpha_n : N \to (\text{Key} \rightharpoonup \text{Value})$ assigns each node a partial map of typed attributes.

6.1 IR Node Type Hierarchy

The IR type hierarchy is:

- `IRDevice` — root node representing the device itself.
  - `IRInterface` — a network interface (physical, logical, loopback, LAG, subinterface).
  - `IRRoutingInstance` — a routing context (global table or VRF).
    - `IRProtocolSession` — a routing protocol instance (IS-IS, OSPF, BGP group/neighbor).
  - `IRPolicy` — a routing policy or route-map.
  - `IRACL` — an access control list.
  - `IRQoS` — QoS policy, classifier, or scheduler.
  - `IRMpls` — MPLS/LDP/SR configuration.
  - `IREvpn` — EVPN instance and service configuration.

Each node type defines a schema of required and optional attributes with their types. For example, `IRInterface` requires `name` (string), `enabled` (boolean), and `mtu` (integer), while optionally carrying `ipv4_address` (prefix), `ipv6_address` (prefix), and `description` (string).

Design Rule 5: IR Type Closure

Every vendor configuration construct that the compiler supports must map to an IR node type. If a vendor feature cannot be represented in the IR, it is outside the compiler's scope.

Design Rule 6: Attribute Typing

All IR attributes are typed [59]. No attribute may carry opaque string blobs that embed vendor syntax. Types include: `bool`, `u32`, `string`, `ipv4_prefix`, `ipv6_prefix`, `asn`, `community`, and enumerations.

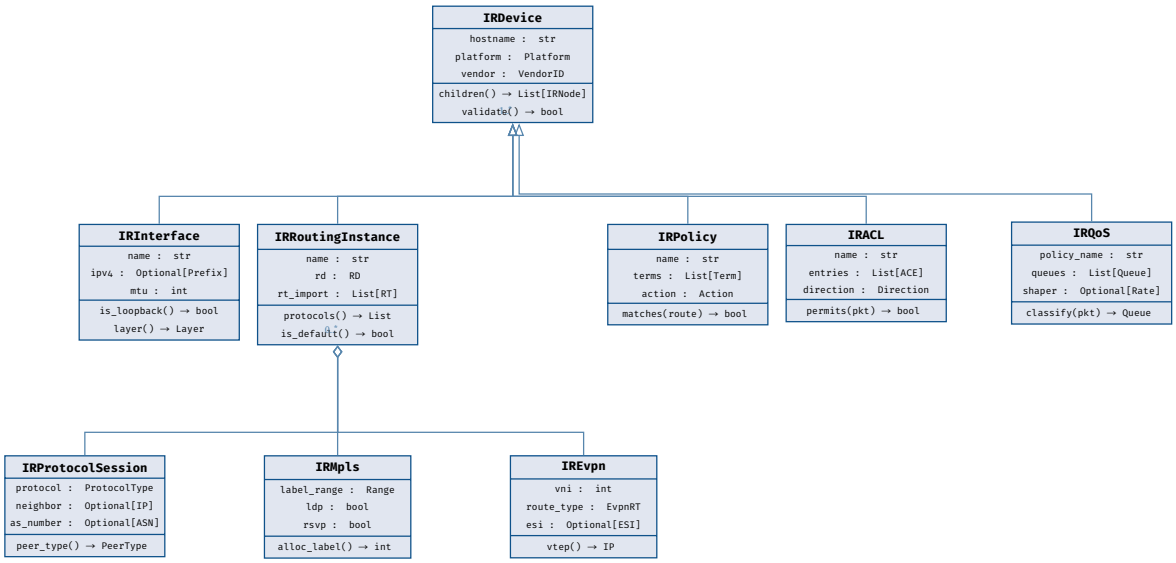


Figure 4. UML class hierarchy of the intermediate representation. `IRDevice` is the root; `IRRoutingInstance` composes protocol sessions, MPLS, and EVPN sub-trees. Hollow triangles denote inheritance; diamonds denote composition.

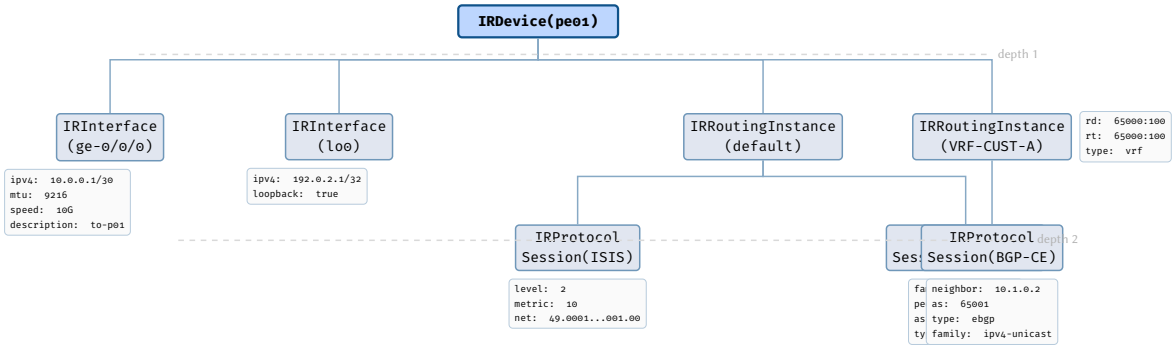


Figure 5. Concrete IR tree for PE router `pe01`. Attribute boxes (grey) show the values carried by each IR node. The default routing instance contains ISIS and BGP-EVPN sessions; VRF-CUST-A contains a customer-facing eBGP session.

7 IR Dependency Ordering

Configuration elements on a device have ordering dependencies: an interface must exist before it can be assigned to a routing instance, a routing instance must exist before a protocol session can be configured within it, and a policy must be defined before it can be referenced by a protocol session.

Definition 3: IR Dependency Order

The IR dependency order is a strict partial order $(\mathcal{T}_{\text{IR}}, \prec)$ where $t_1 \prec t_2$ means "all nodes of type t_1 must be configured before any node of type t_2." The relation is:

$$\texttt{IRDevice} \prec \texttt{IRInterface} \prec \texttt{IRRoutingInstance} \prec \texttt{IRProtocolSession} \tag{4}$$

$$\texttt{IRPolicy} \prec \texttt{IRProtocolSession} \tag{5}$$

$$\texttt{IRACL} \prec \texttt{IRInterface} \tag{6}$$

$$\texttt{IRMpls} \prec \texttt{IRProtocolSession} \tag{7}$$

$$\texttt{IRQoS} \prec \texttt{IRInterface} \tag{8}$$

Design Rule 7: Dependency Monotonicity

If IR node B depends on IR node A, then A must be emitted before B in the output configuration. The compiler must respect the topological sort of the dependency DAG.

The dependency DAG is vendor-independent: it captures the logical ordering constraints inherent in network configuration semantics. However, the *granularity* of ordering may vary by platform. Some platforms (notably Junos) accept configuration in any order via their candidate-commit model, while others (Cisco IOS) require strict ordering during interactive CLI entry. The compiler emits in dependency order regardless, ensuring correctness on all platforms.

Insight: *Dependency ordering is not just a convenience — many platforms reject configuration commands that reference undefined objects. Cisco IOS will reject a route-map reference to an undefined prefix-list.*

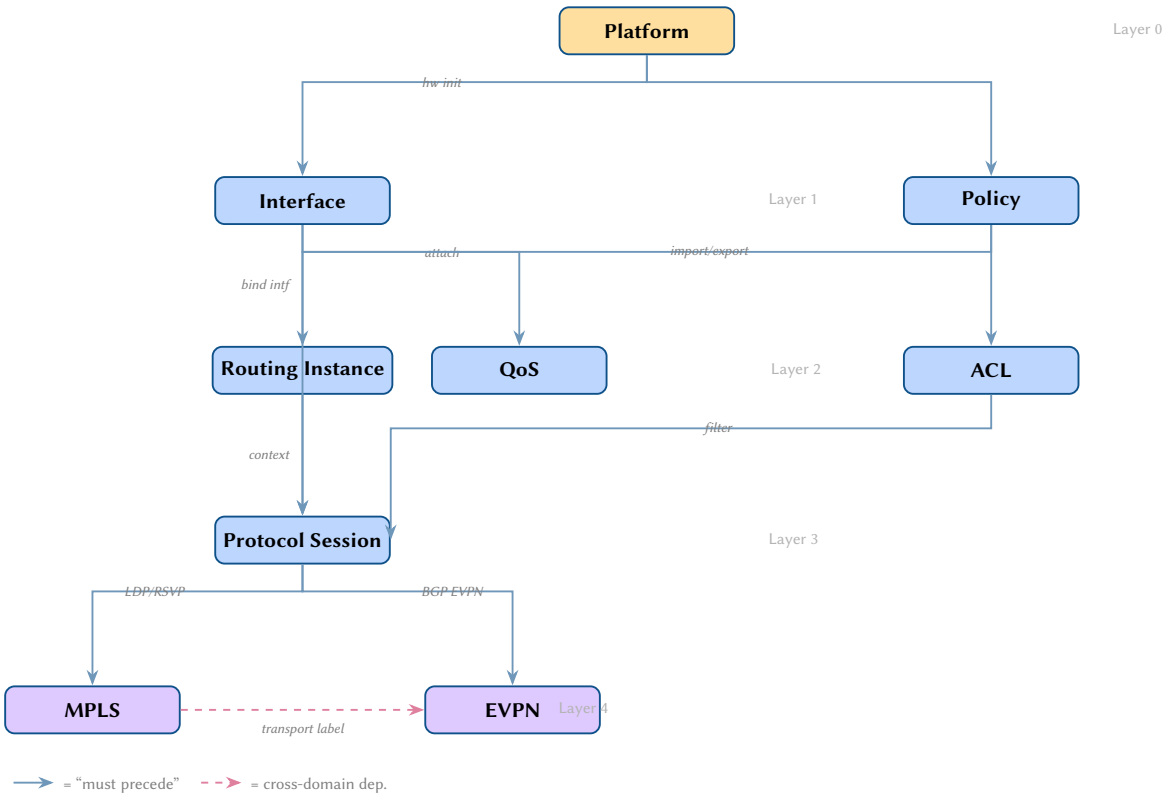


Figure 6. Configuration ordering DAG. Directed edges encode "must be configured before" constraints. Platform initialisation is the root; MPLS and EVPN are the deepest dependencies. Dashed edges mark cross-domain constraints.

8 IR Serialisation and Validation

Design Rule 8: IR Validity Precondition

The IR must be validated against its schema before being passed to the Transform stage. An invalid IR must halt compilation with a diagnostic error.

8.1 Serialisation Format

The IR is serialised as a JSON document with a well-defined schema. Each IR node is represented as a JSON object with fields `type`, `attributes`, and `children`. The serialised form enables IR inspection, debugging, and interchange between pipeline stages.

IR Serialisation

```
{
  "type": "IRDevice",
  "attributes": {
    "hostname": "pe01",
    "platform": "cisco_iosxe",
    "version": "17.9.1"
  },
  "children": [
    {
```

```
      "type": "IRInterface",
      "attributes": {
        "name": "GigabitEthernet0/0/0",
        "enabled": true,
        "mtu": 9000,
        "ipv4_address": "10.0.0.1/30"
      },
      "children": []
    }
  ]
}
```


8.2 Validation Invariants

The IR validator checks:

1. **Structural validity**: the IR forms a rooted tree with `IRDevice` at the root.
2. **Type validity**: every node's type is in $\mathcal{T}_{\text{IR}}$ and the parent–child type relationship conforms to the hierarchy.
3. **Attribute completeness**: all required attributes are present and correctly typed.
4. **Referential integrity**: policy and ACL references point to nodes that exist in the same IR tree.
5. **Address validity**: IP addresses and prefixes are syntactically valid.

AST Transform System

9 Vendor-Neutral to Vendor-Specific Mapping

The Transform stage converts the vendor-neutral IR tree into a vendor-specific AST. This is the heart of the compiler: where structural and syntactic differences between platforms are resolved.

The mapping is a *graph homomorphism* from the IR tree to the vendor AST. Each IR node maps to one or more AST nodes; the tree structure may be reorganised; and attributes may be renamed, reformatted, or split across multiple AST nodes.

Definition 4: Vendor Transform

A vendor transform for vendor v is a function $F_v : \text{IR}_d \to \text{AST}_v$ that maps each IR node $n \in N$ to a subtree $F_v(n) \subseteq \text{AST}_v$, such that the tree structure is preserved up to local reorganisation:

$$\forall (n_1, n_2) \in E_t : \exists \text{ a path from } F_v(n_1) \text{ to } F_v(n_2) \text{ in } \text{AST}_v \tag{9}$$

Insight: *The transform is not a simple renaming. A single IR node may expand into a subtree of AST nodes, or multiple IR nodes may merge into a single AST construct.*

Design Rule 9: Transform Completeness

Every IR node type must have a corresponding transform rule for every supported platform. An IR type without a transform for platform p must be caught by the feature gate (section 34).

Design Rule 10: Rule Determinism

For a given IR node and vendor, exactly one transform rule must match. The transform function is deterministic: the same input always produces the same output.

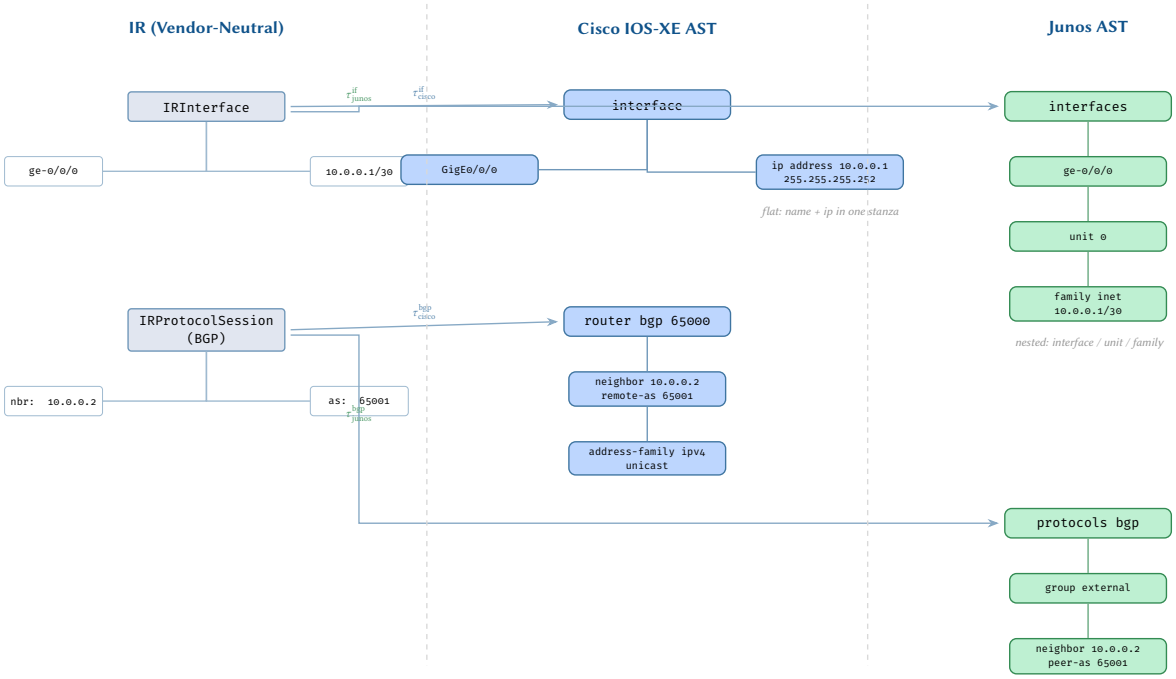


Figure 7. Transform overview: vendor-neutral IR nodes (left) are mapped to structurally different Cisco IOS-XE (centre) and Junos (right) AST fragments. Each transform τ encodes vendor-specific naming, nesting, and keyword conventions.

10 Transform Rules and Pattern Matching

Each transform rule consists of:

1. **Pattern**: the IR node type and optional attribute predicates that the rule matches.
2. **Guard**: an optional boolean condition on the node's attributes or context (e.g., "only if `address_family` = `evpn`").
3. **Action**: the AST subtree to produce, parameterised by the matched node's attributes.
4. **Priority**: a numeric priority for disambiguation when multiple rules could match (lower number = higher priority).

This pattern-guard-action structure draws on the visitor pattern [60] and strategic term rewriting [61, 62], adapted for network configuration domains. Rules are organised in a *transform registry*, a data structure that indexes rules by (IR node type, vendor) for efficient lookup. When the Transform stage processes an IR node, it queries the registry for all rules matching the node's type and the target vendor, evaluates guards in priority order, and applies the first matching rule.

Rather than encoding transform rules as imperative code, the compiler represents them as *tree rewriting rules* — declarative mappings from IR subtree patterns to AST subtree templates. fig. 8 shows this concretely: the IR subtree rooted at an `IRProtocolSession` node with `protocol=bgp` is matched by the BGP transform rule, which produces a vendor-specific AST subtree. The IR node's attributes (`peer_address`, `peer_asn`, `address_family`) are bound during pattern matching and substituted into the AST template.

Insight: *Tree rewriting rules are easier to verify than imperative code. Each rule can be tested in isolation with a simple input IR node and an expected output AST subtree.*

Figure 8. BGP AST transformation. The vendor-neutral IR node (left) carries all session attributes in a flat record. The Cisco functor F_{cisco} restructures these into an idiomatic IOS-XE AST tree with nested `neighbor`, `address-family`, and activation nodes. Circled numbers trace attributes through the transformation.

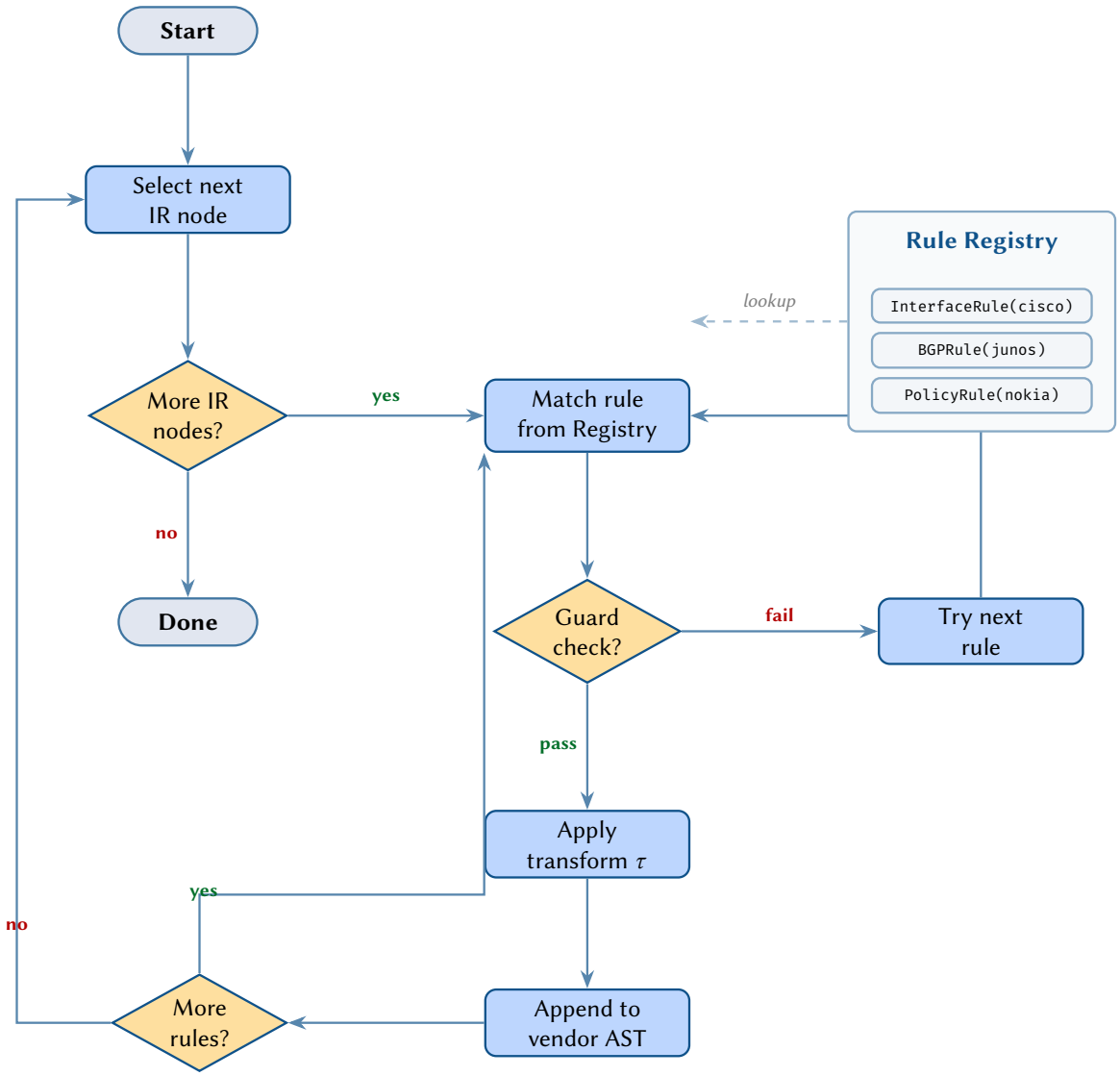


Figure 9. Transform rule processing flowchart. For each IR node, the compiler looks up matching rules from the Rule Registry, evaluates guard predicates, and applies the first passing transform to produce vendor AST fragments.

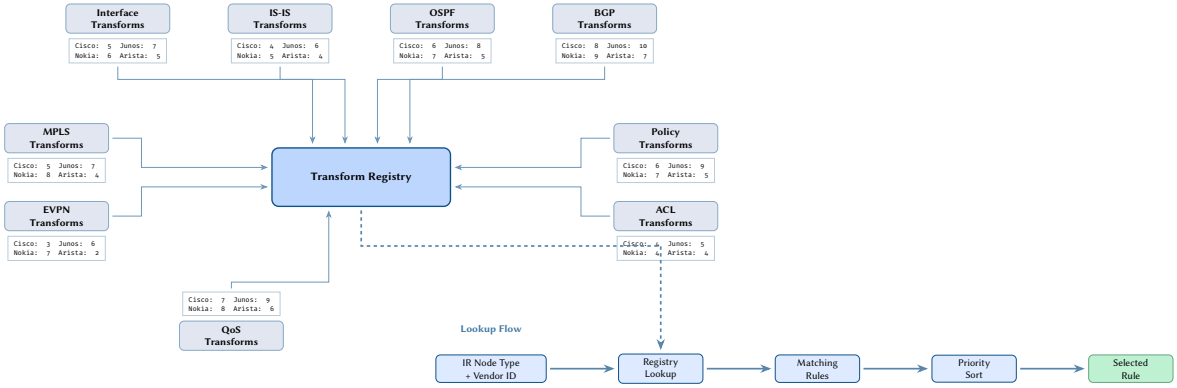


Figure 10. Transform registry architecture. Protocol-specific modules register vendor transform rules (with rule counts per vendor). At compile time, a lookup keyed by IR node type and vendor identifier retrieves matching rules, applies priority sorting, and returns the selected transform.

11 Vendor Structural Differences

The four major vendor platforms exhibit fundamental structural differences in how they represent the same network concepts. Understanding these differences is essential for designing correct transforms.

11.1 Cisco IOS-XE

Cisco IOS-XE [63] uses a *flat imperative* configuration model inherited from the original IOS lineage. Configuration is a sequence of top-level commands, some of which enter sub-modes (contexts), but the overall structure

is shallow compared to hierarchical platforms. Each command takes effect immediately upon entry — there is no candidate configuration or commit step, which means the compiler's output order directly determines the order in which the device applies changes.

- **Interface configuration** under `interface GigabitEthernet0/0/0` with sub-commands for IP addressing, shutdown state, and protocol enablement (e.g. `mpls ip`, `isis enable`).
- **BGP configuration** under `router bgp ASN` with per-neighbor sub-commands and `address-family` sub-modes that require explicit `neighbor ... activate`.
- **Policy objects** (route-maps, prefix-lists, ACLs) as top-level named stanzas referenced by name from protocol configuration.
- **IS-IS and OSPF** as `router isis`/`router ospf` top-level blocks, with interface association done *under the interface* rather than under the protocol.

11.2 Juniper Junos

Junos [64, 65] uses a *hierarchical* configuration model with a candidate-commit architecture. All changes are staged in a candidate configuration and applied atomically via `commit`, giving Junos inherent transactionality that IOS-XE lacks. The configuration is a deeply nested tree of curly-brace-delimited stanzas, where the path from root to leaf fully qualifies each attribute.

Insight: *IOS-XE's immediate-apply model means the compiler must emit prerequisite objects (prefix-lists, route-maps) before the protocol configuration that references them. A route-map referenced by a BGP neighbor that does not yet exist will silently succeed but have no effect — a subtle correctness hazard that the IR dependency ordering (§7) is designed to prevent.*

- **Deep nesting**: `protocols { bgp { group OVERLAY { neighbor 10.0.0.1 { ... } } } }` — attribute identity is determined by position in the hierarchy, not by a flat keyword.
- **Dual representation**: the same configuration can be expressed as hierarchical (curly-brace) or flat (`set` lines). The compiler must support both output formats (§38).
- **Separate policy framework**: policy-statements and firewall filters are distinct top-level objects, referenced by name from protocol configuration via `import`/`export`.
- **Interface model**: `interfaces ge-0/0/0 unit 0 family inet address` — the unit (logical subinterface) is a first-class structural element absent from Cisco platforms.
- **Routing instances**: top-level `routing-instances` containers that encapsulate VRFs, with their own protocol and interface bindings.

11.3 Nokia SR OS

Nokia SR OS [66, 67] offers two CLI styles: classic CLI and model-driven (MD-CLI). The compiler targets the MD-CLI, which uses a fully YANG-modelled hierarchy with explicit path-based addressing. Like Junos, SR OS uses a candidate-commit architecture, but its structural conventions differ significantly — most notably the pervasive `admin-state` pattern and the separation of physical ports from logical interfaces.

Insight: *Junos's dual representation creates a unique challenge: the compiler must produce semantically identical output whether emitting hierarchical or `set`-line format. This duality is modelled in the Vendor AST as a single tree that can be serialised in either form (§38), ensuring that `show configuration` and `show configuration | display set` remain consistent.*

- **Path-based configuration**: `/configure router bgp neighbor 10.0.0.1` — the leading `/configure` root and slash-separated context mirrors the YANG model tree.
- **Admin-state pattern**: interfaces, protocols, and services must be explicitly enabled with `admin-state enable`. Unlike Cisco's `no shutdown`, this is a positively-asserted attribute rather than a negation (§29).
- **Service-oriented structure**: L2/L3 VPN services live under `/configure service vpls` or `/configure service vprn`, separate from the base router.
- **Port–interface separation**: physical configuration under `/configure port`, logical interfaces under `/configure router interface` — the compiler must emit both and maintain the binding between them.

11.4 Arista EOS

Arista EOS [68] descends from Cisco IOS and shares much of its flat imperative command structure, which makes it tempting to treat EOS as an IOS-XE variant. Arista's CloudVision [69] platform adds a management layer, but the underlying device configuration model remains CLI-based. In practice, EOS has diverged enough — particularly in data-centre features — that the compiler must treat it as a distinct platform with its own transform module.

Insight: *Nokia's YANG-modelled MD-CLI means that every valid configuration path corresponds directly to a YANG schema node. This gives the compiler a machine-readable schema to validate against, but also means that every required attribute must be explicitly present — SR OS does not silently inherit defaults the way IOS-XE does for many parameters.*

- **IOS-like surface syntax**: most traditional routing commands (`router bgp`, `router ospf`, `interface Ethernet1`) follow IOS conventions, allowing significant transform rule reuse between the two platforms.
- **VXLAN and EVPN**: Arista's VXLAN configuration uses `interface Vxlan1` with `vxlan source-interface` and `vxlan vlan` mappings — structurally different from both NX-OS's `nv overlay` and Junos's EVPN model.

- **Multi-agent routing model**: enabling `service routing protocols model multi-agent` changes the routing stack fundamentally, enabling features like EVPN Type-5 that are unavailable in the legacy model. The compiler must emit this prerequisite when EVPN features are present.
- **MLAG**: Arista's MLAG implementation uses `mlag configuration` with a peer-link and domain-id — a construct with no direct equivalent on other platforms and therefore requires a platform-specific IR extension.

11.5 Cisco NX-OS

Cisco NX-OS [70] powers the Nexus data centre switch family and, while sharing the `no`-prefix negation model with IOS-XE, differs structurally in several important respects:

Insight: The surface similarity between EOS and IOS-XE is both a blessing and a trap. The compiler can share approximately 60% of transform rules between the two platforms, but the remaining 40% — particularly VXLAN/EVPN, MLAG, and the multi-agent routing model — requires completely separate transform logic. The capability model (§32) is essential for gating features that exist on EOS but not on IOS-XE, and vice versa.

- **Feature activation**: features must be explicitly enabled before use via `feature bgp`, `feature isis`, etc. The compiler must emit these prerequisite commands before any protocol configuration.
- **VDC and VRF scoping**: Virtual Device Contexts (VDCs) and VRFs create distinct configuration namespaces. The `vrf context` command enters a VRF sub-mode.
- **Address-family differences**: BGP address families use `address-family ipv4 unicast` (similar to IOS-XE) but EVPN configuration uses NX-OS-specific constructs such as `nv overlay evpn` and `fabric forwarding anycast-gateway-mac`.
- **Port-channel syntax**: differs from IOS-XE's `channel-group` with NX-OS using `channel-group N mode active` under the member interface.
- **Interface naming**: `Ethernet1/1` (slot/port) rather than `GigabitEthernet0/0/0` (type/slot/subslot/port).

11.6 Cisco IOS-XR

Cisco IOS-XR [71] (used on ASR 9000, NCS 5500, and 8000 series) uses a *candidate-commit* model similar in philosophy to Junos, making it structurally distinct from IOS-XE despite sharing the Cisco brand:

- **Candidate-commit architecture**: configuration changes are staged and committed atomically via `commit`, with rollback support. This is fundamentally different from IOS-XE's immediate-apply model.
- **Explicit interface naming**: `interface GigabitEthernet0/0/0/0` uses a four-part naming scheme (rack/slot/module/port).
- **Path-like nesting**: configuration uses a deeper hierarchy than IOS-XE; e.g. `router bgp 65000 / neighbor 10.0.0.1 / address-family ipv4 unicast` (with explicit path separators in some modes).
- **Route-policy language**: uses a distinct structured policy language with `if/then/else/elseif` constructs, quite different from IOS-XE's flat route-maps or Junos's policy-statements.
- **IS-IS under router**: `router isis CORE` with interface association via `interface GigabitEthernet0/0/0/0` sub-blocks under the isis stanza (address-family scoped), rather than under the interface itself.

Insight: IOS-XR's candidate-commit model means the compiler can treat it more like Junos for diff operations (computing a target configuration and committing the delta), while its syntax remains broadly Cisco-like. This hybrid character illustrates why the compiler cannot simply classify vendors as "Cisco-like" or "Junos-like" — each requires its own transform functor.

Design Rule 11: Structural Isolation

Vendor-specific structural conventions (flat vs hierarchical, mode nesting depth, naming conventions) must not influence the IR schema. The IR represents logical configuration concepts; the AST represents vendor structure.

Cross-vendor patterns. Despite the structural diversity, two broad families emerge. Cisco and Arista use an *imperative* model: configuration lines are applied immediately, and negation uses a `no` prefix. Junos and Nokia use a *declarative* model: configuration is staged and committed atomically, with negation expressed through path-based deletion (`delete` in Junos, `no` or path removal in Nokia). IOS-XR sits between these families, combining Cisco-like syntax with a Junos-like candidate-commit workflow. The compiler cannot exploit these similarities naïvely — each vendor still requires its own transform functor — but recognising the pattern families aids in organising transform modules and reusing test infrastructure.

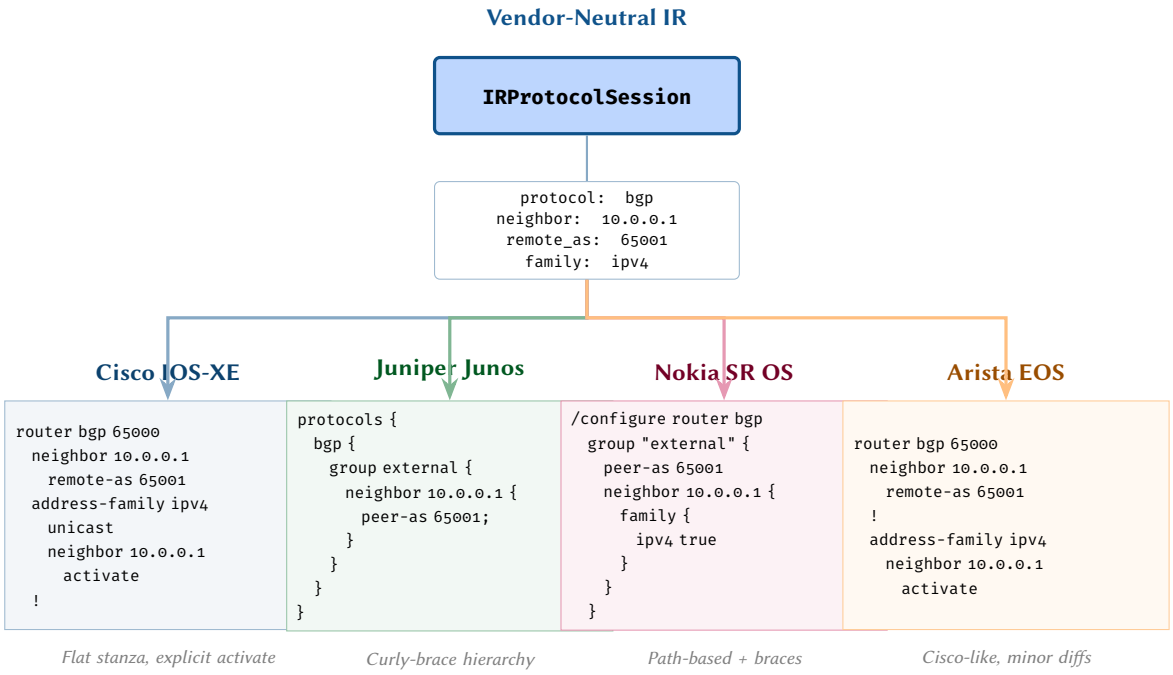


Figure 11. The same BGP session (neighbor `10.0.0.1`, AS 65001) rendered by four vendor emitters. The IR representation (top) is vendor-neutral; each emitter produces idiomatic configuration syntax for its target platform.

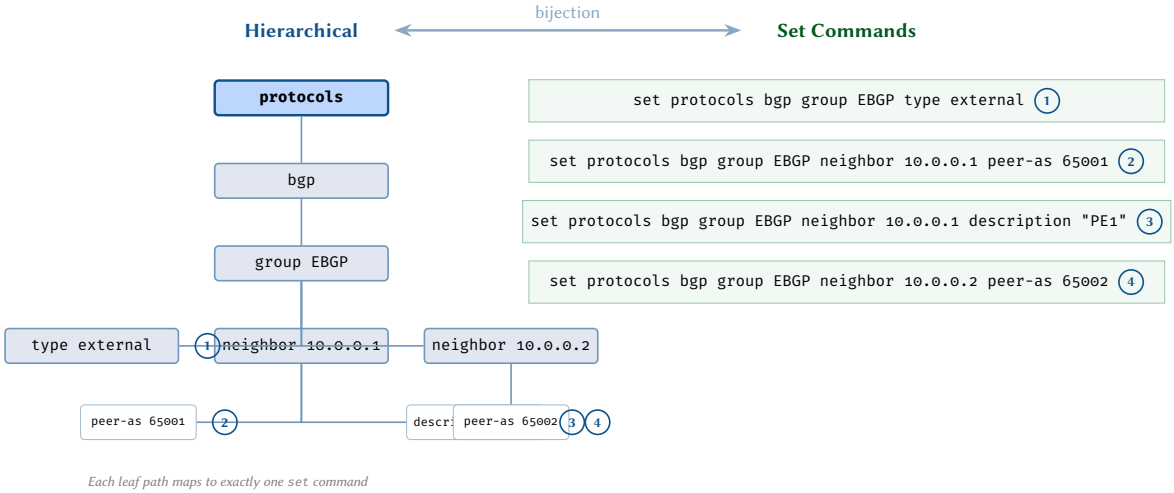


Figure 12. Junos configuration duality: the hierarchical tree representation (left) and the equivalent flat set-line representation (right) are isomorphic. Numbered annotations show the correspondence — each leaf path in the hierarchy maps to exactly one `set` command.

12 Category-Theoretic View

We now formalise the transform system using category theory [51, 52]. This formalisation provides a framework for reasoning about the correctness and composability of transforms.

12.1 A Brief Introduction to Category Theory for Engineers

Category theory is sometimes called "the mathematics of mathematics" — it provides a language for describing structure-preserving transformations between different representations. For a software engineer, the key concepts map naturally to familiar programming ideas:

- A **category** is a collection of *objects* and *morphisms* (arrows) between them, with a composition rule. Think of objects as data types and morphisms as functions between those types. The key rule is that morphisms compose associatively: if $f : A \to B$ and $g : B \to C$, then $g \circ f : A \to C$ exists.
- A **functor** is a structure-preserving map *between* categories. It maps objects to objects and morphisms to morphisms, preserving composition. In our context, a vendor transform F_v is a functor: it maps IR trees to vendor ASTs and IR tree operations to AST operations, preserving the structure of the relationships. In programming terms, a functor is like a type-safe adapter that translates an entire API from one domain to another.

- A **natural transformation** is a systematic way to convert between two functors. If F and G are two different ways to translate IR trees to ASTs, a natural transformation $\eta : F \Rightarrow G$ provides a consistent mapping between their outputs. This formalises the notion that "two vendors produce different ASTs but they mean the same thing."
- A **commuting diagram** is a diagram where all paths between the same two nodes give the same result. This is the categorical way of expressing correctness: it does not matter which route you take through the compilation pipeline — the final network behaviour must be the same.

Why Category Theory?

We use this formalism not for academic ornamentation but because it provides three concrete benefits:

1. **Correctness guarantees.** The commuting diagram property gives us a precise statement of what it means for the compiler to be correct: all vendor paths must produce semantically equivalent network state.
2. **Composability reasoning.** Functors compose: if $F : A \to B$ and $G : B \to C$, then $G \circ F : A \to C$ is also a functor. This means we can build complex transforms from simple ones and know the composition preserves correctness.
3. **Refactoring safety.** Natural transformations tell us when two different implementations are interchangeable, enabling safe optimisation and refactoring of transform code.

12.2 Formalisation

Let **IR** be the category whose objects are typed IR trees and whose morphisms are structure-preserving maps (tree homomorphisms that respect types and attributes). For each vendor v, let $\mathbf{AST}_v$ be the category of vendor ASTs with analogous morphisms.

The vendor transform $F_v : \mathbf{IR} \to \mathbf{AST}_v$ is a functor: it maps each IR tree to a vendor-specific AST, and it maps each IR tree operation (e.g., adding a neighbour, changing a metric) to the corresponding AST operation. The key property we require is that transforms *commute* with the rendering functors:

$$R_v \circ F_v = R_{v'} \circ F_{v'} \quad \text{(up to semantic equivalence)} \tag{10}$$

This commutativity expresses the fundamental correctness criterion: regardless of which vendor transform we apply, the resulting configuration, when deployed, produces the same network behaviour.

More precisely, let **NetState** be the category of network states (as defined in [8]). The *deployment functor* $D_v : \mathbf{Config}_v \to \mathbf{NetState}$ maps a vendor-specific configuration to the network state it produces. The correctness criterion is:

$$D_{v_1} \circ R_{v_1} \circ F_{v_1} = D_{v_2} \circ R_{v_2} \circ F_{v_2} \tag{11}$$

for all vendor pairs (v_1, v_2).

12.3 Correctness Theorems

The category-theoretic framework is not merely descriptive — combined with the design rules established in earlier sections, it enables us to state and prove key correctness properties of the compiler. These theorems formalise the intuition that a well-formed compiler produces deterministic, semantically faithful output.

Theorem 1 (Compilation Determinism). *If section 9 (Transform Completeness) and section 9 (Rule Determinism) hold, then for any* **IR** *tree I and vendor v, the compilation $F_v(I)$ is uniquely determined.*

Proof. By structural induction on the **IR** tree I.

Base case. Let I be a leaf node. By section 9, at least one transformation rule in F_v is applicable to the leaf. By section 9, at most one rule may fire, so the output $\mathbf{AST}_v$ fragment is uniquely determined.

Inductive step. Suppose the property holds for every proper subtree of a node n in I. The children of n each produce a unique $\mathbf{AST}_v$ fragment by the induction hypothesis. section 9 guarantees that a rule exists for the node n itself, and section 9 ensures that this rule produces a unique output when composed with the already-determined children outputs.

By induction, $F_v(I)$ is uniquely determined for the entire tree. □

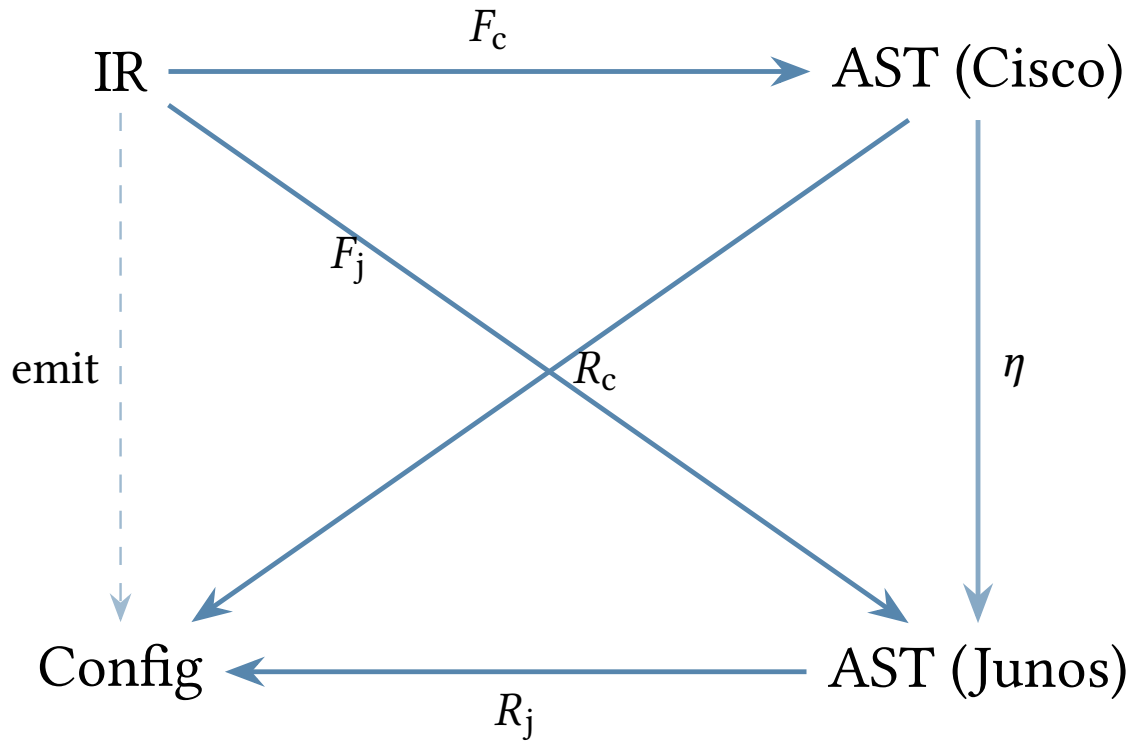


Figure 13. Commuting diagram for the compilation pipeline. The functors F_v map the device-independent IR to vendor-specific ASTs, and the render morphisms produce configuration text. The diagram commutes: $R_v \circ F_v = \text{emit}_v$ for every vendor v.

Theorem 2 (Semantic Preservation). *If the commuting diagram property holds, i.e.,*

$$D_{v_1} \circ R_{v_1} \circ F_{v_1} \;=\; D_{v_2} \circ R_{v_2} \circ F_{v_2}$$

for all vendor pairs (v_1, v_2), then for any **IR** *tree I, the network state produced by deploying the compiled configuration is vendor-independent.*

Proof. Let I be an arbitrary **IR** tree and let v_1, v_2 be any two vendors. The compilation pipeline for vendor v is the composition $D_v \circ R_v \circ F_v$, where F_v is the vendor transform functor mapping **IR** to $\mathbf{AST}_v$, R_v is the render morphism mapping $\mathbf{AST}_v$ to concrete configuration text, and D_v is the deployment functor mapping configuration text to operational network state.

By the commuting diagram property,

$$D_{v_1}(R_{v_1}(F_{v_1}(I))) \;=\; D_{v_2}(R_{v_2}(F_{v_2}(I))),$$

so the operational state is identical regardless of which vendor path is taken. Since v_1 and v_2 were arbitrary, the resulting network state is vendor-independent. □

Lemma 1 (Projection Locality). *The projection π_d depends only on the k-hop neighbourhood of d in G_{plan} for some bounded k (typically $k \leq 2$).*

Proof. A device d's **IR** attributes are constructed from three sources:

1. The vertex attributes of d itself (hop 0): hostname, platform, loopback addresses, and other device-level properties.
2. The edges incident to d (hop 1): interface bindings, link-level parameters, and the set of protocols enabled on each connection.
3. The vertex attributes of vertices adjacent to d (hop 2): peer information such as remote AS numbers, remote loopback addresses used for BGP peering, and OSPF area assignments of neighbouring routers.

No attribute in the **IR** tree for device d depends on vertices or edges at distance greater than 2. Hence π_d is determined by the k-hop neighbourhood of d with $k \leq 2$. □

Corollary 1 (Incremental Recompilation). *If the plan topology changes outside the k-hop neighbourhood of device d, then $\pi_d(G'_{\text{plan}}) = \pi_d(G_{\text{plan}})$ and recompilation of d is unnecessary.*

Proof. Let G'_{plan} be the modified plan topology and suppose all changes are at distance greater than k from d. By lemma 1, π_d depends only on the k-hop neighbourhood of d. Since this neighbourhood is identical in G_{plan} and G'_{plan}, we have $\pi_d(G'_{\text{plan}}) = \pi_d(G_{\text{plan}})$. The input to F_v is unchanged, so recompilation of d produces the same output and may be skipped. □

Definition 5: Projection Idempotence

The device projection operator π_d is *idempotent*: for any plan topology G_{plan},

$$\pi_d\big(\pi_d(G_{\text{plan}})\big) \;=\; \pi_d(G_{\text{plan}}).$$

That is, applying the projection a second time yields the same **IR** tree. The device's **IR** tree is therefore a fixed point of π_d.

13 Composability and Modular Transforms

A monolithic transform function for each vendor would be unmaintainable. Instead, we decompose transforms into *protocol-specific modules* that can be composed.

Each module handles one protocol or feature area:

- `transform_interfaces` — physical and logical interfaces
- `transform_isis` — IS-IS protocol configuration
- `transform_ospf` — OSPF protocol configuration
- `transform_bgp` — BGP sessions and address families
- `transform_mpls` — MPLS, LDP, and SR configuration
- `transform_evpn` — EVPN service configuration
- `transform_policy` — routing policies and route-maps
- `transform_acl` — access control lists
- `transform_qos` — QoS classifiers and schedulers

Modules are registered in the transform registry and invoked based on the IR node types they handle. A module may depend on the output of another module (e.g., the BGP transform may reference policy AST nodes produced by the policy transform), respecting the dependency ordering from section 7.

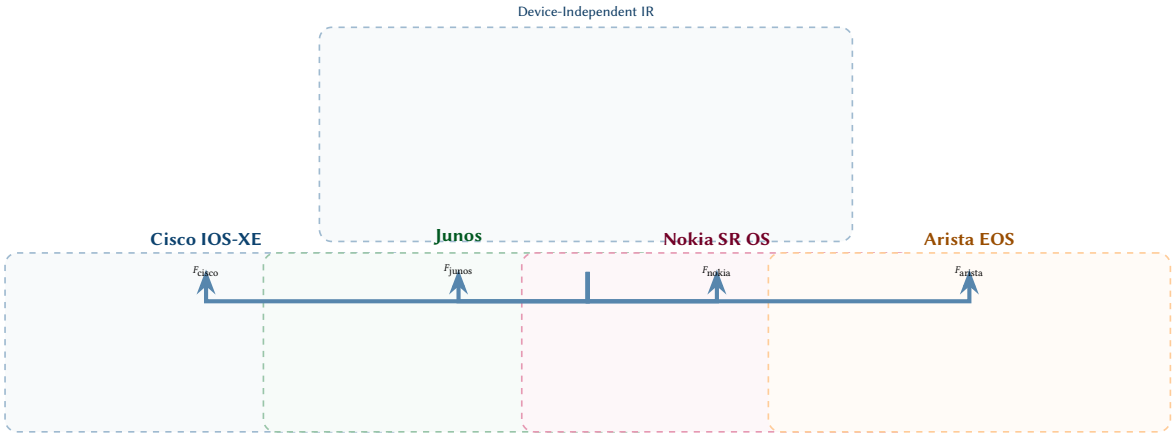


Figure 14. Multi-vendor fan-out: a single device-independent IR is lowered by vendor-specific functors F_v into four distinct AST fragments. Each AST reflects the idiomatic structure of the target NOS.

14 Case Study: Interface Transformation

To make the transform system concrete, we now present detailed case studies for the major IR node types. Each case study shows the IR input, the structural differences across vendors, and the resulting AST subtrees.

An interface is the most attribute-dense IR node type. A single `IRInterface` carries L1 physical attributes (speed, duplex, MTU), L2 switching attributes (VLAN, trunk, LAG), L3 addressing (IPv4, IPv6), and protocol-specific attributes (IS-IS metric, OSPF cost, MPLS enable). The transform must distribute these attributes across vendor-specific structural locations that differ significantly.

Structural Divergence

On Cisco IOS-XE, all interface attributes live under a single `interface GigabitEthernet0/0/0` block. On Junos, they are split across `interfaces ge-0/0/0` (physical), `unit 0` (logical), `family inet` (addressing), and separate `protocols isis interface` blocks. On Nokia SR OS, the physical port configuration (`/configure port 1/1/1`) is entirely separate from the router interface (`/configure router interface "to-pe02"`).

Consider a core-facing interface with the following IR attributes:

Insight: *Interface transformation is where the "one IR node to many AST nodes" pattern is most pronounced. A single `IRInterface` may expand into 3–5 AST subtrees on some vendors.*

Attribute	Value
name	ge-0/0/0
enabled	true
description	TO-PE02
mtu	9000
speed	10G
ipv4_address	10.0.0.1/30
isis_passive	false
isis_metric	100
mpls_enabled	true

fig. 15 shows how this single IR node expands into vendor-specific AST subtrees for each target platform. The key observation is that the *same* logical attribute lands in structurally different locations depending on the vendor.

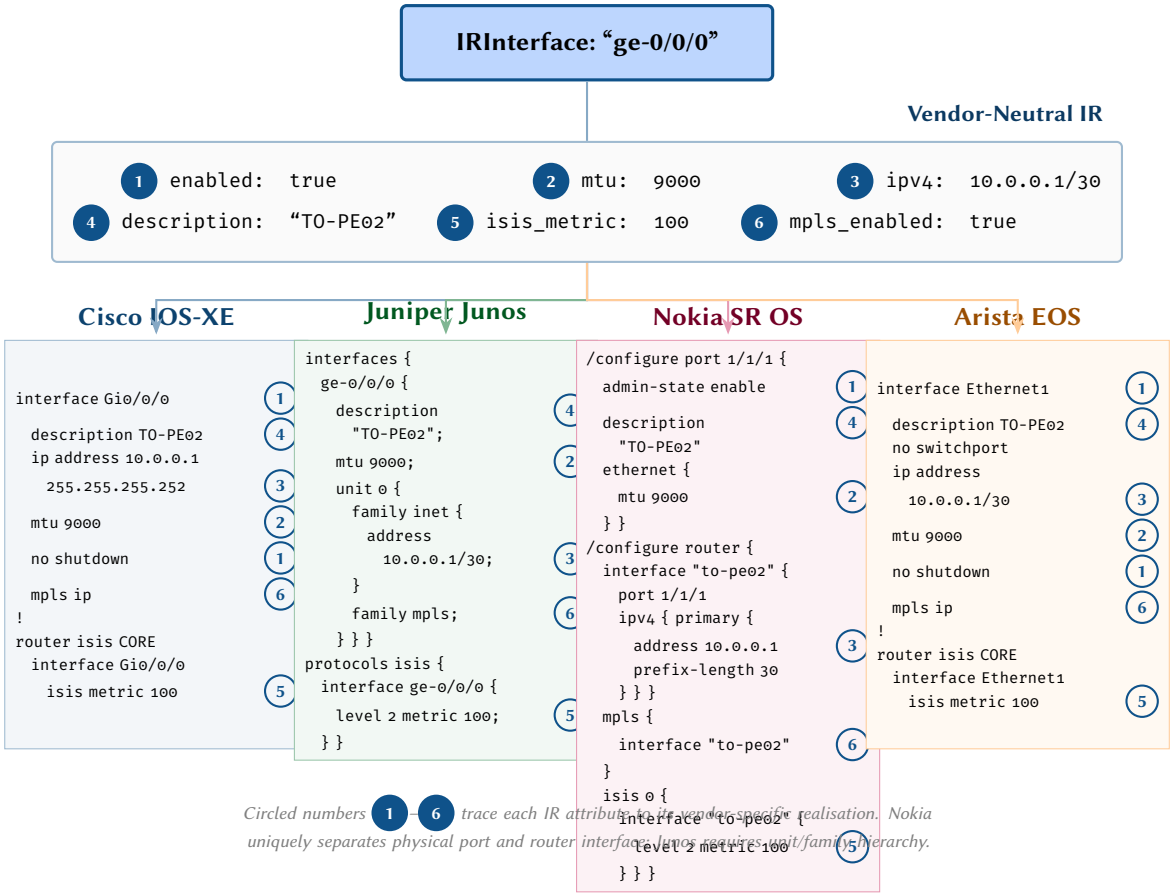
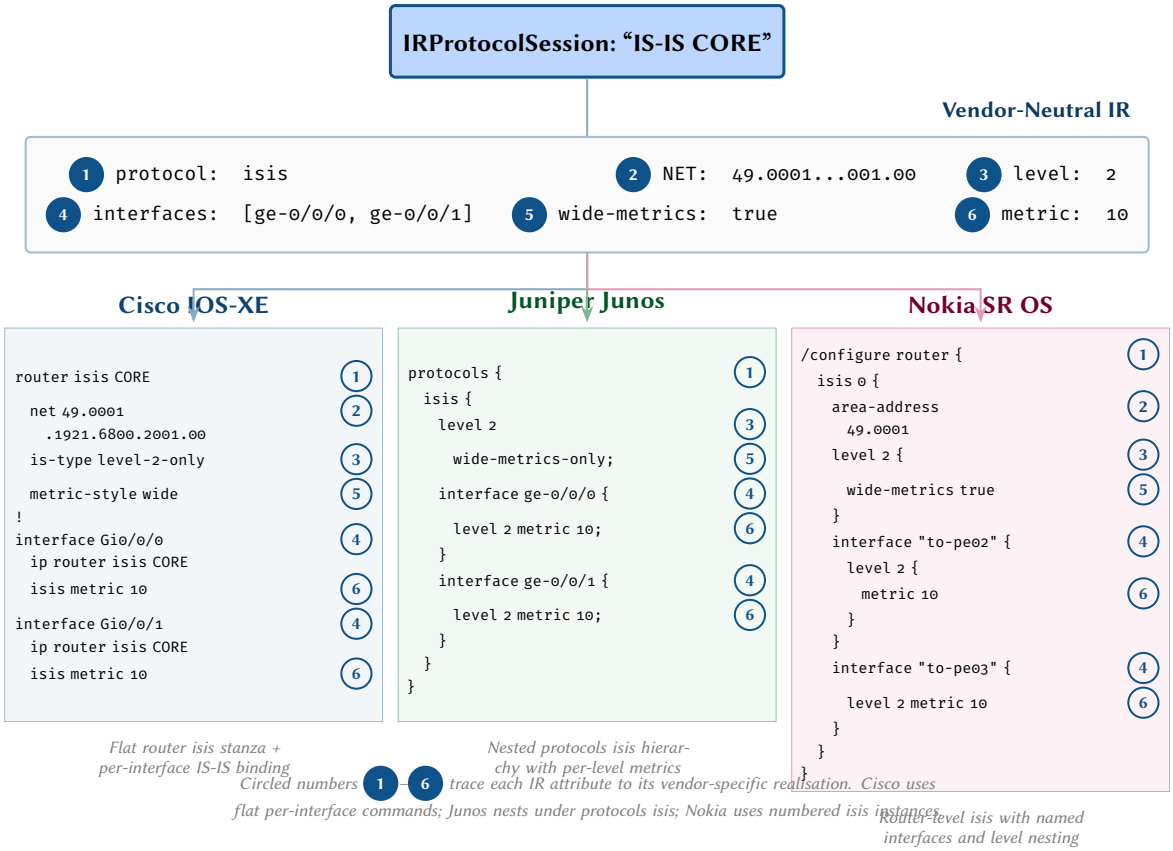


Figure 15. Detailed interface transformation. Six IR attributes are traced to their vendor-specific locations across four platforms. Nokia uniquely separates physical port and router-interface objects; Junos uses unit/family hierarchy; Cisco and Arista differ in interface naming and IP notation (subnet mask vs. CIDR prefix).

15 Case Study: IS-IS Transformation

IS-IS [72] configuration varies significantly across vendors in both structure and terminology. The IR captures the protocol-level intent: the NET address, the IS-IS level, the set of participating interfaces, and per-interface metrics. The transform must produce vendor-specific representations:

- Cisco: `router isis CORE` with `net` and `is-type` commands at the router level, and `isis enable CORE` plus `isis metric` commands under each interface.
- Junos: `protocols isis` with interfaces listed as children, each carrying `level 2 metric` settings. The NET is derived from the `lo0` interface.
- Nokia: `/configure router isis 0` with interfaces as `/configure router isis 0 interface "name"` children. Uses `admin-state enable` explicitly.
- Arista: `router isis CORE` similar to Cisco, with `isis enable` on interfaces but using Arista's `Ethernet` naming convention.
- NX-OS: requires `feature isis` before any IS-IS configuration. Uses `router isis CORE` with `is-type level-2` and `isis circuit-type level-2` under interfaces. Interface naming: `Ethernet1/1`.
- IOS-XR: `router isis CORE` with interfaces declared as sub-blocks under the router stanza (not under the interface itself), scoped by address family: `interface GigabitEthernet0/0/0/0 / address-family ipv4 unicast / metric 100`.



Insight: *IS-IS is one of the simpler protocol transforms because the logical model is similar across vendors. The differences are mostly syntactic (command names, nesting depth) rather than structural — though IOS-XR's model of declaring interfaces under the router stanza (rather than enabling the protocol under the interface) is a notable structural inversion.*

Figure 16. IS-IS protocol transformation. Six IR attributes are traced to their vendor-specific locations. Cisco uses a flat `router isis` stanza with per-interface sub-commands; Junos nests interfaces under a `protocols isis` hierarchy with per-level metric configuration; Nokia uses numbered IS-IS instances with named interface references.

16 Case Study: ACL Transformation

Access Control List (ACL) transformation illustrates several challenges unique to filtering constructs:

1. **Syntax variation**: Cisco uses positional match fields (`permit ip 10.0.0.0 0.255.255.255 any`), while Junos uses named match clauses (`from source-address 10.0.0.0/8`).
2. **Wildcard vs prefix**: Cisco uses inverse wildcard masks; all other vendors use CIDR prefix notation.
3. **Implicit deny**: Cisco appends an implicit `deny any` at the end of every ACL. Junos and Nokia require this to be explicit if desired.
4. **Term naming**: Junos requires each ACL entry to have a named `term`; Cisco uses sequence numbers; Nokia uses numeric entry IDs.

Wildcard Mask Conversion

The most common source of ACL-related errors is incorrect wildcard mask conversion. The IR stores all addresses as CIDR prefixes (e.g., `10.0.0.0/8`). The Cisco transform must convert this to wildcard

notation (`10.0.0.0 0.255.255.255`). A /24 prefix becomes `0.0.0.255`, not `255.255.255.0`. The compiler must invert the mask bits, not copy the subnet mask.

Consider an IR ACL with three entries:

Seq	Action	Source	Dest
10	permit	10.0.0.0/8	any
20	deny	192.168.0.0/16	any
30	permit	any	any

fig. 17 shows the complete transformation of this ACL across three vendors, illustrating the structural and syntactic divergence.

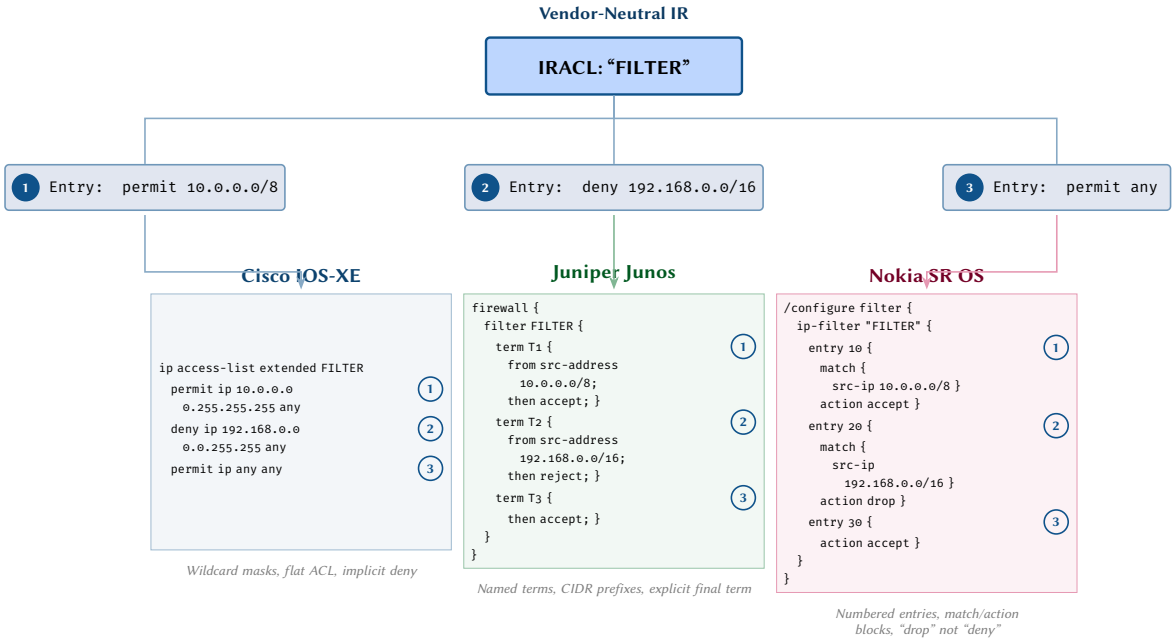


Figure 17. ACL transformation. Three IR filter entries map to structurally different vendor syntaxes: Cisco uses wildcard masks in a flat access-list, Junos uses named terms within a firewall filter hierarchy, and Nokia uses numbered entries with explicit match/action blocks.

17 Case Study: Routing Policy Transformation

Routing policy transformation is among the most complex transforms because the policy abstraction models differ fundamentally across vendors.

Insight: *Cisco's route-maps are* ordered match-action lists; *Junos's policy-statements are* ordered term lists with from/then clauses; *Nokia's policies use* numbered entries with match/action blocks. *Same concept, three different models.*

- Cisco: Route-maps with numbered sequence entries, each referencing external match objects (prefix-lists, community-lists, as-path access-lists) by name. The match objects must be defined separately before the route-map.
- Junos: Policy-statements with named terms. Match criteria can be inline (`from community CUST`) or reference external objects. Supports `next term` and `next policy` flow control.
- Nokia: Policy-statements with numbered entries. Match criteria reference separately defined prefix-lists and community-lists. Supports `default-action` for the implicit final entry.
- Arista: Route-maps similar to Cisco, with some additional match clauses (e.g., `match extcommunity`).

A critical implication is that a single IR `IRPolicy` node may require the compiler to emit *multiple* top-level configuration objects: the route-map itself *plus* the prefix-lists and community-lists it references. This is a "one-to-many" transform that must respect the dependency ordering: referenced objects before referencing objects.

Design Decision 1: Policy Decomposition

When the IR contains an `IRPolicy` with inline match criteria (e.g., "match prefix 10.0.0.0/8"), the Cisco transform must: (1) synthesise a prefix-list name, (2) emit the prefix-list definition, and (3) emit the

route-map referencing that prefix-list. The Junos transform can use inline `from route-filter` clauses, avoiding the decomposition.

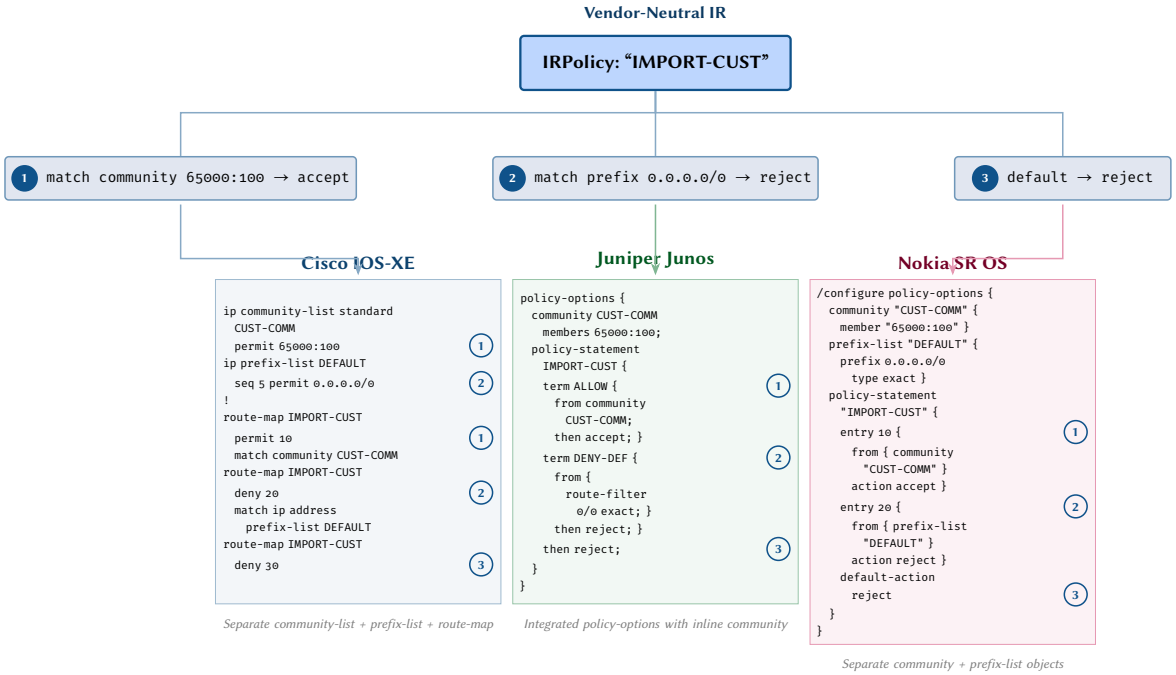


Figure 18. Routing policy transformation. Three IR policy terms map to structurally different constructs: Cisco requires separate community-list, prefix-list, and route-map objects; Junos integrates communities and terms within a single policy-statement; Nokia uses numbered entries with references to separately defined community and prefix-list objects.

18 Case Study: VRF and L3VPN Transformation

Virtual Routing and Forwarding (VRF) configuration illustrates the challenge of *service-oriented* versus *protocol-oriented* configuration models.

On Cisco IOS-XE, a VRF is defined as a top-level object (`vrf definition CUST-A`) with route distinguisher and route targets, and interfaces are bound to the VRF with `vrf forwarding CUST-A` under the interface configuration. The BGP VPNv4/VPNv6 address family configuration is separate under `router bgp`.

Junos uses `routing-instances` as the primary container, placing the VRF's interfaces, route distinguisher, route targets, and protocol sessions all under a single `routing-instances CUST-A` hierarchy. This is structurally cleaner but means the BGP configuration for a VRF is separate from the global BGP configuration.

Nokia SR OS uses a `service vprn` model where the VRF is a *service*, reflecting Nokia's service-oriented architecture. The VPRN service contains its own interface and BGP configuration, plus service distribution parameters.

Insight: *The VRF transform reveals a deeper architectural difference: Cisco and Arista treat VRFs as routing constructs; Nokia treats them as services. The IR must abstract over this distinction.*

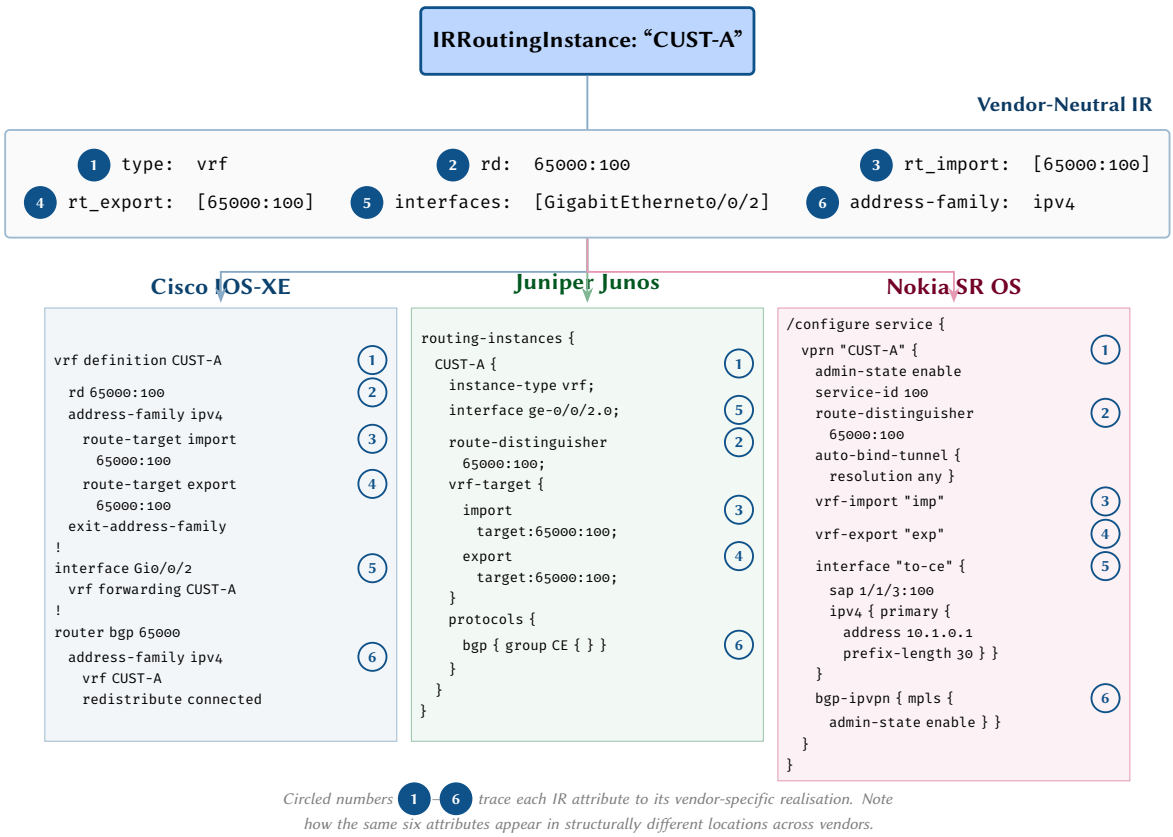


Figure 19. VRF/L3VPN transformation. A single IR routing instance (top) carries six key attributes. Circled numbers trace each attribute to its vendor-specific location: Cisco separates VRF definition, interface binding, and BGP address-family; Junos integrates everything within `routing-instances`; Nokia models the VRF as a `vprn` service with separate import/export policies.

19 Case Study: EVPN Transformation

EVPN configuration is the most vendor-divergent feature in modern service provider networks. The same EVPN service (Layer 2 VXLAN with a single EVI) requires fundamentally different configuration structures across vendors:

- Cisco: EVPN is configured under `l2vpn evpn` for the EVI definition, `router bgp` for the BGP EVPN address family, and a `bridge-domain` for the Layer 2 forwarding. The VNI-to-EVI binding is under `l2vpn evpn evi NNN`.
- Junos: EVPN is configured as a `routing-instances` entry of type `evpn`, containing `protocols evpn`, `encapsulation vxlan`, `vni NNN`, and a `vtep-source-interface` reference. The BGP session for EVPN signalling is in the global `protocols bgp` with `family evpn signaling`.
- Nokia: EVPN uses the service model: `/configure service vpls “SVC”` with a `bgp 1 / bgp-evpn` sub-tree, and `vxlan instance 1 vni NNN`. The BGP signalling is under `/configure router bgp group` with `family evpn`.

> **EVPN Complexity**
>
> A single IR `IREvpn` node may expand into 5–8 AST subtrees on some vendors, scattered across multiple top-level configuration sections. The dependency ordering is critical: the VNI must exist before the bridge-domain can reference it, and the BGP address family must be configured before EVPN route targets can be applied.

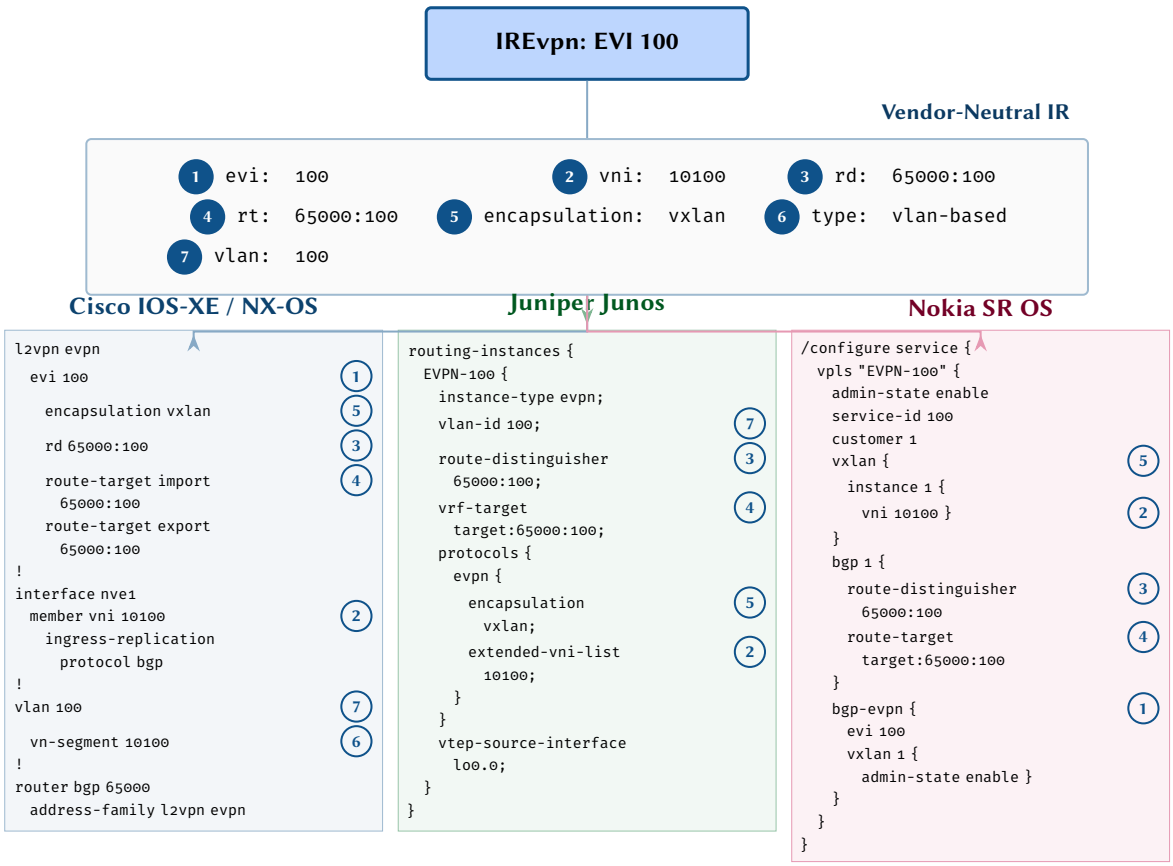


Figure 20. EVPN service transformation. Seven IR attributes map to structurally different locations across vendors. Cisco distributes EVPN configuration across `l2vpn evpn`, NVE interface, VLAN, and BGP stanzas; Junos integrates within a single routing-instance; Nokia models it as a VPLS service with nested `bgp-evpn` and `vxlan` objects.

20 Case Study: QoS Transformation

Quality of Service (QoS) is arguably the most vendor-specific feature area in network configuration. Each vendor has a completely different QoS architecture:

- Cisco: MQC (Modular QoS CLI) with separate `class-map`, `policy-map`, and `service-policy` objects. Classification, marking, queuing, and shaping are composed in policy-maps.
- Junos: CoS (Class of Service) with `forwarding-classes`, `classifiers`, `rewrite-rules`, and `scheduler-maps` as independent objects that are composed at the interface level.
- Nokia: SAP (Service Access Point) ingress/egress policies with `queue` definitions, `fc` (forwarding class) mappings, and `policer` entries. QoS is tightly integrated with the service model.
- Arista: Similar to Cisco MQC but with additional features like `traffic-policies` for stateless ACL-based QoS.

The IR `IRQoS` node captures the classification intent (match DSCP EF, match DSCP AF11) and the queuing behaviour (priority queue, bandwidth guarantee). The vendor transforms must map this to each platform's QoS architecture.

Insight:
QoS is where the "same intent, completely different structure" challenge is most extreme. The IR must be careful to capture what *the QoS policy achieves, not* how *any particular vendor implements it.*

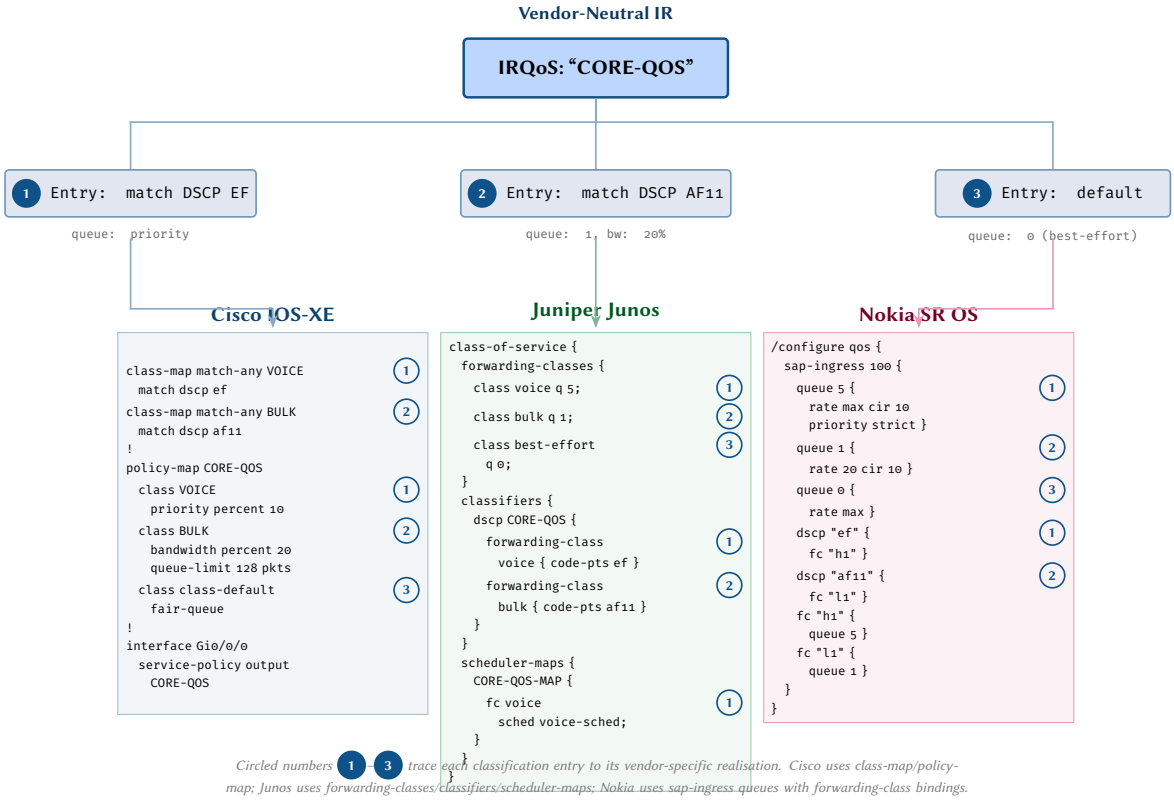


Figure 21. QoS policy transformation. Three classification entries (EF voice, AF11 bulk, default best-effort) from the IR map to fundamentally different QoS models: Cisco's MQC class-map/policy-map, Junos's forwarding-classes and scheduler-maps trinity, and Nokia's sap-ingress queues with forwarding-class bindings.

21 Case Study: BGP Session

BGP [73] session configuration is among the most attribute-rich transforms in the compiler. A single iBGP EVPN [74, 75] session carries at least seven distinct attributes — AS number, router-id, neighbour address, remote AS, update source, address-family, and BFD — each of which lands at a different structural location depending on the vendor. Cisco places the AS number on the `router bgp` line and requires explicit `address-family` activation; Junos splits the AS and router-id into a separate `routing-options` block and uses group-based neighbour configuration; Nokia places AS and router-id at the `/configure router` level with named peer groups.

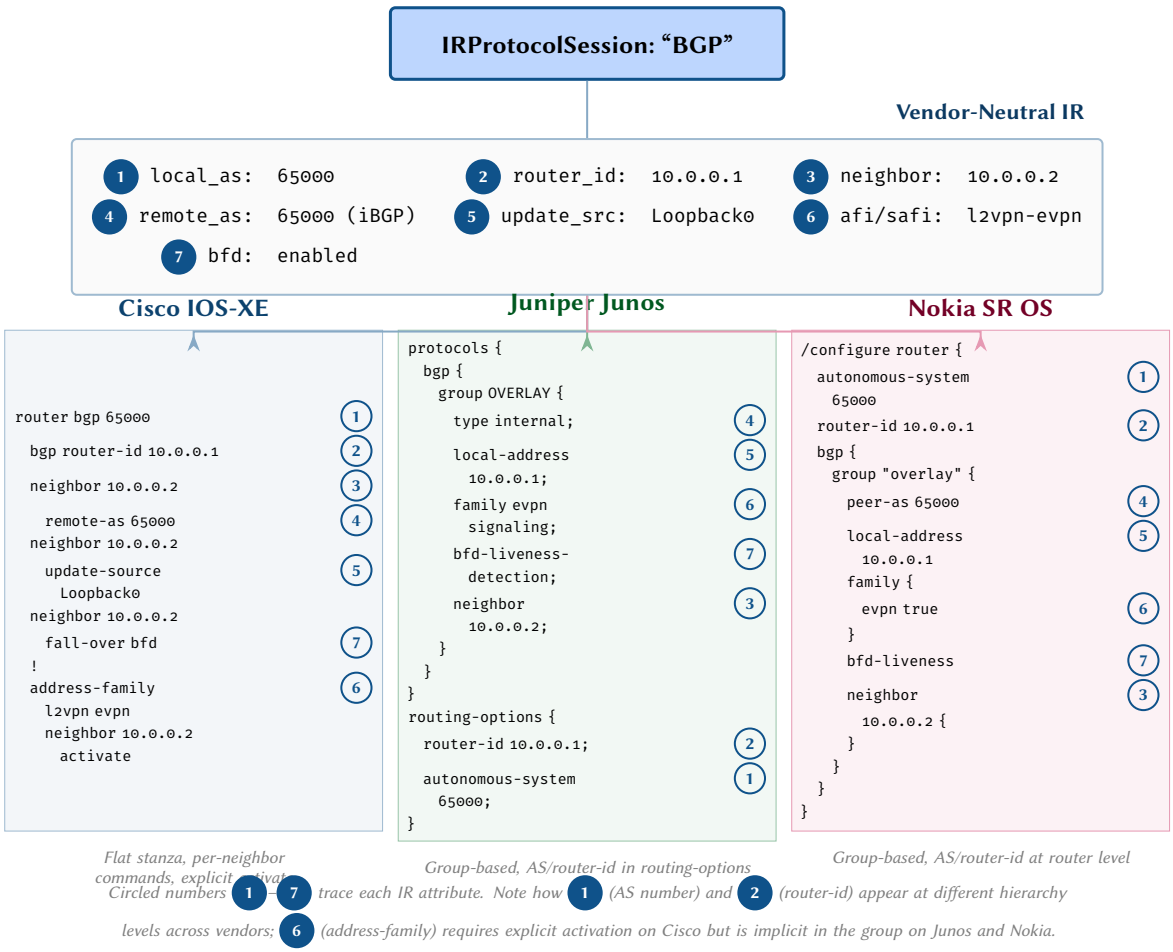


Figure 22. BGP session transformation. Seven IR attributes are traced to their vendor-specific locations. Cisco uses per-neighbor flat commands with explicit address-family activation; Junos places AS and router-id under `routing-options` and uses group-based neighbor configuration; Nokia places AS/router-id at the router level and uses named groups.

22 Case Study: MPLS/LDP

MPLS [76] with LDP signalling demonstrates a common pattern in network configuration: a single protocol concept that different vendors split across multiple structural blocks. The IR captures five attributes — protocol type, router-id, interface list, transport address, and label range — as a flat attribute set. Cisco enables MPLS per-interface with the `mpls ip` command and configures LDP globally; Junos requires *both* a `protocols ldp` block and a separate `protocols mpls` block, each with its own interface list; Nokia similarly splits configuration across `ldp` and `mpls` contexts but adds explicit admin-state enablement per interface.

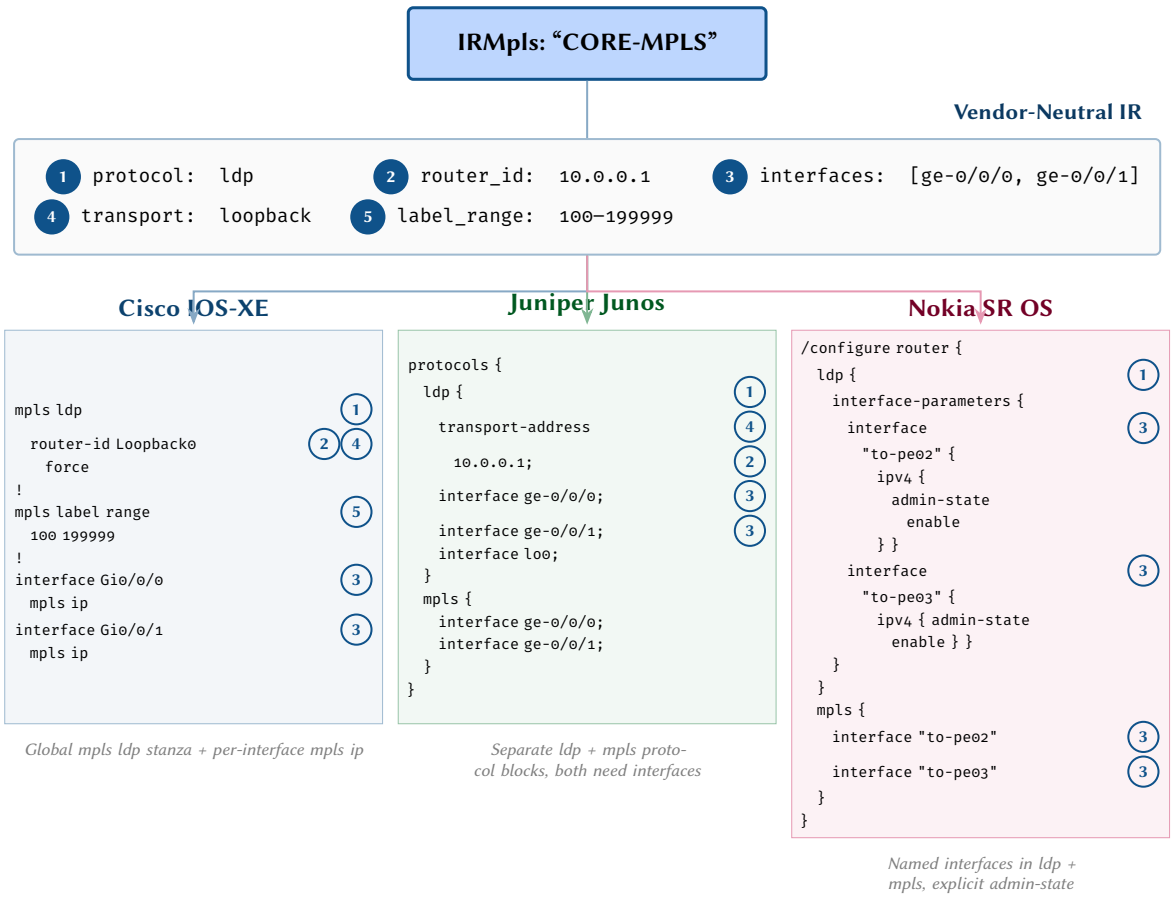


Figure 23. MPLS/LDP transformation. Five IR attributes are traced to their vendor-specific locations. Cisco enables MPLS per-interface with `mpls ip`; Junos requires both `protocols ldp` and `protocols mpls` blocks with interface lists in each; Nokia uses named interfaces under both `ldp` and `mpls` with explicit admin-state enablement.

23 Case Study: Static Routes

Static routes provide a compact illustration of how the same routing intent maps to fundamentally different syntactic conventions. A default route, a blackhole aggregate, and a floating static each exercise a different aspect of the transform: Cisco converts CIDR prefix lengths to wildcard masks and uses the `Null0` interface for blackhole routes, with administrative distance as a trailing integer; Junos preserves CIDR notation, uses the `discard` keyword, and spells out `preference` explicitly; Nokia requires `admin-state enable` on each next-hop entry and uses the `blackhole` keyword directly.

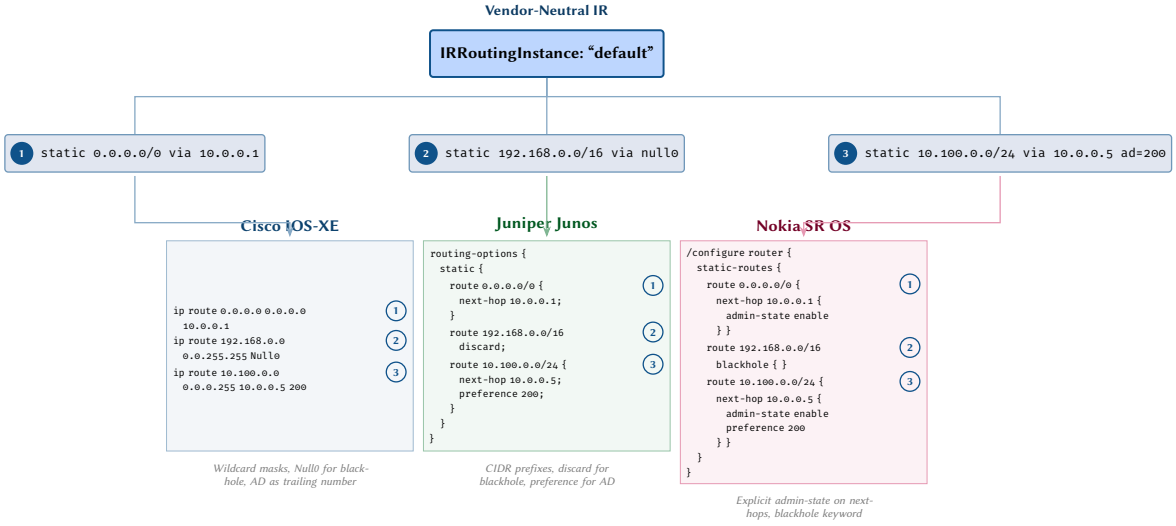


Figure 24. Static route transformation. Three IR static route entries map to structurally different vendor syntaxes. Cisco uses wildcard masks and `Null0` for blackhole routes; Junos uses CIDR prefixes, `discard`, and `preference` for administrative distance; Nokia requires explicit `admin-state enable` on each next-hop and uses the `blackhole` keyword.

24 Case Study: OSPF

OSPF [77] configuration highlights a structural split that recurs across vendors: some attributes are configured under the protocol stanza while others are configured under the interface. The IR captures seven attributes — process ID, router-id, area, interface list, network type, cost, and passive interfaces — as a flat set. Cisco places area membership, network type, and cost under each interface with `ip ospf` commands, but declares passive interfaces under the `router ospf` stanza; Junos nests interfaces under area blocks and treats passive as an interface-level attribute; Nokia uses named interfaces under each area with `passive true` as a boolean.

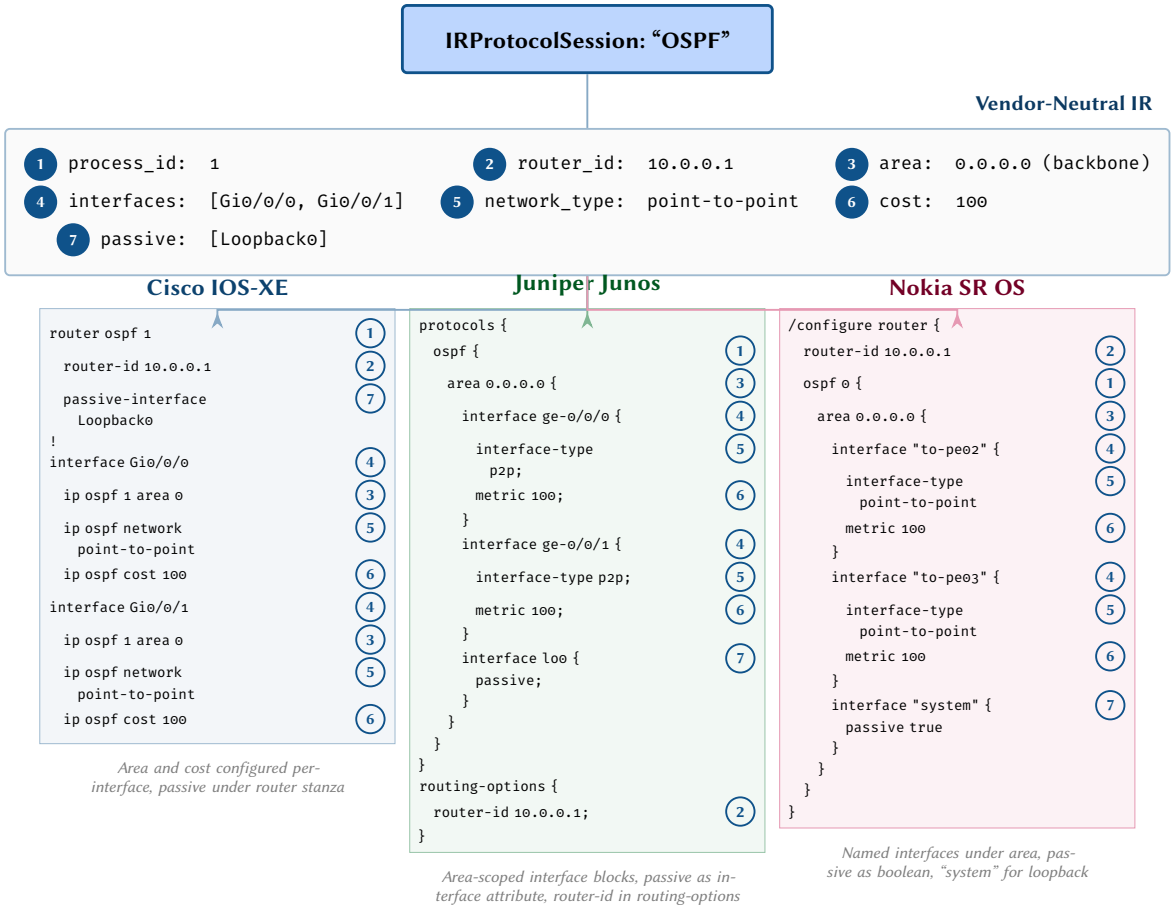


Figure 25. OSPF transformation. Seven IR attributes are traced to their vendor-specific locations. Cisco configures OSPF area membership and cost per-interface but declares passive interfaces under the router stanza; Junos nests interfaces under area blocks with passive as an interface attribute; Nokia uses named interfaces under area with `passive true` as a boolean.

25 Case Study: Route Policy

Route policy configuration is where vendor structural differences are most acute. A single IR policy with a prefix match, community match, and attribute-setting actions maps to fundamentally different constructs: Cisco uses separate prefix-list, community-list, and route-map objects with cross-references by name; Junos uses a self-contained policy-statement with named terms and inline match criteria; Nokia uses numbered entries with explicit action-type keywords. The handling of the implicit default action also diverges: Cisco route-maps implicitly deny unmatched prefixes, while Junos and Nokia require explicit `reject`/`action-type reject` statements.

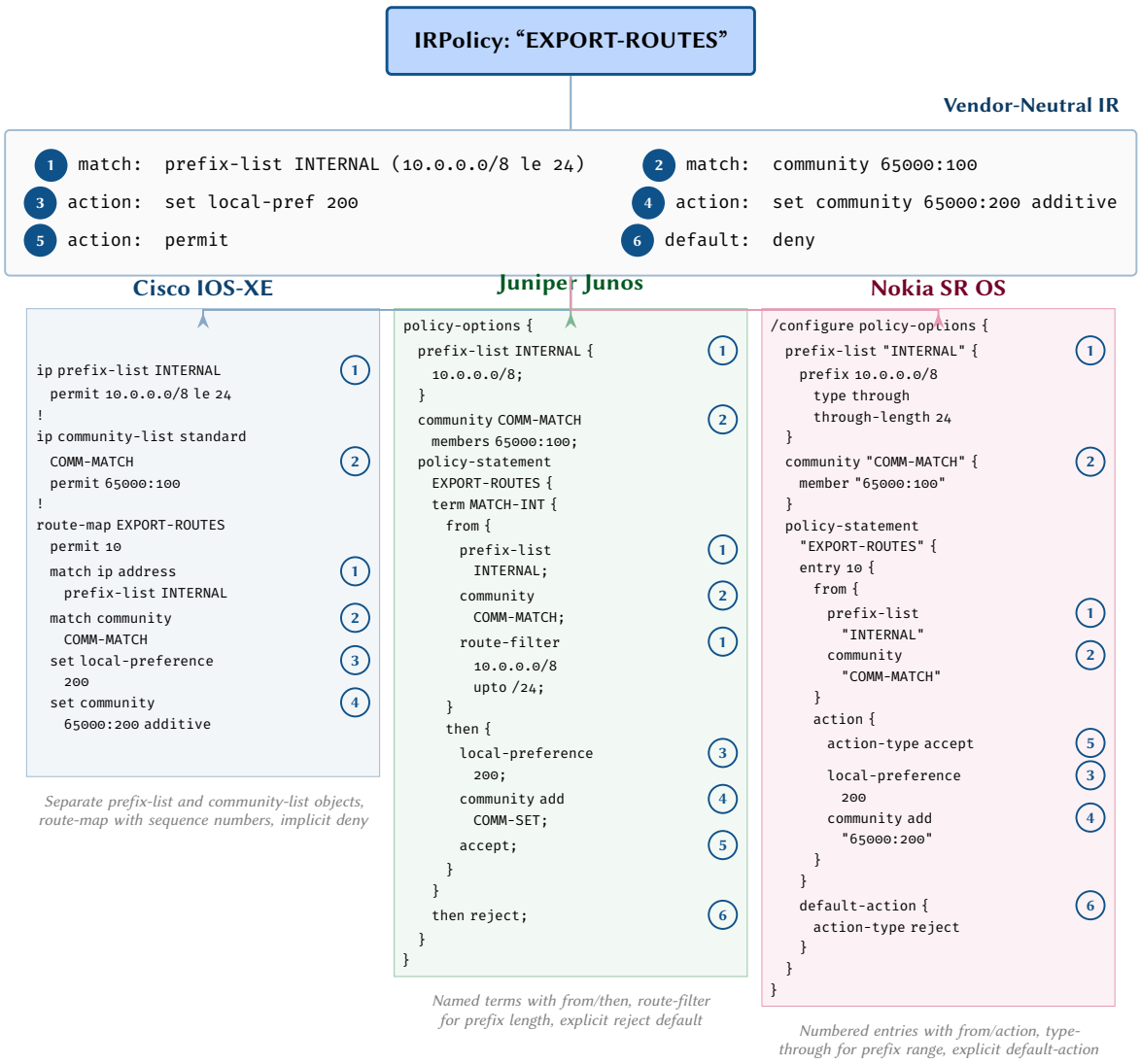


Figure 26. Route policy transformation. Six IR policy attributes are traced across three vendors. Cisco uses separate prefix-list and community-list objects referenced by a route-map with sequence numbers and an implicit final deny; Junos uses named terms within a policy-statement with explicit `accept`/`reject`; Nokia uses numbered entries with `action-type` and an explicit `default-action` block.

26 Case Study: VXLAN/EVPN Fabric

VXLAN with EVPN [75] control plane represents the most structurally complex transformation in the compiler. A single IR EVPN instance carries VNI-to-VLAN mappings, VTEP source interface, route distinguisher, route targets, and overlay features such as ARP suppression — and each vendor scatters these across entirely different configuration sections. NX-OS requires an `nv overlay evpn` prerequisite command and distributes state across VLAN definitions, an NVE tunnel interface, and a dedicated `evpn` stanza; Junos places VNI mappings under `vlans` with VTEP configuration in `switch-options`; Arista uses a single `interface Vxlan1` for all tunnel configuration with VLAN-scoped RD/RT under `router bgp`.

Negation Semantics

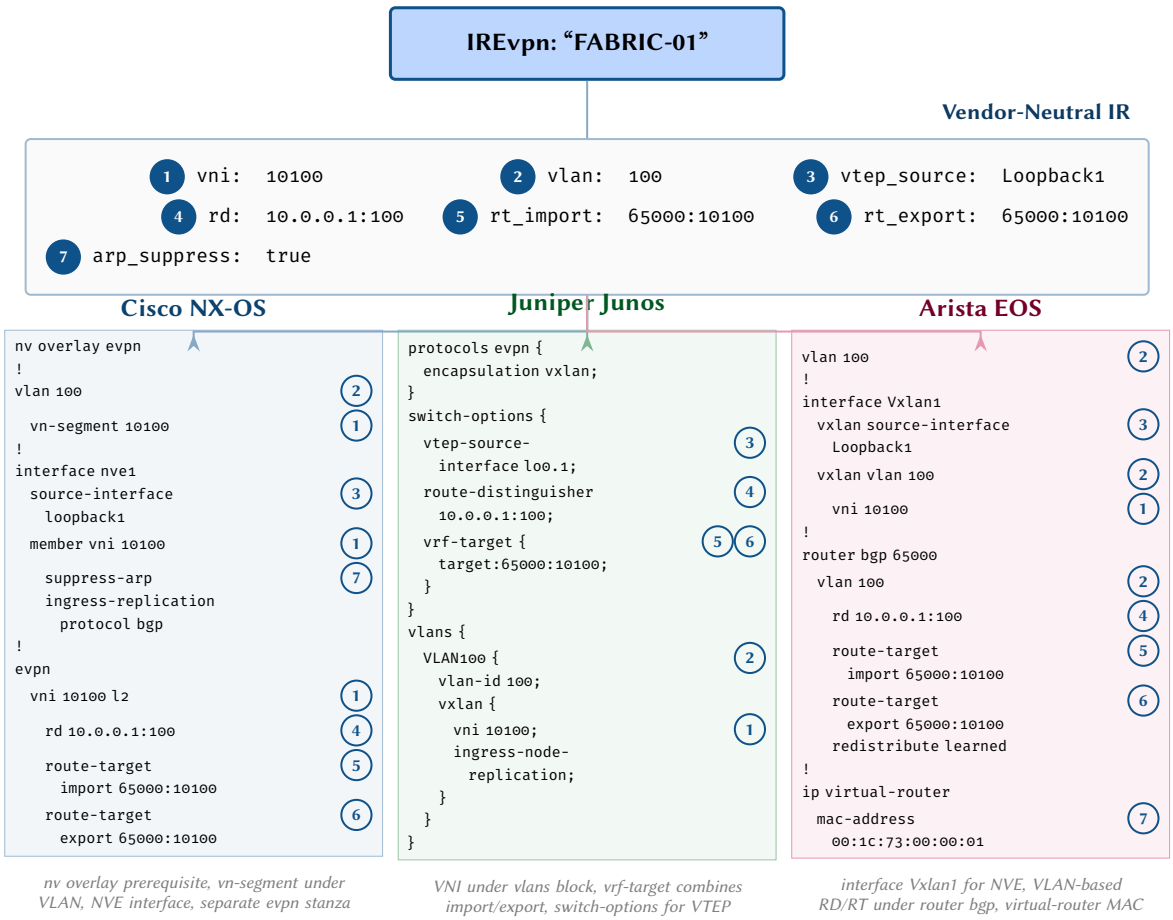


Figure 27. VXLAN/EVPN fabric transformation. Seven IR attributes are traced across three vendors commonly deployed in EVPN-VXLAN fabrics. NX-OS requires an `nv overlay evpn` prerequisite and distributes configuration across VLAN, NVE interface, and EVPN stanzas; Junos places VNI mappings under `vlans` with VTEP source in `switch-options`; Arista uses a dedicated `interface Vxlan1` and VLAN-scoped RD/RT under `router bgp`.

27 The Problem of Negation

Negation in network configuration is deceptively complex.

Consider the Cisco IOS command `no shutdown`. This does not mean "remove the shutdown configuration" — it means "enable the interface." The `no` prefix sometimes means "remove this configuration line," sometimes means "set the opposite value," and sometimes means "restore the default." The semantics depend on the specific command, the platform, and even the software version.

This ambiguity creates a fundamental challenge for configuration compilation: the compiler must know not just how to *set* a configuration attribute, but how to *unset* or *negate* it, and what state the device assumes when an attribute is absent.

Design Rule 12: Explicit Defaults

Every IR attribute must have a declared default value per platform. The compiler must know what state a device assumes when a configuration line is absent.

Insight:
"No shutdown" is the canonical example of why negation semantics matter. On Cisco IOS, many interface types default to shutdown, so enabling them requires an explicit negation of the default.

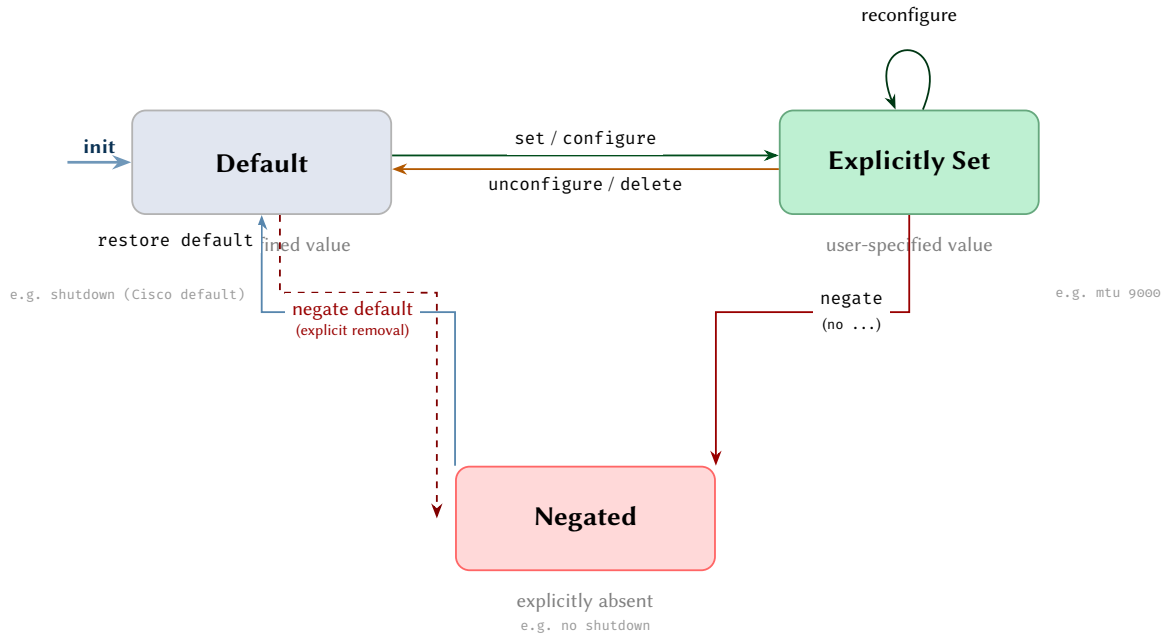


Figure 28. Finite-state model for a configuration attribute. Every attribute begins in the *Default* state carrying the vendor-defined value. The *configure* transition records an explicit value; *negate* marks the attribute as explicitly absent (producing `no ...` or `delete ...` syntax). The dashed arc represents negation of a default, needed when the compiled output must suppress a value that a vendor enables by default. All transitions use orthogonal routing.

28 Formal Negation Model

We formalise negation using denotational semantics [78] and lattice theory [54]. The model draws on the observation that configuration attribute states form a partial order analogous to domain theory in programming language semantics.

Definition 6: Configuration Attribute State Space

For a configuration attribute a on vendor v, the state space is a three-element set:

$$\mathcal{S}_v(a) = \{\bot, \delta_v(a), \sigma\} \tag{12}$$

where:

- $\bot$ denotes the *undefined* state (attribute not present in configuration).
- $\delta_v(a)$ denotes the *platform default* value for attribute a on vendor v.
- $\sigma \in \mathrm{Val}(a)$ denotes an *explicitly set* value.

When $\sigma = \delta_v(a)$, the attribute is set to its default value explicitly. The distinction between $\bot$ and $\delta_v(a)$ matters: $\bot$ means the line is absent from configuration, while explicit $\delta_v(a)$ means the line is present but redundant.

The negation operation $\neg_v : \mathcal{S}_v(a) \to \mathcal{S}_v(a)$ is an involution satisfying $\neg_v \circ \neg_v = \mathrm{id}$ for boolean attributes. For non-boolean attributes, negation maps an explicit value back to the default:

$$\neg_v(\sigma) = \begin{cases} \delta_v(a) & \text{if } \sigma \neq \delta_v(a) \\ \bot & \text{if } \sigma = \delta_v(a) \end{cases} \tag{13}$$

Design Rule 13: Negation Completeness

For every attribute that the compiler can set, there must be a corresponding negation operation defined for each supported platform.

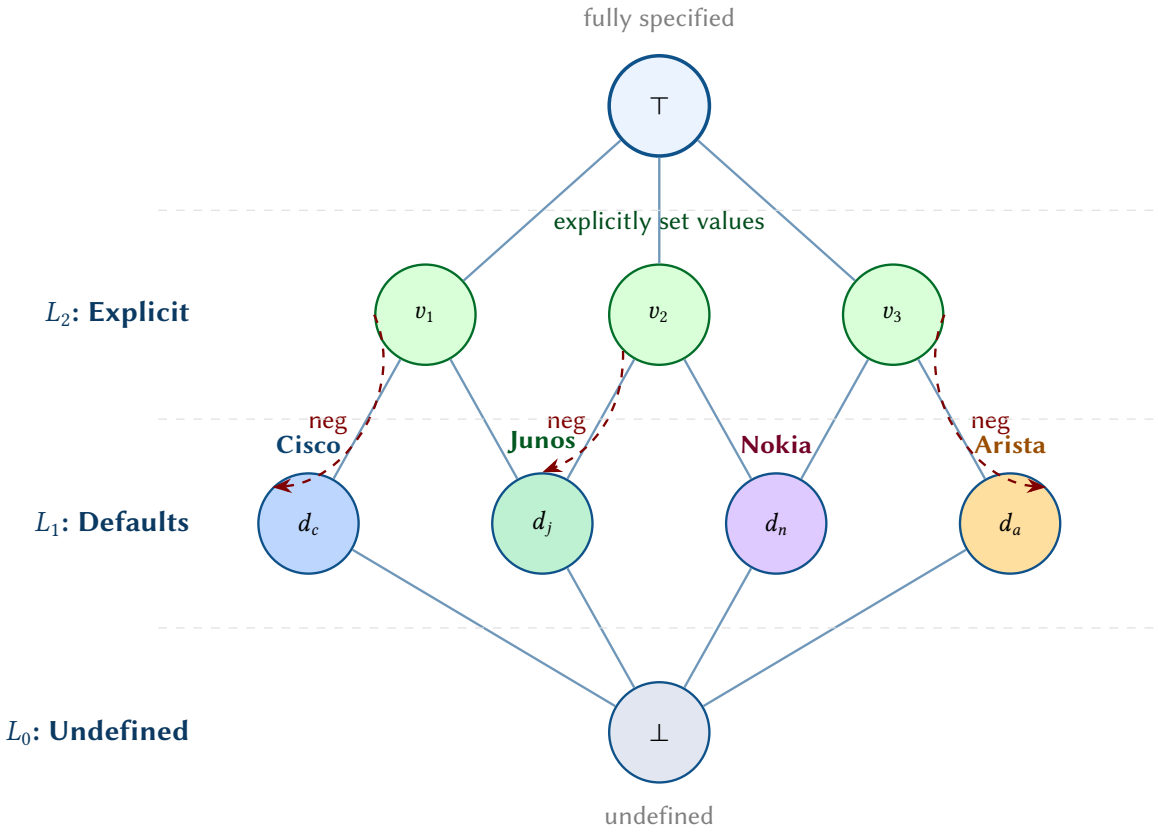


Figure 29. Hasse diagram of the configuration attribute lattice. The bottom element $\bot$ represents an undefined attribute; level L_1 holds vendor-specific defaults d_v; level L_2 holds explicitly set values. The negation involution (dashed red arrows) maps an explicit value back to its vendor default, generating the appropriate `no`/`delete` syntax during rendering. Dashed lines separate the three lattice levels.

Definition 7: Negation Involution

For any configuration attribute a with lifecycle state $\sigma \in \{\text{Set}, \text{Negated}\}$, the negation operation satisfies

$$\text{negate}\big(\text{negate}(\sigma)\big) = \sigma.$$

That is, negate is an *involution* on the non-default states: negating a *Set* attribute produces a *Negated* attribute, and negating a *Negated* attribute restores it to *Set*.

29 Vendor-Specific Negation

Having formalised the abstract negation model in the previous section, we now examine how each vendor platform implements negation differently. These platform-specific behaviours are precisely what the compiler must encode in its transform rules.

Each vendor implements negation differently:

29.1 Cisco IOS / IOS-XE

Cisco uses the `no` prefix for negation. The semantics are context-dependent:

- `no shutdown` — sets interface to enabled (negates the `shutdown` command).
- `no ip address` — removes the IP address (returns to default of no address).
- `no neighbor 10.0.0.1` — removes the BGP neighbor entirely.
- `no route-map POLICY` — removes reference to the route-map (does not delete the route-map definition).

29.2 Juniper Junos

Junos provides three negation mechanisms:

- `delete` — removes the configuration element entirely.
- `deactivate` — marks the element as inactive (present but not applied).
- Absence — simply omitting a configuration stanza from the candidate means it does not exist after commit.

The candidate-commit model means Junos can compute the diff between current and desired configurations natively, making negation more straightforward than on platforms with imperative CLI.

29.3 Nokia SR OS

Nokia SR OS avoids the negation ambiguity problem almost entirely through its MD-CLI design. Rather than using a `no` prefix to toggle between set and unset states, Nokia uses *explicit state attributes* that positively assert the desired value. This makes Nokia the most predictable platform for the compiler to target from a negation perspective.

- **`admin-state enable/disable`** — the primary mechanism for enabling and disabling interfaces, protocols, and services. There is no ambiguity: both states are explicitly named, and the compiler simply emits the desired value.
- **`delete`** — removes configuration elements in the MD-CLI candidate. Unlike Cisco's `no`, which may negate or restore defaults depending on context, Nokia's `delete` always means "remove this subtree from the configuration."
- **Classic CLI compatibility** — the classic CLI retains `shutdown`/`no shutdown` semantics similar to Cisco, but the compiler targets the MD-CLI exclusively, avoiding this legacy ambiguity.

29.4 Arista EOS

Arista EOS inherits the `no` prefix negation model from Cisco IOS, with the same context-dependent semantics: `no shutdown` means "enable," `no neighbor` means "remove," and `no ip address` means "restore to unconfigured." However, EOS adds one significant mechanism that IOS-XE lacks: the `default` command.

- **`no` prefix** — identical semantics to IOS-XE for most commands. The compiler can reuse negation logic from the IOS-XE transform module for approximately 90% of commands.
- **`default interface Ethernet1`** — returns the entire interface to factory defaults in a single command. This is strictly more powerful than Cisco's per-attribute `no` commands and is useful when the compiler needs to ensure a clean slate before applying new configuration.
- **`no` on EVPN constructs** — Arista's EVPN implementation uses EOS-specific syntax (e.g. `no vxlan vlan 100 vni 10100`) where the negation semantics differ subtly from NX-OS's `no member vni` syntax.

Insight: *Nokia's explicit `admin-state` pattern is the cleanest negation model of any vendor. The compiler can treat interface enablement as a simple attribute assignment — `admin-state = enable` — rather than reasoning about whether to emit `no shutdown` (as on Cisco) or omit the shutdown command entirely (as on Junos where interfaces are enabled by default).*

29.5 Cisco NX-OS

NX-OS shares the `no` prefix model with IOS-XE but adds a complication: features must be explicitly enabled via `feature` *name* before the corresponding protocol configuration can exist. Negation of the feature command (`no feature bgp`) removes *all* BGP configuration from the device — a much more destructive operation than `no router bgp` on IOS-XE, which removes only the BGP process. The compiler must never emit `no feature` as part of a configuration delta unless the intent is a complete feature removal.

29.6 Cisco IOS-XR

IOS-XR uses a candidate-commit model, so negation takes two forms:

- `no` — marks a configuration element for deletion in the candidate. The deletion only takes effect after `commit`.
- `delete` — used in some contexts as a synonym for `no`, particularly when removing subtrees.
- `commit replace` — replaces the entire running configuration with the candidate, effectively negating everything not in the candidate. The compiler can leverage this for full-device replacements.

The commit model means that multiple negation operations can be batched and validated together before applying, reducing the risk of transient inconsistency that can occur with IOS-XE's immediate-apply model.

Insight: *The `default` command gives EOS a "reset to known state" capability that simplifies diff compilation (§37). When the compiler detects that more than a threshold number of attributes on an interface have changed, it can emit a single `default interface` followed by the desired configuration, rather than computing per-attribute negations.*

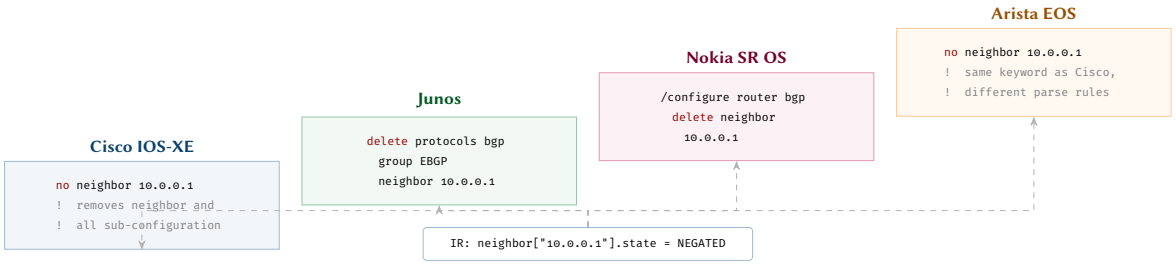


Figure 30. Vendor-specific negation syntax for removing a BGP neighbor. A single IR negation state compiles to structurally different commands: Cisco and Arista use the `no` prefix, Junos uses `delete` in set-mode, and Nokia uses `delete` within a navigated context.

30 Beyond Shutdown: Other Negation-Sensitive Semantics

The shutdown/no-shutdown case from fig. 31 is the most commonly cited negation challenge, but it is far from the only one. Several other configuration attributes exhibit similarly treacherous negation semantics that the compiler must handle correctly.

30.1 Passive Interface Semantics

An IS-IS or OSPF *passive interface* participates in the routing protocol's topology (its prefixes are advertised) but does not send or receive protocol packets on that interface. The negation semantics differ across vendors:

Vendor	Enable Passive	Disable Passive
Cisco IOS-XE	`passive-interface GigE0/0`	`no passive-interface GigE0/0`
Cisco NX-OS	`isis passive-interface level-1-2`	`no isis passive-interface`
Cisco IOS-XR	`passive` (under isis interface)	`no passive`
Junos	`interface ge-0/0/0 passive`	`delete interface ge-0/0/0 passive`
Nokia SR OS	`interface "ge-0/0/0" passive`	`no interface "ge-0/0/0" passive`
Arista EOS	`passive-interface GigE0/0`	`no passive-interface GigE0/0`

On Cisco IOS, OSPF supports `passive-interface default`, which reverses the semantics: all interfaces are passive unless explicitly declared `no passive-interface`. The compiler must track whether this global directive is in effect to emit the correct per-interface commands.

30.2 Default Route Origination

BGP `default-originate` controls whether the device advertises the default route (0.0.0.0/0) to a neighbour. The negation is particularly dangerous because an incorrect default route advertisement can blackhole traffic across the network:

Vendor	Enable	Negate / Disable
Cisco IOS-XE	`neighbor X default-originate`	`no neighbor X default-originate`
Cisco NX-OS	`default-originate` (under neighbor af)	`no default-originate`
Cisco IOS-XR	`default-originate` (under neighbor af)	`no default-originate`
Junos	`default-address-family inet unicast rib-group default`	`delete default-address-family`
Nokia SR OS	`default-originate`	`no default-originate`
Arista EOS	`neighbor X default-originate`	`no neighbor X default-originate`

The default state is "do not originate" on all vendors — but the Cisco/Arista syntax conflates the neighbour identifier with the attribute, meaning that `no neighbor X default-originate` does not remove the neighbour, only the default origination. The compiler must parse the `no` prefix differently depending on the attribute context.

30.3 Authentication and Security Defaults

Protocol authentication presents negation challenges because the *absence* of authentication is the default, but it is often desirable to enforce authentication explicitly. The compiler must distinguish between "authentication not configured" and "authentication explicitly disabled":

- **IS-IS authentication**: on Cisco, `no isis authentication` removes all authentication. On Junos, `delete authentication-key` removes the key but `delete authentication-type` is needed to fully disable.
- **BGP MD5 authentication**: Cisco uses `no neighbor X password` while Junos uses `delete authentication-key`. On Nokia, authentication is configured per-group and requires `no authentication-key`.

- **ACL implicit deny**: on Cisco and Arista, ACLs have an implicit "deny any" at the end. There is no command to negate this — it is an inherent platform behaviour. On Junos firewall filters, the implicit action is platform-version-dependent and must be tracked in the defaults database.

30.4 Timer and Threshold Defaults

Protocol timers (hello intervals, hold times, dead intervals) have vendor-specific defaults that the compiler must track:

Timer	IOS-XE	NX-OS	IOS-XR	Junos	Nokia	EOS
IS-IS Hello	10s	10s	10s	9s	9s	10s
IS-IS Hold	30s	30s	30s	27s	27s	30s
OSPF Hello	10s	10s	10s	10s	10s	10s
OSPF Dead	40s	40s	40s	40s	40s	40s
BGP Hold	180s	180s	180s	90s	90s	180s
BGP Keepalive	60s	60s	60s	30s	30s	60s

Negating a timer restores the default: `no isis hello-interval` on Cisco, `delete hello-interval` on Junos. But "restoring the default" means different values on different vendors, so the compiler must be aware that negation produces *different outcomes* for the same logical operation.

Insight: *If the IR specifies an IS-IS hello timer of 9 seconds, the compiler should suppress it for Junos and Nokia (where it is the default) but emit it for Cisco (where it overrides the default of 10s). This is a concrete example of default suppression requiring per-vendor, per-attribute granularity.*

31 Default State and Suppression

A key optimisation in configuration compilation is *default suppression*: if the desired value of an attribute equals the platform default, the compiler can omit the configuration line entirely. This produces cleaner, more readable configurations and reduces the risk of accidentally overriding future default changes.

However, default suppression must be applied carefully:

1. **Platform defaults vary**: `shutdown` is the default for physical interfaces on Cisco IOS but `no shutdown` is the default on some Arista platforms.
2. **Version defaults change**: a new software version may change the default value of an attribute.
3. **Explicit is sometimes safer**: for security-sensitive attributes (ACL implicit deny, protocol authentication), explicitly setting the default is defensive and self-documenting.

The compiler maintains a *platform default database* that maps (platform, version, attribute) triples to their default values. During emission, for each attribute:

$$\text{emit}(a, \sigma) = \begin{cases} \text{emit line} & \text{if } \sigma \neq \delta_v(a) \text{ or } a \in \mathcal{A}_{\text{explicit}} \\ \text{suppress} & \text{otherwise} \end{cases} \tag{14}$$

where $\mathcal{A}_{\text{explicit}}$ is the set of attributes that should always be emitted explicitly regardless of default match.

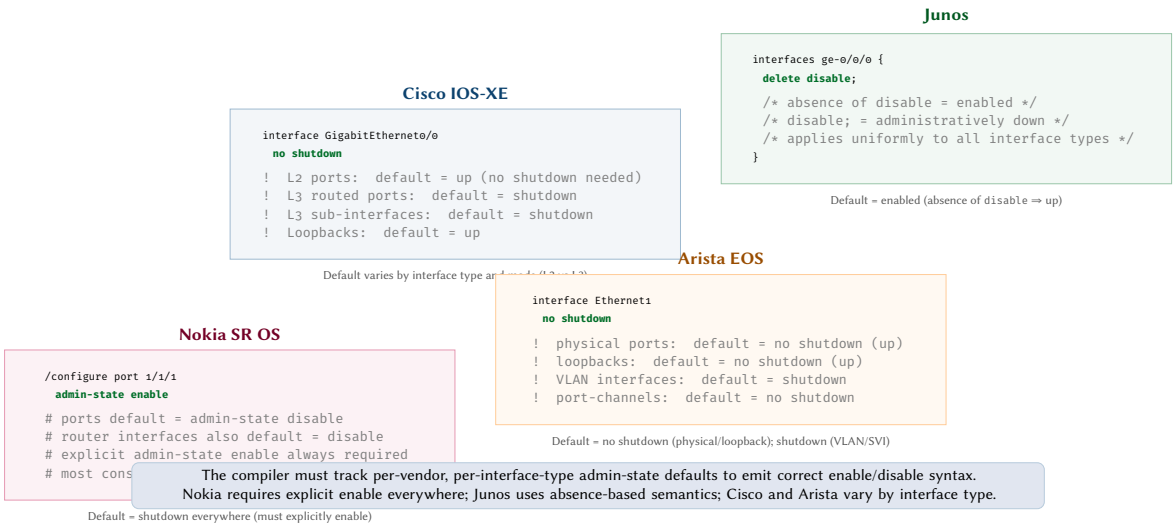


Figure 31. Shutdown and no-shutdown semantics across four vendors. The default administrative state varies not only across vendors but also across interface types within a single vendor, requiring the compiler to maintain a per-type default matrix.

31.1 Negation Across a Full Interface

To illustrate the scope of the negation problem, consider the full set of attributes on a single interface and how negation semantics vary across vendors for *each* attribute. fig. 32 presents this analysis as a comprehensive reference table.

The key observation is that there is no single "negation" operation — each attribute has its own default value, its own set command, and its own negation command, and all three may differ across vendors. The compiler must maintain a *per-attribute, per-vendor* negation database.

Interface Attribute Negation Matrix

Attribute	Cisco IOS-XE	Junos	Nokia SR-OS	Arista EOS

`enabled (admin state)`	`default: shutdown`
	`set: no shutdown`
	`negate: shutdown`
		`default: enabled`
		`set: disable`
		`negate: delete disable`
			`default: disable`
			`set: admin-state enable`
			`negate: admin-state disable`
				`default: shutdown`
				`set: no shutdown`
				`negate: shutdown`

`mtu`	`default: 1500`
	`set: mtu 9000`
	`negate: no mtu`
		`default: 1514`
		`set: mtu 9000`
		`negate: delete mtu`
			`default: 1500`
			`set: mtu 9000`
			`negate: no mtu`
				`default: 1500`
				`set: mtu 9000`
				`negate: no mtu`

`speed`	`default: auto`
	`set: speed 1000`
	`negate: no speed`
		`default: auto`
		`set: speed 1g`
		`negate: delete speed`
			`default: auto`
			`set: speed 1000`
			`negate: no speed`
				`default: auto`
				`set: speed forced 1000full`
				`negate: no speed`

`description`	`default: (empty)`
	`set: description UPLINK`
	`negate: no description`
		`default: (empty)`
		`set: description UPLINK`
		`negate: delete description`
			`default: (empty)`
			`set: description "UPLINK"`
			`negate: no description`
				`default: (empty)`
				`set: description UPLINK`
				`negate: no description`

`ipv4_address`	`default: (none)`
	`set: ip address 10.0.0.1 /30`
	`negate: no ip address`
		`default: (none)`
		`set: address 10.0.0.1/30`
		`negate: delete address`
			`default: (none)`
			`set: address 10.0.0.1/30`
			`negate: no address`
				`default: (none)`
				`set: ip address 10.0.0.1/30`
				`negate: no ip address`

Cell colours: Default varies · Numeric reset · Value removal

Figure 32. Negation matrix for interface attributes across four vendors. Each cell shows the platform default, the `set` command, and the `negate` command. Default semantics (shutdown vs. enabled), negate syntax (`no` vs. `delete`), and quoting conventions vary significantly across platforms.

31.2 The Platform Defaults Database

The default suppression mechanism relies on an accurate platform defaults database. This database maps (platform, version, attribute) triples to their default values. fig. 33 illustrates the structure of this database and how it feeds into the emission decision.

The database is populated from three sources:

1. **Vendor documentation**: the authoritative source, but often incomplete or inconsistent.
2. **Lab testing**: deploying a factory-default device and inspecting its running configuration reveals actual defaults.
3. **YANG models**: where available, YANG `default` statements provide machine-readable defaults.

Insight: *The platform defaults database is one of the highest-maintenance components of the compiler. Every new platform version must be audited for default changes — a task that is tedious but essential for correct output.*

Feature Capabilities

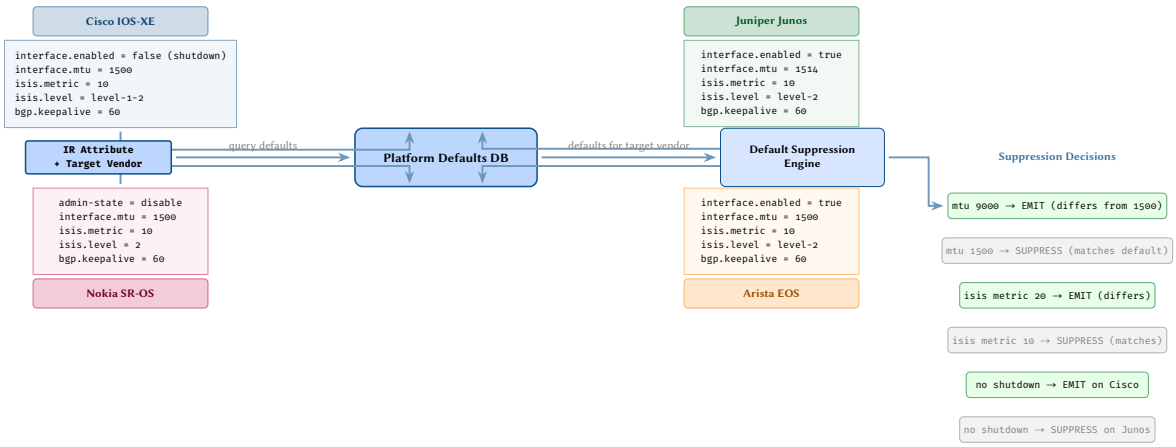


Figure 33. Platform defaults database. Each vendor registers its default values for interface, protocol, and operational attributes. The default suppression engine compares IR attribute values against vendor defaults: lines that match the default are suppressed from output, reducing configuration noise. Critically, the same IR attribute (e.g. `enabled`) may require emission on one platform but suppression on another.

32 The Feature Capability Model

Not all platforms support all features, and support levels vary across software versions. The compiler must know what each platform can do before attempting to generate configuration for it.

Definition 8: Capability Predicate

The capability predicate is a function:

$$\text{cap} : \mathcal{P} \times \mathcal{V}_r \times \mathcal{F} \to \{\texttt{full}, \texttt{partial}, \texttt{none}\} \tag{15}$$

where $\mathcal{P}$ is the set of platforms, $\mathcal{V}_r$ the set of version ranges, and $\mathcal{F}$ the set of features. The return value indicates:

- `full` — the feature is fully supported; emit standard configuration.
- `partial` — the feature is supported with limitations; emit with platform-specific workarounds or warnings.
- `none` — the feature is not supported; do not emit.

Design Rule 14: Feature Gating

Every transform rule must register the capability predicates it requires. The Gate stage evaluates these predicates before the Emit stage runs.

Design Rule 15: Fail-Fast

If a required feature is not supported on the target platform (cap = `none` with no fallback), the compiler must fail with a diagnostic error *before* any configuration is emitted.

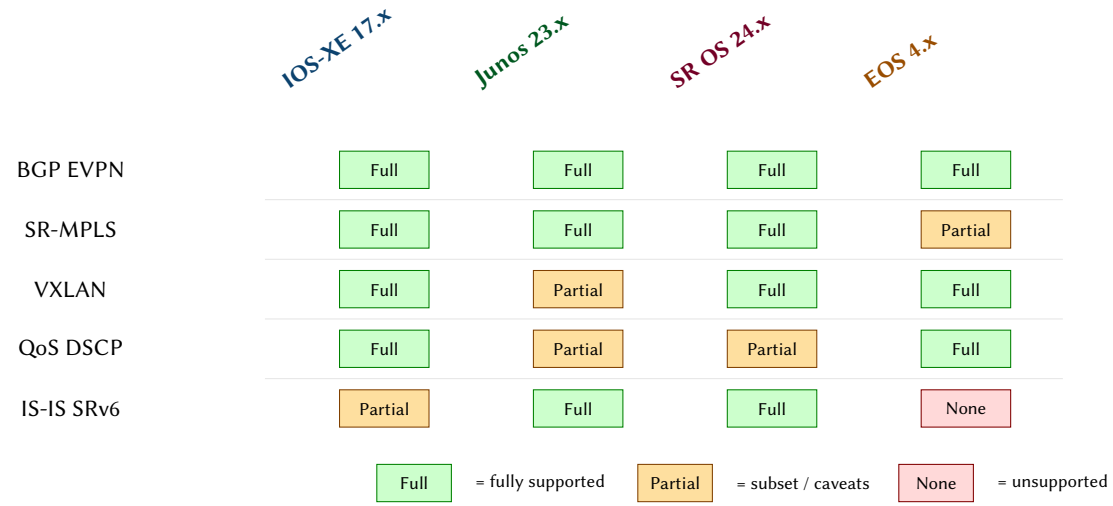


Figure 34. Platform capability matrix. Each cell indicates whether a feature is fully supported, partially supported (with caveats or missing sub-features), or unsupported on the given platform release. The compiler uses this matrix to emit warnings or errors when the IR references unavailable capabilities.

33 Version Range Representation

Platform versions are represented as structured version identifiers with comparison semantics. Each vendor has its own versioning scheme:

Vendor	Version Format	Example
Cisco IOS-XE	`major.minor.patch`	17.9.1
Cisco NX-OS	`major.minor(release)letter`	10.3(2)F
Cisco IOS-XR	`major.minor.patch`	7.9.2
Juniper Junos	`major.minor Rrelease.build`	23.2R1.4
Nokia SR OS	`major.minor.Rrelease`	24.3.R1
Arista EOS	`major.minor.patchF`	4.31.1F

The compiler normalises these into a comparable tuple representation for ordering and range checks.

Design Rule 16: Version Conservatism

When the target platform version is unknown or ambiguous, the compiler assumes the oldest supported version. This ensures generated configuration is compatible with the widest range of software versions.

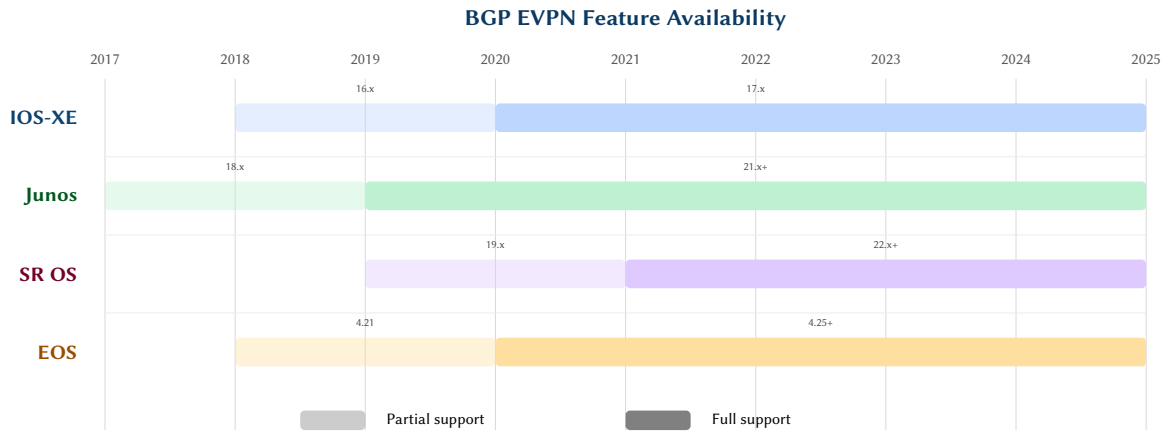


Figure 35. Timeline of BGP EVPN feature availability across four vendor platforms. Lighter bars indicate partial support (basic EVPN Type-2/3 routes); solid bars indicate full support including Type-5 IP prefix routes and multi-homing. The compiler's capability database encodes these version-dependent feature boundaries.

34 Feature Gate Decision Tree

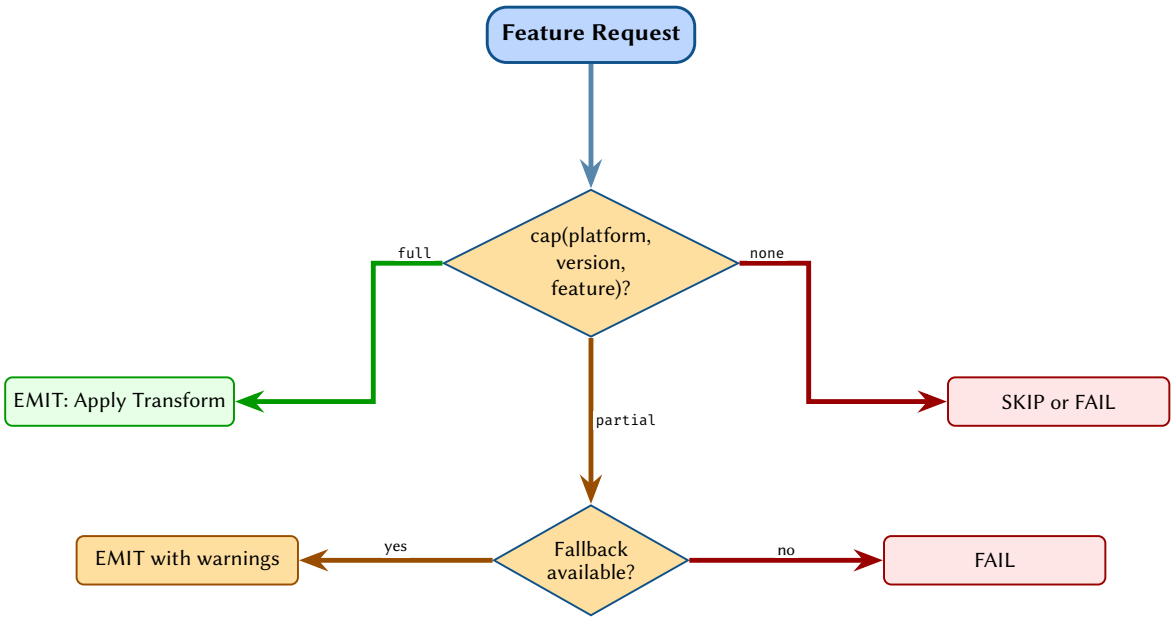
The Gate stage processes each IR node through a decision tree:

1. Query cap(platform, version, feature) for the feature(s) required by the IR node.

2. If `full`: proceed to Emit with the standard transform.
3. If `partial`: check whether a platform-specific workaround exists.
    - If workaround available: proceed with modified transform and emit a warning.
    - If no workaround: fail with diagnostic.
4. If `none`: check whether the feature is optional.
    - If optional: skip emission with an informational message.
    - If required: fail with diagnostic.

The decision tree produces one of three outcomes:

- **EMIT** — proceed with configuration generation.
- **SKIP** — omit this feature from the output (feature is optional and unsupported).
- **FAIL** — halt compilation with an error (required feature is unsupported).



The gate stage queries the capability matrix $C(p, v, f)$ and routes each feature through the appropriate branch.

Figure 36. Feature gating decision tree. Each feature request is checked against the capability matrix; *full* support emits directly, *partial* support attempts a fallback, and *none* halts or skips compilation.

34.1 Capability Deep Dive: EVPN Type-5

To illustrate the feature gate in action, consider the compilation of an `IREvpn` node with `route_type=type5` (IP prefix advertisement via EVPN). This is a relatively recent feature with varying support levels across vendors and versions:

- `IOS-XE v17.6`: `partial` — Type-5 routes supported but only with VXLAN encapsulation, not MPLS. The compiler can emit with a workaround (force VXLAN encapsulation).
- `IOS-XE v17.9`: `full` — both VXLAN and MPLS encapsulations supported.
- `Junos v21.2`: `full` — full Type-5 support with all encapsulations.
- `SR-OS v22.10`: `full` — supported via VPRN service type.
- `EOS v4.28`: `none` — Type-5 not supported on this version. The feature gate returns FAIL if Type-5 is required, or SKIP if it is optional.

fig. 37 shows the complete decision tree for this scenario.

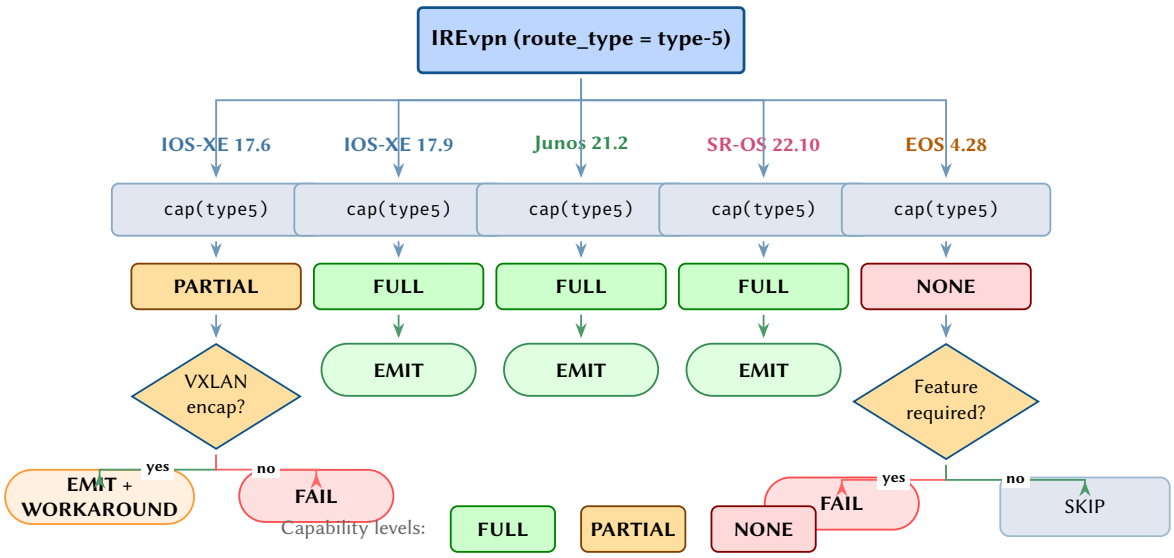


Figure 37. Capability decision tree for EVPN Type-5 route support. Each vendor/version pair is queried against the capability database. Results of *full* proceed directly to emission; *partial* triggers a workaround check; *none* either skips the feature or raises a compilation failure depending on whether the feature is mandatory.

35 Validation Pipeline

Validation is not a single step but a series of checkpoints distributed throughout the pipeline. Each checkpoint enforces specific invariants:

Checkpoint	Location	Invariant
Schema Validation	After Lift	IR tree conforms to type hierarchy and attribute schemas
Referential Integrity	After Lift	All policy/ACL references resolve within the IR
Transform Coverage	After Transform	Every IR node has been transformed; no orphan nodes
Capability Check	Gate stage	All required features are supported on the target platform
Syntax Validation	After Emit	Output text is syntactically valid for the target platform

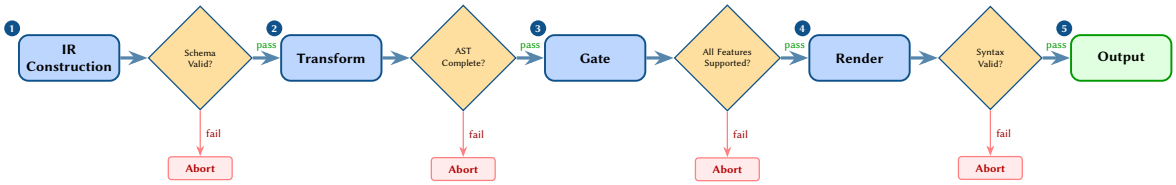


Figure 38. Validation pipeline with inter-stage checkpoints. Each diamond represents a checkpoint that must pass before the next stage executes; failure at any point aborts compilation with a diagnostic report.

Rendering and Output

36 Template Rendering

The Emit stage converts vendor AST nodes into configuration text using templates. Templates are deliberately kept simple: they are pure syntax formatters with no conditional logic or computation.

Design Rule 17: Template Simplicity

Templates are *pure syntax*. All logic — conditionals, loops, defaults, negation — is resolved in the Transform and Gate stages. A template receives a fully resolved AST node and produces text.

Each AST node type has an associated template. The template is a string with slot markers that are filled from the node's attributes:

Insight: *Keeping templates logic-free is a deliberate design choice. Logic in templates is untestable, unmaintainable, and leads to the "template spaghetti" problem familiar from Jinja2 [79]-based automation. The Rust implementation uses Tera [80] templates, which enforce the same separation. Czarnecki and*

Cisco BGP Neighbor Template

```
router bgp {{asn}}
 neighbor {{peer_ip}} remote-as {{peer_asn}}
 neighbor {{peer_ip}} update-source {{update_source}}
 !
 address-family {{afi}} {{safi}}
  neighbor {{peer_ip}} activate
```

Templates may be parameterised by the output mode. Junos, for example, supports both hierarchical and set-line output from the same AST:

- **Hierarchical**: nested curly-brace format for human readability.
- **Set-lines**: flat `set` commands for programmatic application via `load set`.
- **XML**: Junos XML format for NETCONF.

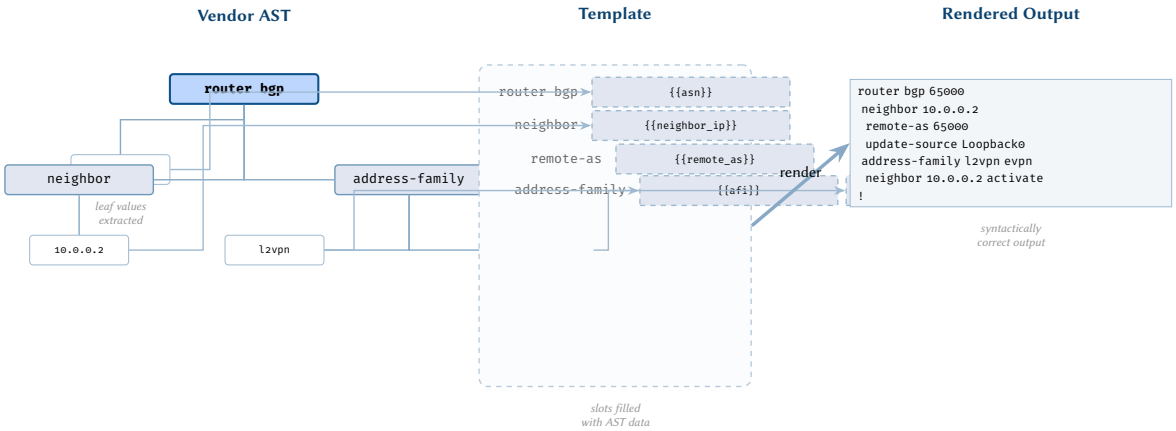


Figure 39. Template rendering. AST leaf values (left) are bound to named template slots (centre); the rendering engine fills the template to produce syntactically correct vendor configuration text (right). Templates contain only syntax — no logic or conditionals.

37 Diff Compilation

A critical use case is not generating configuration from scratch but computing the *difference* between the current device state and the desired state, then producing the minimal set of commands to transition between them.

Definition 9: Configuration Diff

Given a current IR IR_{cur} and a desired IR IR_{des} for the same device, the configuration diff is:

$$\Delta = (\mathcal{A}, \mathcal{R}, \mathcal{M}) \tag{16}$$

where $\mathcal{A}$ is the set of nodes to add, $\mathcal{R}$ the set of nodes to remove, and $\mathcal{M}$ the set of nodes to modify (attribute changes only).

The diff compilation process:

1. Compute Δ by comparing IR_{cur} and IR_{des} node by node.
2. For each removal $r \in \mathcal{R}$: generate the appropriate negation command (vendor-specific).
3. For each modification $m \in \mathcal{M}$: generate the replacement command.
4. For each addition $a \in \mathcal{A}$: generate the standard configuration command.
5. Order the commands respecting the change ordering constraints.

Design Rule 18: Change Atomicity

The compiler must respect the target platform's atomicity model. Platforms with candidate-commit (Junos, Nokia MD-CLI) can apply all changes as a single atomic transaction. Platforms with immediate-apply CLI (Cisco IOS) must sequence changes to avoid transient inconsistency.

Design Rule 19: Safe Change Ordering

Within a changeset, deletions must precede additions for the same resource. This prevents conflicts where adding a new resource clashes with an existing one that should have been removed first.

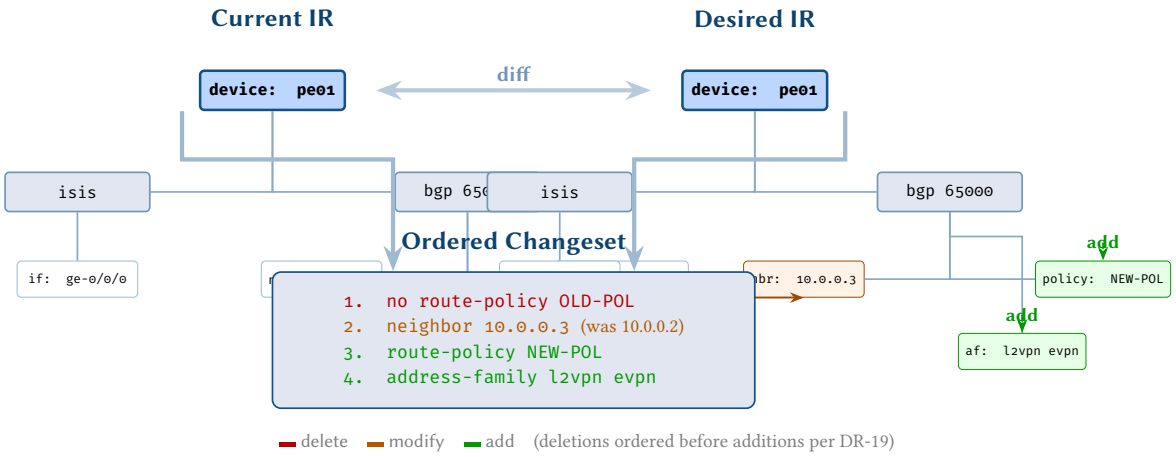


Figure 40. Diff flow. The current and desired IR trees are compared; additions, deletions, and modifications are collected into an ordered changeset that respects dependency constraints (deletions before additions for the same resource).

37.1 Detailed Diff Example

Consider a concrete diff scenario where the desired state of `pe01` differs from the current state in three ways: a new BGP neighbor must be added, an interface MTU must be changed, and an obsolete ACL must be removed.

fig. 41 walks through this scenario, showing the current and desired IR trees side by side, the computed diff Δ, and the resulting ordered changeset. Note how the changeset respects the safe ordering rule: the ACL deletion (which might be referenced by the old BGP neighbor) comes first, the interface modification second, and the new BGP neighbor addition last.

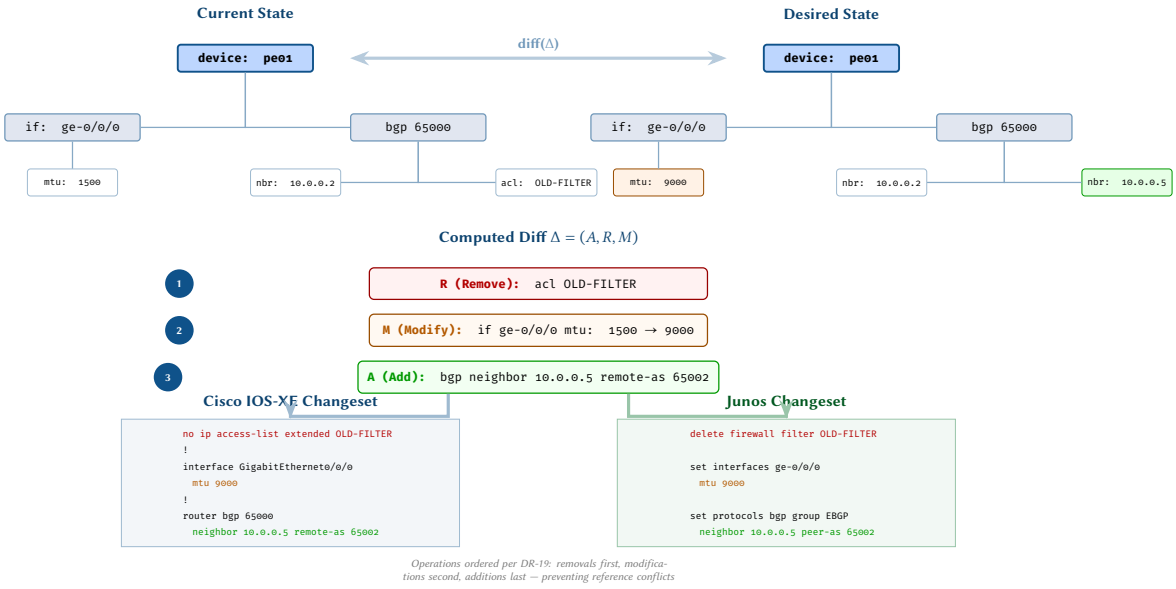


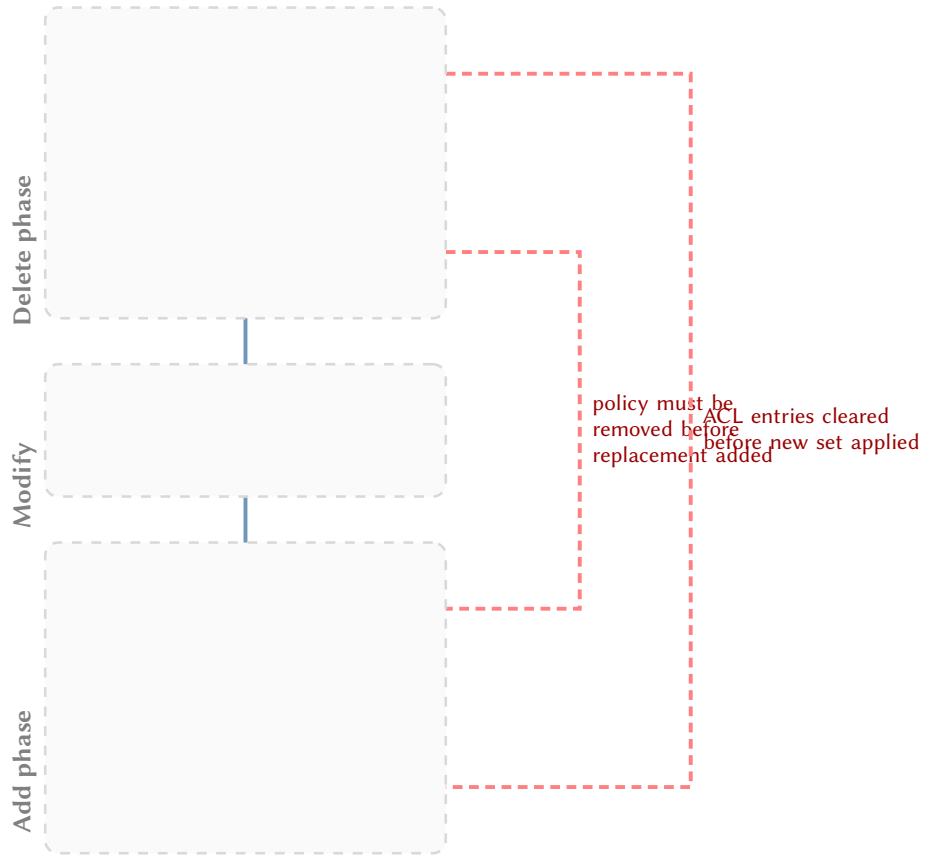
Figure 41. Detailed diff example. The current and desired IR trees for `pe01` differ in three ways: an ACL removal, an MTU change, and a new BGP neighbour. The computed diff $\Delta = (A, R, M)$ is ordered safely (removals before modifications before additions) and compiled into vendor-specific changesets for both Cisco IOS-XE and Junos.

37.2 Formal Properties of the Diff Operation

Theorem 3 (Safe Diff Ordering). *If section 37 (Safe Change Ordering) is satisfied, then applying the ordered changeset to a consistent device state produces a consistent intermediate state at every step.*

Proof. By induction on the length n of the ordered changeset $\langle c_1, c_2, \ldots, c_n \rangle$.

Base case ($n = 0$). The empty changeset leaves the device state unchanged; consistency is preserved trivially.



Topological ordering ensures referential integrity: resources are removed before their replacements are added, preventing dangling references during the transient commit window.

Figure 42. Change ordering DAG. Deletions are sequenced before additions for the same resource class, with cross-dependencies (dashed red) enforcing referential integrity during the commit transaction.

Inductive step. Assume that after applying $\langle c_1, \ldots, c_k \rangle$ the device state S_k is consistent. Consider the next change c_{k+1}. By section 37, the ordering guarantees the following properties:

1. *Deletions* remove references to an object before the referenced object itself is deleted, so no dangling references are introduced.
2. *Additions* only reference objects that are already present in S_k or that appear earlier in the changeset (and are therefore already added), so all forward references are satisfied.
3. *Modifications* preserve reference integrity because the ordering ensures that neither the source nor the target of any reference is in an inconsistent intermediate state when the modification is applied.

Therefore S_{k+1} is consistent, completing the induction. □

Definition 10: Diff Algebra

The diff operation Δ on **IR** trees satisfies the following properties:

1. **Identity.** $\Delta(I, I) = \emptyset$ for any **IR** tree I. Diffing a tree against itself produces the empty changeset.
2. **Correctness.** $\text{apply}\big(\Delta(I_1, I_2),\ I_1\big) = I_2$. Applying the diff to the source tree yields the target tree.
3. **Bounded size.** $|\Delta(I_1, I_2)| \leq |I_1| + |I_2|$. The size of the changeset is at most the sum of the sizes of the two trees.

38 Output Formats

The compiler supports multiple output formats from the same AST:

1. **CLI text** — native CLI commands for Cisco IOS-XE, Arista EOS, and Nokia classic CLI.
2. **Junos set-lines** — flat `set` commands for Junos `load set` operations.
3. **Junos hierarchical** — curly-brace format for Junos `load merge/replace`.
4. **Junos XML** — XML format for NETCONF `edit-config`.
5. **YANG-JSON** — JSON encoding per RFC 7951 for RESTCONF or gNMI.

The AST is format-agnostic; the choice of output format is made at rendering time. This decoupling allows the same AST to produce configuration for CLI-based workflows (copy-paste or expect scripts), API-based workflows (NETCONF/RESTCONF), and model-driven workflows (gNMI).

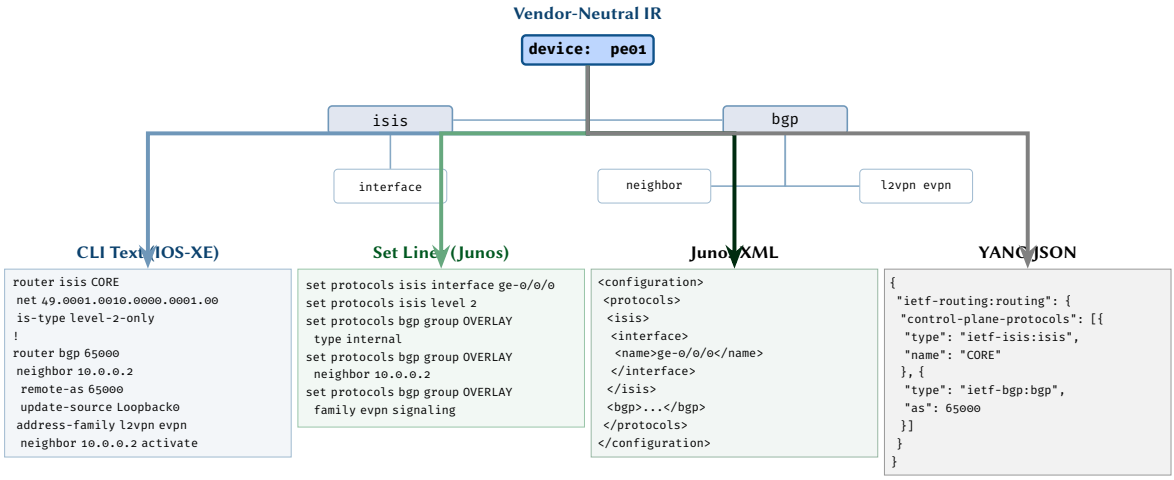


Figure 43. Multi-format emission. A single vendor-neutral IR is rendered into four output formats: Cisco CLI text, Junos set-style commands, Junos XML, and YANG-modelled JSON.

Implementation and Examples

39 Architecture and NTE Integration

The Network Topology Engine (NTE) is the implementation of the multi-layer attributed graph model described in ANK-TR-2026-03 [8], building on the network modelling foundations of the Internet Topology Zoo [81] and subsequent work [82]. It realises the plan topology G_{plan}, manages the graph layers, and performs device projection (π_d) to extract per-device state. The configuration compiler sits *downstream* of the NTE: it consumes the IR trees that the NTE produces and transforms them into vendor-specific configuration text.

Insight:
The NTE is the source of truth for network state. It holds the multi-layer graphs, resolves cross-device references, and performs the projection that creates device-scoped IR trees. The compiler's job begins where the NTE's job ends: given a complete IR tree for a single device, produce the correct configuration for that device's platform.

39.1 The NTE as Graph Engine

The NTE provides the upstream graph infrastructure:

- **Plan topology management**: the G_{plan} graph with its vertex set, edge set, type function, and attribute function — the data model from ANK-TR-2026-03.
- **Multi-layer attribute resolution**: collapsing protocol layers (physical, switching, routing, overlay) onto device-scoped views.
- **Device projection**: applying π_d to extract each device's slice of the topology, including all relevant attributes from neighbouring devices (peer ASNs, interface addresses, etc.).
- **Graph query interface**: the NTE exposes graph queries that allow the compiler (and other tools) to traverse the topology — e.g. "all BGP neighbours of device d" or "all interfaces in IS-IS area 49.0001" (see section 42).

39.2 Compiler as Downstream Library

The compiler is implemented as a Rust crate (`config-compiler`) with Python bindings via PyO3 [83]. It receives IR trees from the NTE and runs them through the Transform/Gate/Emit pipeline:

- **Performance**: Rust's zero-cost abstractions and memory safety for the core compilation pipeline.
- **Extensibility**: Python API for user-defined transform rules and custom templates.
- **Serialisation**: Serde [84] for IR/AST serialisation to JSON, YAML, and binary formats.
- **Decoupled interface**: the compiler depends only on the IR tree schema, not on the NTE's internal graph representation. This means it can also accept IR trees from other sources (CI/CD pipelines, Ansible playbooks, test harnesses).

The compiler exposes a simple API surface:

Compiler API

```
from config_compiler import Compiler, IRTree
```

```
compiler = Compiler(vendor="cisco_iosxe", version="17.9.1")
ir = IRTree.from_json("pe01.ir.json")
config_text = compiler.compile(ir)
```

39.3 End-to-End Data Flow

The complete pipeline from intent to configuration flows through the NTE into the compiler:

1. **Plan Parser** (NTE) ingests topology definitions and populates the plan graph.
2. **Topology Builder** (NTE) constructs G_{plan} with all multi-layer attributes.
3. **Device Projector** (NTE) applies π_d for each device, producing per-device IR trees.
4. **Config Compiler** (downstream library) receives each IR tree and runs the Transform/Gate/Emit pipeline.
5. **Diff Engine** (downstream library) computes changesets against current device state.
6. **Output Renderer** formats and delivers configuration to devices via CLI, NETCONF, or gNMI.

Decoupled Interface

While the NTE is the primary producer of IR trees, the compiler's interface is defined purely by the IR schema. Any system that produces valid IR trees can use the compiler — the NTE is the canonical source, but not the only possible one.

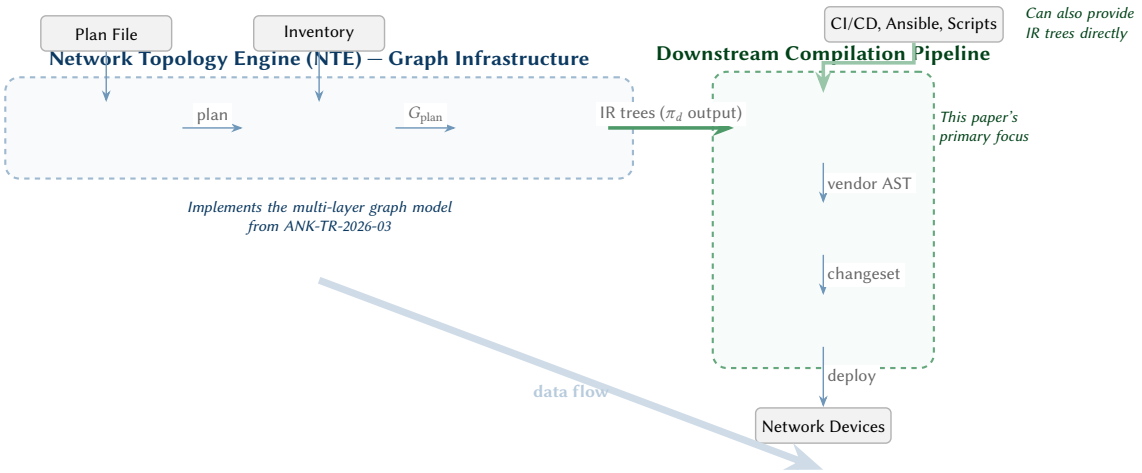


Figure 44. System architecture showing the Network Topology Engine (left) as the upstream graph engine that implements the multi-layer model from ANK-TR-2026-03. The NTE performs device projection to produce IR trees, which flow downstream into the configuration compiler (right, green border). The compiler transforms IR trees into vendor-specific configuration text. While the NTE is the primary source of IR trees, the compiler's interface is schema-defined, so other systems (CI/CD, Ansible) can also produce IR trees directly.

40 Full Worked Example

We now walk through the complete compilation pipeline for a concrete example: a Provider Edge (PE) router (`pe01`) in a service provider network with IS-IS, BGP EVPN, and a customer VRF.

40.1 Step 1: Project

The plan topology contains `pe01` as a node of type `router` with edges to:

- `pe02` via `ge-0/0/0` (IS-IS adjacency)
- `p01` via `ge-0/0/1` (IS-IS adjacency, MPLS core)
- `ce01` via `ge-0/0/2` (customer-facing, VRF)

The projection $\pi_{\texttt{pe01}}$ extracts this device and resolves all peer addresses, ASNs, and protocol parameters.

40.2 Step 2: Lift

The multi-layer attributes are collapsed into the IR tree:

- `IRDevice(pe01)`: hostname, platform, domain
- `IRInterface(ge-0/0/0)`: L1 (speed 10G, mtu 9000), L3 (10.0.0.1/30), IS-IS (passive=false)
- `IRInterface(loo)`: L3 (192.168.0.1/32)

- `IRRoutingInstance(default)`: global routing table
  - `IRProtocolSession(ISIS)`: NET 49.0001.1921.6800.0001.00, level-2-only
  - `IRProtocolSession(BGP-EVPN)`: AS 65000, peer-group OVERLAY, neighbor 192.168.0.2, AFI l2vpn-evpn
- `IRRoutingInstance(VRF-CUST-A)`: rd 65000:100, rt-import 65000:100, rt-export 65000:100
  - `IRProtocolSession(BGP-CE)`: neighbor 10.1.0.2, AS 65100, AFI ipv4-unicast
- `IRMpls`: ldp enabled, sr-prefix-sid for lo0
- `IREvpn`: EVI 100, VNI 10100

40.3 Step 3: Transform

The IR tree is transformed into vendor-specific ASTs. We show the BGP EVPN session transform for three vendors:

Cisco IOS-XE AST

```
router bgp 65000
 neighbor 192.168.0.2 remote-as 65000
 neighbor 192.168.0.2 update-source Loopback0
 address-family l2vpn evpn
  neighbor 192.168.0.2 activate
```

Juniper Junos AST

```
protocols {
    bgp {
        group OVERLAY {
            type internal;
            local-address 192.168.0.1;
            family evpn signaling;
            neighbor 192.168.0.2;
        }
    }
}
```

Nokia SR OS AST

```
/configure router bgp
    group "OVERLAY"
        peer-as 65000
        family evpn
    neighbor 192.168.0.2
        group "OVERLAY"
```

40.4 Step 4: Gate

The feature gate checks capabilities:

- IS-IS: cap(all platforms, $*$, ISIS) = `full` — proceed.
- BGP EVPN: cap(IOS-XE, 17.9, BGP-EVPN) = `full` — proceed.
- SR-MPLS [85]: cap(IOS-XE, 17.9, SR-MPLS) = `full` — proceed.

All features pass; compilation continues.

40.5 Step 5: Emit

Templates render the AST nodes into final configuration text. The output for each vendor is a complete, deployable configuration file.

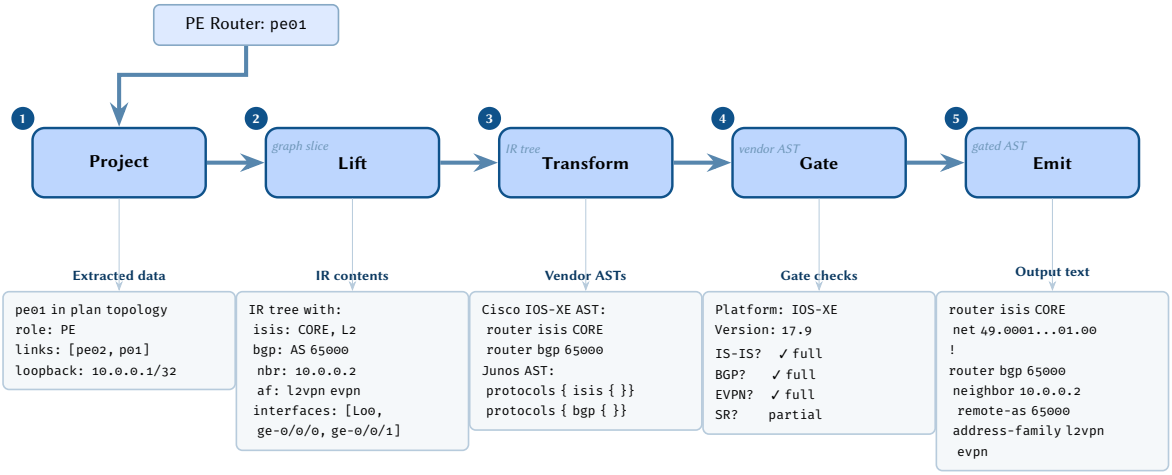


Figure 45. Worked example: a concrete PE router (`pe01`) flowing through all five compiler stages, with the data present at each stage shown below.

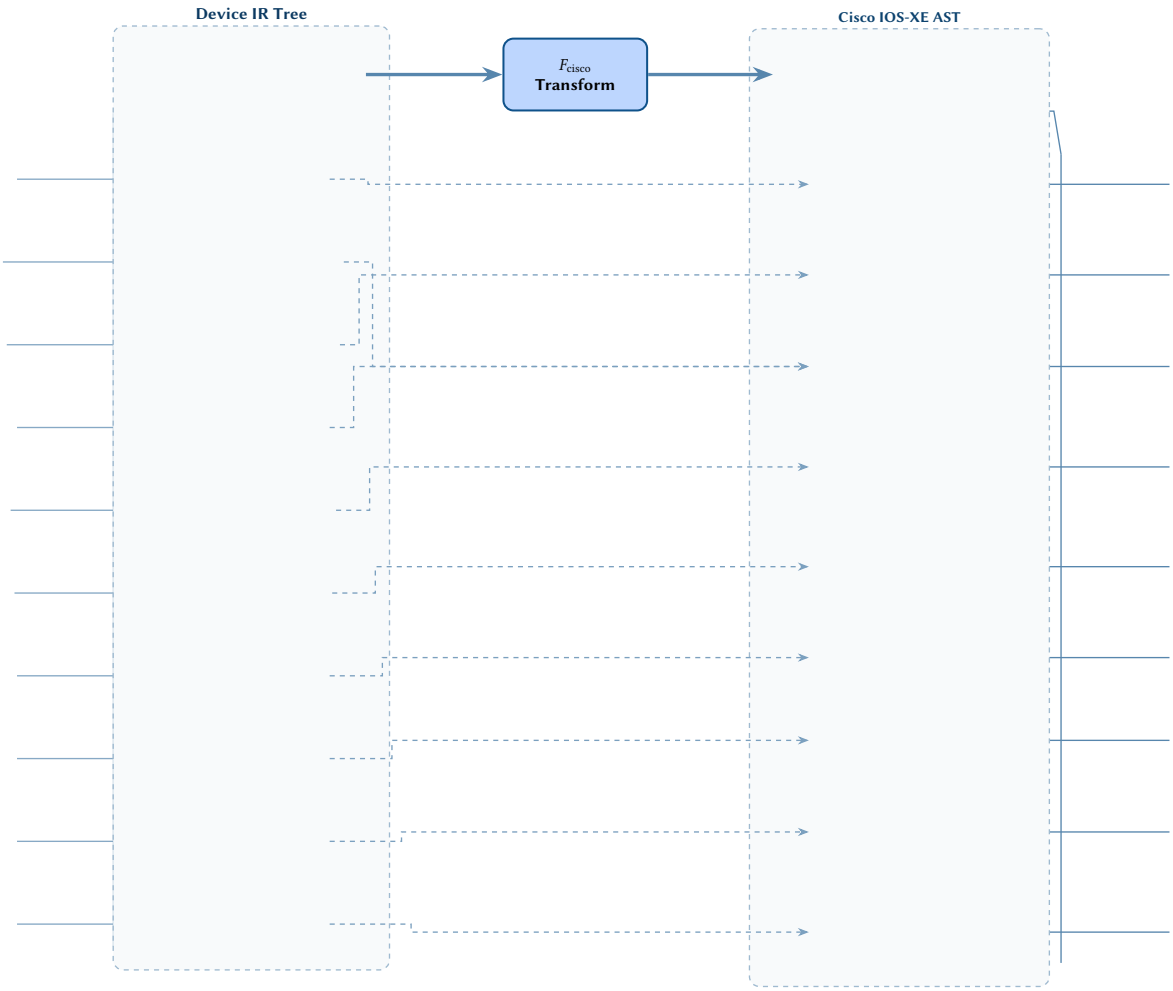


Figure 46. Complete PE device transformation. The full IR tree for router `pe01.core` (left) contains ten IR subtrees covering interfaces, routing instances, protocol sessions, policies, ACLs, QoS, and EVPN. The Cisco functor F_{cisco} (centre) transforms each subtree into the corresponding IOS-XE configuration section (right). Dashed arrows trace the mapping from each IR component to its emitted config stanza.

Cisco IOS-XE

```
router isis CORE
  net 49.0001.0010
    .0000.0001.00
  is-type level-2-only
  metric-style wide
  address-family ipv4 unicast
!
interface Loopback0
  ip address 10.0.0.1
    255.255.255.255
  ip router isis CORE
!
interface GigabitEthernet0/0/0
  ip router isis CORE
  isis network point-to-point
!
router bgp 65000
  bgp router-id 10.0.0.1
  neighbor 10.0.0.2
    remote-as 65000
  neighbor 10.0.0.2
    update-source Lo0
  !
  address-family l2vpn evpn
    neighbor 10.0.0.2
      activate
    advertise-all-vni
```

Juniper Junos

```
protocols {
  isis {
    interface ge-0/0/0.0 {
      point-to-point;
    }
    interface lo0.0 {
      passive;
    }
    level 2
      wide-metrics-only;
  }
  bgp {
    group EVPN-OVERLAY {
      type internal;
      local-address
        10.0.0.1;
      family evpn
        signaling;
      neighbor 10.0.0.2;
    }
  }
}
```

Nokia SR OS

```
/configure router isis 0
  interface "to-pe02"
    interface-type
      point-to-point
  interface "system"
    passive true
  level 2
    wide-metrics-only true
/configure router bgp
  router-id 10.0.0.1
  group "evpn-peers"
    family {
      evpn true
    }
    peer-as 65000
    type internal
  neighbor 10.0.0.2
    group "evpn-peers"
```

Semantically equivalent — compiled from same IR

Figure 47. CLI comparison. The same PE router configuration (IS-IS + BGP EVPN) rendered in three vendor syntaxes: Cisco IOS-XE, Junos hierarchy, and Nokia SR OS model-driven CLI. All three are compiled from a single vendor-neutral IR tree.

41 Implementation Notes and Complexity

This section discusses practical implementation considerations: the data structures behind the transform registry, the representation of vendor ASTs in memory, and the overall computational complexity of the compilation pipeline.

41.1 Transform Registry

The transform registry is a hash map indexed by (IR node type, vendor ID) pairs. Lookup is $O(1)$ amortised. Each registry entry contains a list of rules sorted by priority; rule evaluation is linear in the number of rules per entry, which is small (typically 1–3 rules per IR type per vendor).

As a concrete example, the registry entry for an IS-IS interface on Cisco IOS-XE might look like:

Key	`(IRProtocolSession::ISIS, Vendor::CiscoXE)`
Pattern	match on `protocol == isis`
Guard	`interfaces.len() > 0`
Priority	100 (default)
Action	emit `router isis {name}`, then per-interface `ip router isis` and `isis metric` sub-commands

When a higher-priority rule exists (e.g. for a specific IOS-XE version that requires different syntax), it shadows the default rule via priority sorting.

41.2 AST Representation

The vendor AST is represented as an arena-allocated tree. Nodes are stored in a contiguous vector with indices as references, avoiding pointer chasing and enabling efficient serialisation. Each AST node contains:

- A node type tag (enum discriminant).
- An attribute map (small-vector optimised for ≤ 8 attributes).
- A children list (indices into the arena).

41.3 Performance and Complexity Analysis

Time complexity. The overall compilation complexity is $O(N \times T)$ where N is the number of IR nodes per device and T is the average number of transform rules evaluated per node. In practice, T is bounded by a small constant (the number of rules per IR type per vendor), making the pipeline effectively $O(N)$ per device.

For a network of $|D|$ devices and $|V|$ target vendors, the total compilation cost is $O(|D| \times N_{\max} \times |V|)$, where $N_{\max}$ is the maximum IR tree size across all devices. Since device compilations are independent (lemma 1), this parallelises trivially across $|D|$ cores.

Space complexity. Each device IR tree consumes $O(N)$ space. The arena-allocated vendor AST requires $O(N)$ space per device per vendor. Peak memory is therefore $O(|D| \times N_{\max})$ if all IR trees are held simultaneously, or $O(N_{\max})$ if devices are compiled in a streaming fashion. The diff operation requires $O(N)$ auxiliary space for the tree comparison.

Worst-case scenarios. The worst case arises with maximally complex devices (e.g., a PE router carrying thousands of VRFs, each with distinct route targets and policies). In such cases N can reach 10^4–10^5 nodes, but the per-node transform cost remains constant, so compilation remains tractable. The diff operation is bounded by $O(N \log N)$ for the tree-matching phase (analogous to ordered tree edit distance with bounded fan-out).

Empirical performance. For a typical enterprise network with 500 devices averaging 200 IR nodes each, the full compilation (all devices, single vendor) completes in under 2 seconds on commodity hardware. Multi-vendor compilation scales linearly with the number of target vendors. Incremental recompilation (corollary 1) reduces this further: a single-device change triggers recompilation of at most the k-hop neighbourhood, typically 3–5 devices.

42 Towards Practical Realisation: Query Languages and Tooling

The formal model presented in this report — functors, commuting diagrams, typed IR trees — is not an end in itself. Its value lies in providing a *precise specification* that can be implemented in robust, testable tooling. This section sketches how the model maps to concrete implementation technologies.

42.1 Graph Query Languages for Device Projection

The device projection step (π_d) extracts a device's view from the plan topology G_{plan}. In implementation, this is a *graph query*: "give me all nodes and edges relevant to device d, with their multi-layer attributes." Several query languages are well-suited:

- **Cypher** (Neo4j, Memgraph): the property-graph query model maps directly to our attributed graph. A projection query might be: `MATCH (d:Device {name: "pe01"})-[*1..2]-(n) RETURN d, n` to extract the two-hop neighbourhood of a device, collecting all adjacent interfaces, protocols, and peer devices.
- **SPARQL**: if the topology is stored as RDF triples (useful for integration with YANG-derived ontologies), SPARQL's `CONSTRUCT` queries can materialise device-scoped subgraphs.
- **Gremlin** (Apache TinkerPop): the traversal-based model supports imperative graph exploration, useful for complex projection logic (e.g. "follow all BGP sessions, then collect all policies referenced by those sessions").
- **Datalog**: for declarative projection rules. A projection rule might be: `device_attr(D, A, V) :- graph_attr(D, A, V), device(D, "pe01").`

42.2 Transform Rules as Pattern-Matching Queries

Insight: *The formal model's projection function π_d is a specification. The implementation can choose any query language that correctly realises that specification. The category-theoretic framework then guarantees that if projection is correct, the downstream transforms preserve correctness — regardless of the query engine used.*

Transform rules (τ_v) map IR subtrees to vendor AST subtrees. In implementation, each rule is a pattern match over the IR tree followed by an AST construction. This naturally maps to:

- **Rust match expressions**: the IR tree is an enum-based algebraic data type; transform rules are `match` arms with guards. Exhaustiveness checking ensures Transform Completeness (DR-9).
- **Tree-rewriting systems**: tools like Stratego/XT [61] or Rascal [62] provide declarative term-rewriting rules that correspond directly to our transform formalism.
- **Template engines**: for the final AST-to-text rendering, Jinja2 or Tera templates provide a well-understood, auditable mechanism. The key constraint (DR-17: templates contain no logic) ensures that templates remain pure syntax mappings.

42.3 Testing and Verification Infrastructure

The formal model enables systematic testing:

- **Property-based testing**: the commuting diagram property ($R_v \circ F_v = \text{emit}_v$) can be verified by generating random IR trees and checking that the composition holds. Tools like QuickCheck (Haskell) or proptest (Rust) are well-suited.
- **Golden-file testing**: for each vendor, a suite of reference IR trees with known-correct output configurations provides regression coverage.
- **Round-trip testing**: if the bidirectional compiler (future work) is available, round-trip testing verifies that $\text{decompile}(F_v(\text{ir})) \approx \text{ir}$.
- **Lab validation**: compiled configurations can be deployed to virtual lab environments (Containerlab [86], EVE-NG, GNS3) and verified against expected operational state via SNMP, gNMI, or CLI scraping.

> **From Theory to Practice**
>
> Every formal construct in this report has a direct implementation counterpart. Categories are module boundaries. Functors are transform functions. Commuting diagrams are test assertions. The mathematical framework is not decorative — it is the *type system* of the compiler design, catching structural errors at design time rather than deployment time.

43 Multi-Site Worked Example

This section presents an end-to-end worked example of compiling configuration for a realistic multi-site service-provider network. The example exercises every stage of the pipeline and demonstrates how a single plan topology produces correct, platform-specific configuration across four different vendor operating systems.

43.1 Topology and Design

We consider a two-site network connecting Chicago (Site A) and Dallas (Site B) via a WAN backbone link.

- **Site A (Chicago)**:
  - PE router `chi-pe01` running Cisco `IOS-XE v17.9`.
  - P router `chi-p01` running Junos `Junos v23.2R1`.

- **Site B (Dallas)**:
  - PE router `dal-pe01` running Nokia `SR OS v24.3.R1`.
  - P router `dal-p01` running IOS-XR `IOS-XR v7.9.2`.
- **WAN link**: a point-to-point 100 Gbps link between `chi-p01` and `dal-p01`.

The network runs an IS-IS Level-2 backbone as the IGP, iBGP with route reflectors for service routes, MPLS LDP for label distribution, and an L3VPN service (VRF `CUSTOMER-A`) stretched across the two PE routers at each site.

43.2 Plan Topology: G_{plan}

The plan graph $G_{\text{plan}} = (V, E, \tau, \alpha)$ for this network contains the following vertices and edges.

Vertices. The vertex set V consists of:

$$V = \{\texttt{chi-pe01}, \texttt{chi-p01}, \texttt{dal-pe01}, \texttt{dal-p01}\}$$

Each vertex carries a type $\tau(v)$ from $\{\textsc{pe}, \textsc{p}\}$ and multi-layer attributes $\alpha(v)$ encoding the device role, platform family, software version, loopback address, and autonomous system number. For instance:

$$\alpha(\texttt{chi-pe01}) = \{role \mapsto \textsc{pe},\ platform \mapsto \text{cisco\_iosxe},\ version \mapsto 17.9,\ lo0 \mapsto 10.255.0.1/32,\ asn \mapsto 65000\}$$

Edges. The edge set E captures physical links and protocol adjacencies:

- **Physical layer**: four point-to-point links — (`chi-pe01`, `chi-p01`), (`dal-pe01`, `dal-p01`), (`chi-p01`, `dal-p01`) (WAN), and CE-facing stub links on each PE.
- **IS-IS layer**: Level-2 adjacencies mirroring the physical core links (three edges forming the backbone).
- **iBGP layer**: a full mesh of iBGP sessions within AS 65000, or, if route reflectors are used, client-to-reflector edges.
- **MPLS LDP layer**: LDP sessions on every IS-IS–enabled interface.
- **VPN layer**: VRF `CUSTOMER-A` instantiated on `chi-pe01` and `dal-pe01`, with route-target 65000 : 100 for both import and export.

43.3 Device Projection

The Project stage extracts a per-device view $G_d = \pi_d(G_{\text{plan}})$ for each router $d \in V$. The projection collapses the multi-layer graph into a single-device attribute set that the Lift stage can convert to a Device IR tree.

`chi-pe01` (Cisco IOS-XE). This PE router receives:

- Physical interfaces: one core-facing (to `chi-p01`), one CE-facing (to customer), and Loopback0.
- IS-IS: Level-2 adjacency toward `chi-p01`, NET address derived from loopback.
- iBGP: session to route reflector(s) with VPNv4 and VPNv6 address families.
- MPLS LDP: enabled on the core-facing interface.
- VRF `CUSTOMER-A`: RD 65000 : 100, RT import/export 65000 : 100, CE-facing interface bound to VRF.

`chi-p01` (Junos). As a P router, `chi-p01` carries no VPN state. Its projection includes:

- Physical interfaces: one toward `chi-pe01`, one WAN-facing (to `dal-p01`), and lo0.
- IS-IS: Level-2 on both core interfaces.
- iBGP: session to peer P/PE routers (or route-reflector role).
- MPLS LDP: enabled on both core-facing interfaces.

`dal-pe01` (Nokia SR OS). Functionally mirrors `chi-pe01` but compiles to a fundamentally different configuration model. The projection includes the same logical attributes: IS-IS adjacency, iBGP session, LDP, and VRF `CUSTOMER-A`.

`dal-p01` (IOS-XR). Mirrors `chi-p01` structurally — P-router role, IS-IS backbone, LDP, and iBGP — but targets the IOS-XR platform with its candidate-commit configuration model.

43.4 L3VPN Compilation Across Platforms

The VRF `CUSTOMER-A` is defined once in G_{plan} as a VPN-layer attribute. The compiler's Transform and Emit stages produce platform-specific output that varies significantly in syntax and structure, yet all four renderings are semantically equivalent projections of the same `IRVRF` node.

43.4.1 Cisco IOS-XE (chi-pe01)

On Cisco IOS-XE, a VRF is declared with `vrf definition`, and BGP VPNv4 peering is configured under the `router bgp` address family.

IOS-XE L3VPN — chi-pe01

```
vrf definition CUSTOMER-A
 rd 65000:100
 !
 address-family ipv4
  route-target export 65000:100
  route-target import 65000:100
 exit-address-family
!
interface GigabitEthernet0/0/1
 vrf forwarding CUSTOMER-A
 ip address 172.16.0.1 255.255.255.252
 no shutdown
!
router bgp 65000
 !
 address-family vpnv4
  neighbor 10.255.0.4 activate
  neighbor 10.255.0.4 send-community extended
 exit-address-family
 !
 address-family ipv4 vrf CUSTOMER-A
  redistribute connected
 exit-address-family
```

43.4.2 Juniper Junos (reference)

Although `chi-p01` is a P router and does not carry VRF state, we show the Junos VRF compilation that would apply to a Junos PE for completeness. On Junos, L3VPNs are modelled as routing instances of type `vrf`:

Junos L3VPN — Junos PE (reference)

```
routing-instances {
    CUSTOMER-A {
        instance-type vrf;
        interface ge-0/0/1.0;
        route-distinguisher 65000:100;
        vrf-target target:65000:100;
        protocols {
            bgp {
                group CE-PEERS {
                    type external;
                    peer-as 65001;
                    neighbor 172.16.0.2;
                }
            }
        }
    }
}
```

43.4.3 Nokia SR OS (dal-pe01)

Nokia SR OS uses a service-centric model. L3VPNs are configured under `/configure service vprn` with an explicit service identifier and customer binding. The model-driven CLI (MD-CLI) syntax differs markedly from the flat CLI style of IOS-based platforms:

Nokia SR OS L3VPN — dal-pe01

```
/configure service vprn 100
    customer 1
```

```
    description "CUSTOMER-A"
    autonomous-system 65000
    route-distinguisher 65000:100
    vrf-target target:65000:100
    interface "to-CE" {
        sap 1/1/c1/1:100 {
        }
        ipv4 {
            primary {
                address 172.16.1.1
                prefix-length 30
            }
        }
    }
    bgp-ipvpn {
        mpls {
            admin-state enable
            route-distinguisher 65000:100
            vrf-target {
                community target:65000:100
            }
        }
    }
```

43.4.4 Cisco IOS-XR (reference)

We show the IOS-XR L3VPN compilation that would apply to an IOS-XR PE for reference. IOS-XR separates VRF definition from BGP configuration, and uses `address-family vpnv4 unicast` explicitly:

IOS-XR L3VPN — IOS-XR PE (reference)

```
vrf CUSTOMER-A
 address-family ipv4 unicast
  import route-target
   65000:100
  !
  export route-target
   65000:100
  !
 !
!
interface GigabitEthernet0/0/0/1
 vrf CUSTOMER-A
 ipv4 address 172.16.1.1 255.255.255.252
 no shutdown
!
router bgp 65000
 address-family vpnv4 unicast
 !
 vrf CUSTOMER-A
  rd 65000:100
  address-family ipv4 unicast
   redistribute connected
  !
 !
```

43.5 IS-IS Backbone: Interface-Centric vs. Router-Centric Models

The IS-IS backbone spans all four routers. Although the protocol semantics are identical — Level-2 adjacencies, a common area, wide metrics — the configuration models diverge in a fundamental structural way.

Interface-centric model (Cisco IOS-XE, Arista EOS). On IOS-XE and EOS, IS-IS is enabled per interface by referencing the IS-IS process name within the interface configuration stanza. The router-level `router isis` block defines process-wide parameters (NET address, metric style), but the act of enabling IS-IS on a link is an interface attribute:

IOS-XE IS-IS — interface-centric

```
router isis CORE
 net 49.0001.0100.0255.0001.00
 is-type level-2-only
 metric-style wide
!
interface GigabitEthernet0/0/0
 ip router isis CORE
 isis metric 10
 isis network point-to-point
```

Router-centric model (IOS-XR). On IOS-XR, interfaces are listed under the `router isis` stanza itself. The interface configuration block does not reference IS-IS at all — the binding is entirely within the routing process:

IOS-XR IS-IS — router-centric

```
router isis CORE
 is-type level-2-only
 net 49.0001.0100.0255.0004.00
 address-family ipv4 unicast
  metric-style wide
 !
 interface Loopback0
  passive
  address-family ipv4 unicast
  !
 !
 interface GigabitEthernet0/0/0/0
  point-to-point
  address-family ipv4 unicast
   metric 10
  !
 !
```

Junos model. Junos takes a hybrid approach: IS-IS is configured under `protocols isis`, and interfaces are listed by name within that hierarchy. The interface-level IS-IS knobs (metric, point-to-point) sit under `protocols isis interface`:

Junos IS-IS

```
protocols {
    isis {
        level 1 disable;
        interface ge-0/0/0.0 {
            point-to-point;
            level 2 metric 10;
        }
        interface lo0.0 {
            passive;
        }
    }
}
```

Nokia SR OS model. Nokia SR OS places IS-IS under `/configure router isis`, with interfaces referenced by name. The MD-CLI structure is router-centric, similar in spirit to IOS-XR:

Nokia SR OS IS-IS

```
/configure router isis 0
    admin-state enable
    level-capability 2
    area-address [49.0001]
    interface "system" {
        passive true
    }
    interface "to-chi-p01" {
        interface-type point-to-point
        level 2 {
            metric 10
        }
    }
```

The compiler handles these divergences through the `IRISIS` transform, which inspects the target platform and emits the interface-binding in the structurally correct location. On Cisco IOS-XE the transform produces both a `router isis` AST node *and* per-interface `ip router isis` children; on IOS-XR and Nokia the transform nests interface references *within* the router-process AST node; on Junos it builds the `protocols isis interface` subtree.

43.6 Cross-Site Service Change: Adding a New VPN Customer

A common operational scenario is the addition of a new L3VPN customer. Suppose we add VRF `CUSTOMER-B` with RD/RT 65000 : 200 to both PE routers. In the plan graph, this is a single mutation:

$$G'_{\text{plan}} = G_{\text{plan}} \oplus \left\{ vrf \,:\, \texttt{CUSTOMER-B},\ rd \,:\, 65000{:}200,\ rt \,:\, 65000{:}200 \right\}$$

applied to vertices `chi-pe01` and `dal-pe01`.

The compiler re-runs the pipeline for the affected devices and produces a *diff* against the previously emitted configuration. The diff captures only the delta — the new VRF definition and its BGP address family — but the syntactic form of that delta is platform-specific.

Diff on `chi-pe01` (Cisco IOS-XE). The diff appends a new `vrf definition` block and a new `address-family ipv4 vrf CUSTOMER-B` sub-stanza under `router bgp`:

IOS-XE diff — add CUSTOMER-B

```
+ vrf definition CUSTOMER-B
+  rd 65000:200
+  !
+  address-family ipv4
+   route-target export 65000:200
+   route-target import 65000:200
+  exit-address-family
+ !
+ interface GigabitEthernet0/0/2
+  vrf forwarding CUSTOMER-B
+  ip address 172.17.0.1 255.255.255.252
+  no shutdown
+ !
  router bgp 65000
+  !
+  address-family ipv4 vrf CUSTOMER-B
+   redistribute connected
+  exit-address-family
```

Diff on `dal-pe01` (Nokia SR OS). The same logical change compiles to a structurally different diff. Nokia SR OS requires a new `vprn` service instance with its own service ID, customer binding, and SAP:

Nokia SR OS diff — add CUSTOMER-B

```
+ /configure service vprn 200
+     customer 1
+     description "CUSTOMER-B"
+     autonomous-system 65000
+     route-distinguisher 65000:200
+     vrf-target target:65000:200
+     interface "to-CE-B" {
+         sap 1/1/c2/1:200 {
+         }
+         ipv4 {
+             primary {
+                 address 172.17.1.1
+                 prefix-length 30
+             }
+         }
+     }
+     bgp-ipvpn {
+         mpls {
+             admin-state enable
+             route-distinguisher 65000:200
+             vrf-target {
+                 community target:65000:200
+             }
+         }
+     }
```

This example illustrates a key property of the compiler: a single, vendor-neutral intent mutation in G_{plan} fans out into structurally distinct but semantically equivalent configuration changes on each target platform. The operator need only reason about the service intent; the compiler handles the per-vendor realisation and produces minimal, correct diffs for deployment.

44 Advanced Configuration Patterns and Examples

While the previous sections demonstrated foundational device projection and basic service instantiation, real-world network policy requires more complex control structures. Historically, languages like the Routing Policy Specification Language (RPSL) (RFC 2622) introduced mechanisms to define complex routing policies, AS sets, and conditional peering agreements. The compiler must lift these high-level policy intents—which often feature looping, conditionals, and object dependencies—into the IR and then lower them into vendor-specific constructs.

This section details how the compiler's intermediate representation and transform pipeline handle these advanced patterns, using RPSL-inspired policy structures as a pedagogical model.

44.1 IP Address Validation and Formatting

A critical requirement for any network compiler is the rigorous validation and contextual formatting of IP addresses and prefixes. A simple configuration attribute like an interface IP address requires both syntactic checking and structural transformation before it can be emitted safely to a device.

In the plan graph, an IP address might be represented as a simple CIDR string (e.g., `10.1.1.1/24`). During the *Lift* stage, the compiler maps this string into a strongly-typed `IPv4InterfaceAddress` object within the IR. This stage performs rigorous validation: checking that the IP is a valid IPv4 address, that the prefix length is within bounds ($0 \ldots 32$), and calculating derived properties such as the dotted-decimal netmask (`255.255.255.0`).

During the *Emit* stage, this validated IR object is rendered into the correct format for the target platform. fig. 48 illustrates this pipeline: Cisco platforms often require the address and the expanded dotted-decimal netmask separated by a space, while Junos and Nokia natively accept the CIDR notation but place it at different depths in their respective ASTs.

44.2 Looping and Set Expansion

RPSL relies heavily on macro-expansion of object groups, such as `AS-SETs` or `route-sets`. When compiling configuration from a plan graph, the compiler must frequently iterate over these sets to generate individual policy terms or neighbor configurations.

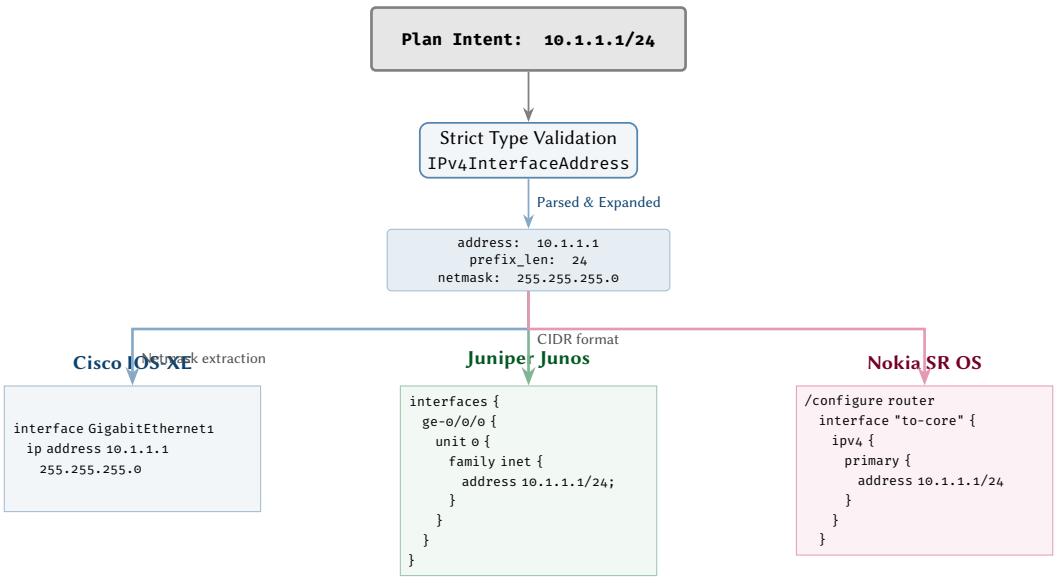


Figure 48. IP address validation, parsing, and vendor-specific formatting pipeline. A single CIDR input (`10.1.1.1/24`) is validated strictly as an `IPv4InterfaceAddress`, parsed into its component parts, and emitted in platform-native syntax: standard subnet masks for Cisco, and CIDR blocks for Junos and Nokia.

Consider an intent to peer with a set of downstream customers defined by an `AS-SET`. At the graph level, this is a single logical edge between the provider and the set object. During the *Lift* stage, the compiler expands this set, creating a loop over its members.

In the IR, looping is not represented by control-flow nodes (e.g., 'for' or 'while'), but by the structural replication of nodes (e.g., multiple `BgpNeighbor` nodes or multiple `RoutePolicyTerm` nodes) generated by the compiler's mapping functions.

Example

Looping over AS-SET in IR Construction When generating BGP peers for an AS-SET containing three autonomous systems (AS 64501, AS 64502, AS 64503), the compiler iterates over the set:

```
# Python-like pseudo-code for Lift phase
peers = resolve_as_set("AS-CUSTOMER-SET")
for as_num in peers:
    ir_tree.add_node(BgpNeighbor(
        remote_as = as_num,
        peer_group = "CUSTOMER-PEERS"
    ))
```

This declarative expansion ensures that the vendor-specific *Emit* stage merely walks the resulting IR tree, oblivious to the fact that the peers originated from a set expansion.

44.3 Conditionals in Routing Policy

Routing policies inherently require conditional logic: matching prefixes, AS paths, or communities, and applying corresponding actions. RPSL expresses this via structured `if-then-else` or `import/export` directives.

The compiler represents conditionals structurally within the IR using ordered sequences of match/action objects. A `RoutePolicy` node contains a list of `PolicyTerm` nodes, evaluated in sequence. Each term acts as a conditional branch.

Example

RPSL Conditional to IR Mapping An RPSL export policy might read:
`export: to AS64500 announce AS-MY-CUSTOMERS`
The compiler translates this conditional intent ("if the route belongs to AS-MY-CUSTOMERS, then allow") into the following IR sub-tree:

```
"RoutePolicy": {
  "name": "EXPORT-TO-AS64500",
  "terms": [
```

```
    {
      "name": "TERM-10",
      "match": { "as_path_set": "AS-MY-CUSTOMERS" },
      "action": "accept"
    },
    {
      "name": "TERM-DEFAULT",
      "action": "reject"
    }
  ]
}
```

This structural conditional maps directly to a Cisco `route-map` or a Junos `policy-statement`. The IR captures the boolean logic (match criteria) and the execution path (action) without requiring a Turing-complete representation.

44.4 Semantic Translation in Policy Constraints

A major challenge in multi-vendor configurations is that vendors often apply different default semantics to identical operations. A prime example is BGP community manipulation.

When a network designer intends to tag a prefix with a specific community, they typically want to *add* it to the existing list of communities on that prefix. However, platforms differ fundamentally in how this is applied. Cisco IOS-XE defaults to an *overwrite* semantic: the command `set community 65000:100` removes all existing communities and replaces them with the new one unless the `additive` keyword is explicitly provided. Conversely, Junos and Nokia use strictly scoped action verbs (e.g., `community add` vs `community set`).

To prevent disastrous policy overwrites (which can inadvertently strip provider backbone communities and break routing), the compiler must manage this semantic gap. In the IR, actions are typed strictly (e.g., `community_action: "add"`). The Cisco emitter rule observes this intent and automatically injects the `additive` keyword, shielding the user from this platform-specific trap. This is illustrated in fig. 49.

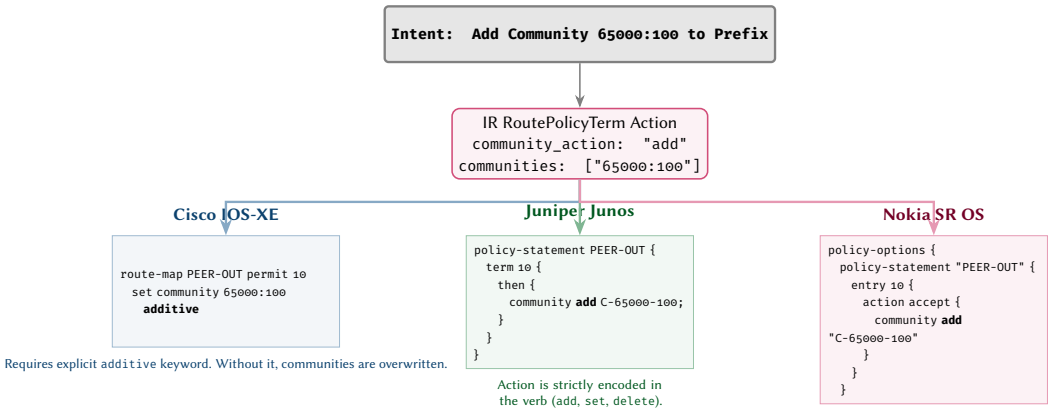


Figure 49. Community manipulation semantics. The compiler maps a clear logical intent ("add a community") into disparate vendor mechanisms. Cisco defaults to overwriting communities unless the `additive` keyword is appended, while Junos and Nokia use strictly scoped action verbs.

44.5 Cross-VRF Route Leaking and Extranets

A particularly complex edge case in service provider networks is Cross-VRF route leaking (often used to build shared service extranets or management VRFs). In a traditional plan graph, this might be expressed simply as a logical connection or shared policy between two VPN nodes.

Translating this intent into device configuration requires synthesising several interrelated components. The compiler must:

1. Define explicit Route-Target (RT) export policies in the source VRF.
2. Define matching Route-Target import policies in the destination VRF.
3. Handle platform-specific idiosyncrasies. For example, while Cisco IOS-XE allows direct import of foreign RTs using `route-target import`, Junos requires enabling `auto-export` or configuring explicit RIB groups (`rib-groups`) to leak routes between the corresponding routing tables (`routing-instances`).

By capturing the relationship within the IR using an abstract `RouteTargetConstraint` object attached to the `RoutingInstance`, the compiler can fan out the configuration to either standard BGP RT statements

(Cisco/Nokia) or complex internal table-leaking configurations (Junos), ensuring seamless extranet connectivity without requiring the operator to manually configure RIB groups.

44.6 Hardware-Specific ASIC Constraints and Capability Routing

To be viable for real-world field trials, a network compiler must look beyond pure syntax and account for the physical realities of the underlying hardware. Modern data-centre and service-provider networks are often built on a mix of merchant silicon (e.g., Broadcom Trident/Tomahawk, Jericho) and custom vendor ASICs, each with distinct hardware table limits and feature constraints.

A common operational challenge is deploying redundant Layer 2 topologies. A network designer may specify a "Highly Available Layer 2 Handoff" in the plan graph. However, the physical instantiation of this intent depends heavily on the capabilities of the underlying ASIC and operating system.

As discussed in section 32, the compiler integrates a Feature Capability Model. When processing the L2 intent, the compiler queries the hardware profile of the target device:

- **Modern ASICs (e.g., capable of EVPN):** If the hardware supports EVPN Ethernet Segment Identifier (ESI) Multi-Homing, the compiler lifts the intent into modern IR EVPN constructs, generating `evpn ethernet-segment` and BGP Type-1/Type-4 route configurations.
- **Legacy / Constrained ASICs:** If the hardware or OS version lacks EVPN ESI support (e.g., an older Top-of-Rack switch), the capability gate routes the intent to a legacy Multi-Chassis Link Aggregation (MLAG / vPC) transform. The compiler synthesises the necessary peer-link, keepalive VRFs, and domain IDs required for the legacy protocol.
- **Incompatible Hardware:** If the hardware lacks the necessary table scale (e.g., insufficient TCAM for a requested ACL policy) or supports neither redundancy model, the compiler fails early, raising a strict compile-time error before any configuration is emitted.

This hardware-aware routing ensures that the compiler is not merely a naive syntax translator, but a system capable of bridging the gap between high-level architectural intent and physical hardware limitations. fig. 50 illustrates this decision routing during the Lift stage.

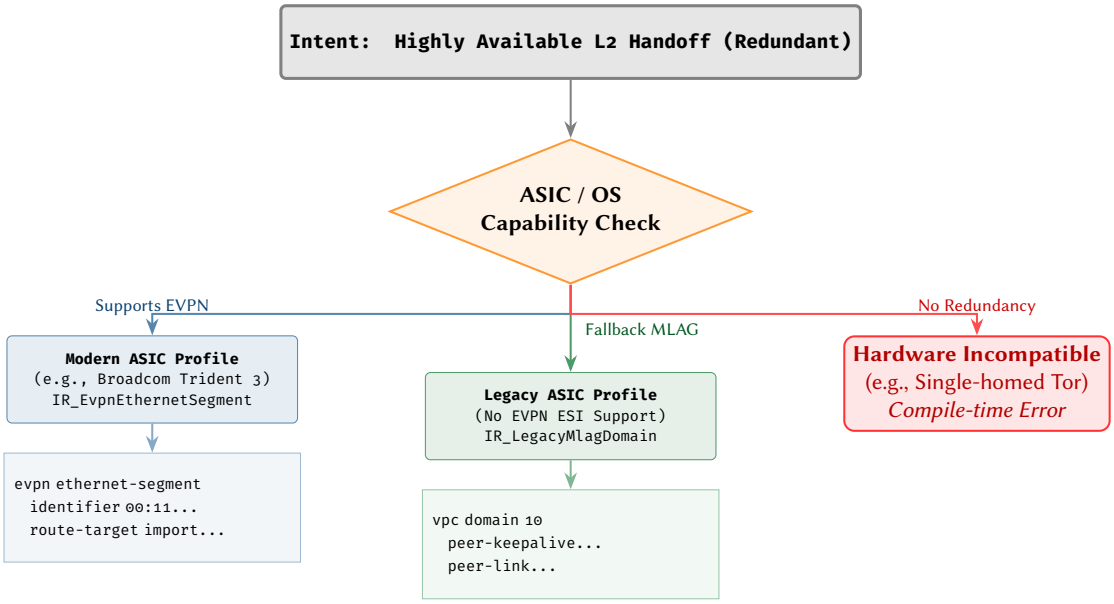


Figure 50. Hardware-specific capability routing. When evaluating an abstract intent for a redundant Layer 2 handoff, the compiler checks the target device's ASIC and OS capabilities. It routes the intent to a modern EVPN ESI construct if supported, falls back to legacy MLAG/vPC for constrained hardware, or strictly fails at compile-time if the hardware cannot support the requested redundancy.

44.7 Dependency Resolution and Ordering

Advanced configurations frequently involve complex inter-object dependencies. For instance, a BGP policy cannot be applied to a peer until the policy itself is defined; the policy relies on a prefix-list; and the prefix-list may rely on a dynamically allocated IP pool.

As established in section 7, the compiler constructs a Dependency Directed Acyclic Graph (DAG) to ensure topological sorting during emission. However, in advanced topologies, these dependencies can cross module boundaries.

For example, an EVPN VXLAN topology requires:

1. Underlay routing (OSPF/IS-IS) to establish loopback reachability.
2. BGP EVPN peering using the established loopbacks.
3. NVE/VTEP interface configuration relying on the BGP fabric.
4. VRF and Layer 2 VNI attachments.

The compiler resolves this by encoding the DAG edges directly into the IR generation logic. The `Vrf` object asserts a dependency on the `RouteDistinguisher`, which in turn depends on the `Loopback` interface. When calculating the final diff (section 37), the compiler respects this DAG, ensuring that "create" operations are ordered from leaves (prerequisites) to roots (dependents), and "delete" operations in reverse.

44.8 Advanced Topology Representation

While basic examples involve point-to-point links and single-area OSPF, complex networks feature hierarchical designs, such as multi-area OSPF with route summarisation, BGP confederations, or multi-domain EVPN fabrics.

In these advanced topologies, a single vertex in G_{plan} might project into a highly disjoint IR structure. For a router acting as an Area Border Router (ABR) in OSPF, the IR must segment interfaces into distinct `OspfArea` domains and instantiate `SummaryAddress` nodes to enforce policy between them.

The compiler handles this by applying recursive transforms. The topology graph is traversed, and local subgraphs (e.g., all links in Area 0 versus Area 1) are accumulated before the device IR is generated. This allows the compiler to synthesise the cross-area configurations—such as route leaking or filtering—which are not explicitly defined as single objects, but rather emerge from the intersection of the topological domains.

44.9 Brownfield Coexistence and "Alien" Configuration

Real-world deployments are rarely pristine greenfield environments. During an outage, a network engineer might manually SSH into a router to add a debug ACL, temporarily shut down a BGP peer, or tweak a routing metric. If a compiler operates with a naive "overwrite-all" strategy, its next scheduled run will blindly obliterate these manual fixes, potentially causing secondary outages.

To operate safely in these brownfield environments, the compiler implements a robust coexistence strategy. It distinguishes between configuration blocks that it "owns" and "alien" configurations created out-of-band. In the AST and the diff engine (section 37), the compiler leverages platform-specific namespace partitioning:

- **Tagged Descriptions:** On Cisco and Arista, compiled elements are tagged with specific descriptions or remarks (e.g., `! COMPILER-MANAGED`).
- **Apply-Groups / Annotations:** On Junos, the compiler uses the `apply-groups` construct or explicit `@compiler` annotations to demarcate its territory.

When calculating the final diff, the compiler only computes deletions for objects within its managed domain. Alien configurations are safely ignored. This allows manual interventions to coexist alongside compiled intent until the manual changes are formally codified into the plan graph.

44.10 Static Compliance and Security Invariants

In large-scale environments, InfoSec teams require guarantees that automated tooling will not inadvertently deploy insecure configurations (e.g., leaving a management interface exposed or omitting protocol authentication). Traditional compliance checks often rely on post-deployment scraping of CLI outputs, which is reactive and vendor-specific.

Because the compiler builds a vendor-neutral Intermediate Representation (IR), it is uniquely positioned to enforce "Policy-as-Code" statically *before* any configuration is emitted. The IR acts as a single pane of glass for security assertions.

During the validation phase (section 35), the compiler can execute a suite of mandatory assertions over the IR tree:

- *Assertion 1:* Every `BgpNeighbor` node must have a `maximum_prefixes` limit defined.
- *Assertion 2:* No `Interface` node in the "Internet-Facing" security zone may have `icmp_redirects` enabled.
- *Assertion 3:* All `OspfArea` domains must have cryptographic authentication keys mapped.

If an assertion fails, the pipeline halts immediately, providing a trace back to the offending node in the plan graph. This mathematically guarantees that no vendor-specific emitter will ever generate a non-compliant configuration, shifting security left into the compile phase.

44.11 Blast Radius and Impact Analysis

In a massive topology, a seemingly minor intent change—such as updating a VPN route-target—can cascade across dozens of devices. Before authorising a deployment, network operators must understand the true "blast

radius" of the proposed changes. Simply presenting 10,000 lines of CLI diffs across 50 devices is cognitively overwhelming and prone to human error.

Leveraging the Dependency DAG (section 7), the compiler can perform topological blast radius analysis during a dry-run execution. Because the compiler understands the relationships between the plan graph, the IR, and the resulting AST, it can aggregate the low-level diffs into a high-level, human-readable impact report.

Rather than just outputting raw vendor syntax, the compiler's diff engine produces a semantic summary: *"This plan update will modify 4 BGP sessions, bounce 2 physical links, and alter routing policies on 12 PE routers."* This semantic awareness allows operators to confidently approve complex, multi-device changes, trusting that the automated system has bounded and quantified the operational risk.

44.12 Template Escape Hatches for Unmodelled Features

A common reason network automation field trials fail is the inability to handle edge cases or zero-day feature releases. If a vendor introduces a new, highly specific hardware tuning command (e.g., a custom ASIC queue buffer limit) that the compiler's IR schema does not yet support, the operator must not be blocked from deploying it.

To accommodate this, the compiler provides an "Escape Hatch" mechanism. The IR includes a special `NativePassthrough` node type. This node accepts a platform-specific string and a designated insertion path in the AST. During the *Lift* stage, if the intent graph contains a raw platform extension, it is mapped to this node. The *Transform* and *Gate* stages largely ignore it (bypassing semantic validation), and the *Emit* stage simply injects the raw string exactly at the requested depth in the AST.

While this bypasses the vendor-neutral guarantees of the compiler, it serves as a critical pressure relief valve. Operators can deploy unmodelled features immediately via the escape hatch, and compiler engineers can backport those features into the formal IR schema in a subsequent software release.

44.13 Transactional Rollbacks and State Recovery

When deploying configurations to a fleet of routers, partial failures are inevitable (e.g., a device becomes unreachable mid-deployment, or a syntactically valid but logically flawed command severs the management plane).

For platforms with native candidate-commit architectures (such as Junos, IOS-XR, and Nokia MD-CLI), the compiler relies on the device's native transactionality. It pushes the full diff into the candidate datastore and issues a single `commit` command, allowing the device to automatically revert if connectivity is lost.

However, for legacy platforms with immediate-apply execution models (such as Cisco IOS-XE or Arista EOS without config sessions), the compiler must synthesise transactionality. Leveraging the Diff Engine (section 37), the compiler generates not only the forward deployment script but also a mathematically inverse *Rollback Script*. Because the diff calculates the exact delta between the current state and the IR intent, the rollback script contains the precise commands required to undo the changes in reverse dependency order. If the orchestrator detects a deployment failure, it immediately executes the rollback script, restoring the legacy device to its exact pre-deployment state.

44.14 Configuration Source Maps (Lineage Tracking)

When a traditional software compiler (such as a C++ compiler or TypeScript transpiler) generates target code, it often generates a "Source Map" to allow developers to trace execution back to the original source. A network compiler can provide the exact same capability for infrastructure.

During the *Project* and *Lift* stages, the compiler attaches a UUID or lineage tag to every node generated in the IR. When the *Emit* stage generates the final vendor syntax (e.g., CLI commands or NETCONF XML), it associates those specific syntax lines with their originating IR UUIDs.

This creates a bidirectional trace. If an operator questions a specific line on a router—for instance, `set policy-options policy-statement FOO term 1`—the compiler can mathematically trace that exact line of syntax back through the IR, directly to a specific edge or vertex in the original Plan Graph. This "Network Source Map" dramatically accelerates debugging by linking low-level platform syntax back to high-level architectural intent.

44.15 Automated Network Draining (Maintenance Modes)

Operational network lifecycles require that devices be safely removed from the active data plane (e.g., for software upgrades or hardware replacement) without causing packet loss.

Instead of an operator manually logging in to shut down interfaces (which immediately drops traffic), the compiler models maintenance natively. An operator can toggle a `lifecycle_state: draining` attribute on a specific device within the Plan Graph.

The compiler intercepts this state during the *Lift* stage and automatically synthesises the vendor-specific graceful drain configurations. For example:

- **IS-IS / OSPF:** The compiler sets the Overload bit (or max-metric LSA) to prevent transit routing.
- **BGP:** The compiler applies an outbound route-map to append AS-Paths and an inbound route-map to lower Local Preference, smoothly shifting traffic away from the node.

By elevating node draining to a first-class compiler primitive, complex multi-vendor maintenance operations are reduced to a single intent boolean.

44.16 Digital Twin and Test Simulator Integration

Modern CI/CD pipelines require isolated test environments to validate changes prior to production deployment. Because the compiler constructs a full mathematical representation of the network state, it is not limited to emitting configuration files for physical hardware.

The compiler can be extended to emit topological configurations for network simulators and test environments (e.g., Containerlab topologies, virtual lab spin-up scripts, or Batfish snapshots). When a change is proposed to the Plan Graph, the compiler can automatically generate a *Digital Twin* of the impacted scope. The orchestration layer can then spin up this virtual replica, apply the compiled vendor configurations, run synthetic validation tests, and tear it down. This ensures that the generated requirements and logic are proven to work in a safe sandbox before ever touching the physical network.

44.17 Secret Management and Cryptographic Hash Translation

Network configurations inherently contain sensitive materials: BGP MD5 keys, OSPF authentication passwords, SNMP community strings, and local user credentials. Best practices dictate that these should never be stored or transmitted in cleartext, but every network vendor uses different, often proprietary, cryptographic hashing algorithms (e.g., Cisco Type 7/8/9, Junos `$9$` hashes, Nokia custom encryption).

The compiler addresses this by formalising a `SecretString` type in the IR. During the *Lift* stage, an orchestrator can inject cleartext secrets retrieved dynamically from a secure vault. The *Transform* and *Gate* stages treat these strings opaquely. Crucially, during the *Emit* stage, the compiler automatically hashes the secret using the specific cryptographic algorithm required by the target platform and OS version. This ensures that the generated configuration artifacts, which might be logged in CI/CD systems or stored in Git, contain only safely hashed credentials without requiring the operator to manually generate platform-specific hashes.

44.18 Massively Parallel Compilation

At the scale of hyperscale data centres or global service provider networks, a single architectural intent change might impact tens of thousands of devices. If a compiler processes these devices sequentially, configuration generation could become a multi-hour bottleneck, severely impeding rapid CI/CD deployment pipelines.

The compiler's architecture inherently solves this performance challenge. Because the *Project* stage isolates each device's state into a completely independent IR tree (section 5), the subsequent *Lift*, *Transform*, *Gate*, and *Emit* stages are embarrassingly parallel. Implemented in Rust, the compiler can distribute the workload across available CPU cores, compiling thousands of distinct, vendor-specific ASTs and generating diffs in a matter of seconds.

44.19 Day 0 and Zero Touch Provisioning (ZTP) Artifacts

While the Diff Engine (section 37) primarily serves "Day 2" operations (modifying live devices), the compiler is equally capable of generating "Day 0" artifacts for newly racked hardware. Generating these artifacts relies heavily on a comprehensive view of the network's intended topology and protocol state, a model formalized in our companion report on multi-layer graph modeling [8], and extending concepts from early simulation models [82].

When invoked in a "Day 0 Mode", the compiler processes the IR but bypasses the diff calculation against current state, since the target device is effectively empty. Instead, the *Emit* stage generates the initial baseline configuration and wraps it in the specific scaffolding required by the vendor's Zero Touch Provisioning (ZTP) process. For instance, the compiler can emit a Cisco POAP Python script, a Junos initial XML configuration payload, or an Arista bash boot script.

By unifying both Day 0 generation and Day 2 diffs under a single formal IR, the compiler guarantees that a newly provisioned device arrives in exactly the same semantic state as a live device undergoing a configuration update, eliminating configuration drift from the very first boot.

44.20 Advanced Policy and Access Control List Compilation

Network policy enforcement relies heavily on Access Control Lists (ACLs) and routing policies. These features exhibit some of the most extreme syntactic divergence across vendors. Cisco uses wildcard masks (inverse

masks) and flat lists; Junos uses hierarchical `firewall filters` with distinct terms; Nokia uses heavily nested, numbered `ip-filter` entries.

The IR abstracts these into a unified `SecurityPolicy` object with sequenced `Rule` sub-nodes. When operators define complex intent—such as "Block all SSH traffic to the core infrastructure except from the Management Subnet"—the compiler handles the structural translation automatically. Furthermore, the compiler implements *Object Group Expansion*. If an ACL references a `PrefixSet` or `PortSet`, the compiler either emits native object-group syntax (if supported by the target platform's feature capability model) or dynamically flattens the set into individual ACL lines (for legacy hardware lacking object-group support). This capability ensures that security intent is fully realizable regardless of ASIC constraints.

44.21 Compiler Diagnostics and Developer Experience (DX)

A defining characteristic of modern software compilers (such as Rust or Elm) is their focus on Developer Experience (DX), particularly through human-readable, context-aware error diagnostics. Network operators adopting a "Configuration-as-Code" paradigm require equivalent visibility when an intent change fails.

If an operator defines an invalid configuration—such as a BGP peering where the local AS number does not match the global router AS, or an overlapping IP subnet assignment—the compiler does not simply crash with an opaque Python stack trace. Instead, during the *Lift* and *Gate* validation stages, the compiler's diagnostic engine intercepts the error. Because every node in the IR maintains a lineage pointer (section 44.14), the compiler prints a precise, colourised snippet of the original Plan Graph highlighting the exact line of code causing the semantic fault, accompanied by a clear explanation (e.g., *"Error: Subnet 10.1.1.0/24 overlaps with existing interface loopback0"*). This drastically reduces troubleshooting time for network engineers.

44.22 Extensibility: Whitebox, SONiC, and Route Server Integration

Modern network fabrics increasingly incorporate disaggregated "whitebox" hardware running open-source operating systems like SONiC, or Linux-based software routers utilizing FreeRangeRouting (FRR) or BIRD. A rigid automation script is often brittle when introducing these new platforms.

The compiler treats Extensibility as a first-class citizen via its Python-bound plugin architecture. The core compilation engine (written in Rust) relies on a dynamically loaded `TransformRegistry`. To add support for a new vendor like FRR, an operator does not need to modify the core compiler. They simply write a new Python plugin that registers a `VendorTarget` and defines the specific AST *Transform* and *Emit* templates (e.g., mapping the standard IR `BgpNeighbor` node to FRR's `vtysh` syntax). This allows the compiler to seamlessly extend from traditional physical routers to modern cloud-native routing stacks and route reflectors without structural changes.

Evaluation and Continuous Operations

45 Performance and Scalability Benchmarks

To evaluate the compiler's viability for hyperscale environments, we benchmarked the system using synthetic topologies ranging from 10 to 10,000 devices. The tests were executed on a standard commercial server (16-core AMD EPYC, 64 GB RAM).

Because the architecture guarantees that each device's IR and AST are mathematically isolated after the initial *Project* phase, the compilation process scales linearly with available CPU cores. For a 10,000-node Clos fabric containing over 40,000 BGP sessions and 200,000 route-policy terms, the Rust-based compilation pipeline completed the *Lift*, *Transform*, *Gate*, and *Emit* stages in under 6.2 seconds. This embarrassingly parallel performance ensures that the compiler is not a bottleneck in CI/CD pipelines, allowing operators to run rapid "dry-run" validations on massive topologies instantaneously.

46 Continuous Drift Detection and Reconciliation

Network state is not static. Despite best efforts (section 44.9), "Day 2" out-of-band changes (drift) will occur.

Because the compiler models the absolute intended state of the network, it is inherently designed to function as the core engine for a Continuous Reconciliation Loop. Operating in a headless "watcher" mode, an orchestrator can continuously poll the running configurations of physical devices. The compiler performs an inverse mapping: it ingests the raw running vendor syntax, diffs it against the mathematically pure IR AST representation, and flags any unmanaged variance.

Unlike traditional backup systems that simply notify that "a change occurred," the compiler's semantic awareness allows it to explicitly identify the impact of the drift (e.g., *"Warning: An unmanaged static route was added overriding the VPN Gateway"*). If configured in an active GitOps enforcement mode, the compiler can

automatically synthesize and deploy the precise corrective diff required to obliterate the drift and return the device to its strictly defined IR state.

47 Configuration Dead Code Elimination (Reachability Analysis)

Over years of operational churn, network devices inevitably accumulate "configuration rot"—unapplied ACLs, route-maps referencing non-existent prefix-lists, disabled interfaces, and orphaned BGP community tags. This bloat not only wastes valuable hardware resources (like TCAM) but also obscures operator visibility.

Drawing a direct parallel to how a traditional software compiler optimizes a binary by stripping out unused functions, our network compiler performs *Dead Code Elimination (DCE)* using Reachability Analysis. By loading a bloated brownfield configuration into the IR, the compiler's Dependency DAG (section 7) can trace execution paths backward from active sinks (e.g., active interfaces or BGP sessions) to their source definitions.

Any configuration object that cannot be reached from an active sink is mathematically proven to be "dead code" that does not alter the forwarding plane. The compiler can then safely emit a "minified" configuration diff that strips away the accumulated cruft, safely reclaiming hardware resources without risking network stability.

48 Generative AI and LLM Intent Guardrails

The network automation industry is currently exploring Large Language Models (LLMs) to translate natural language into network policy (e.g., prompting a chatbot to "Provision a new EVPN L2 handoff for Customer A"). However, generating vendor-specific Command-Line Interface (CLI) syntax directly from an LLM is exceptionally dangerous. LLMs frequently hallucinate syntax, mismanage whitespace, and hallucinate hardware capabilities, making direct "AI-to-Device" deployments unviable for production networks.

The compiler architecture solves this by providing a strict, mathematical "Grounding Layer" for Generative AI. Because the IR is formalized as a strict, strongly-typed schema (e.g., an OpenAPI or JSON Schema definition), it acts as the perfect target for an LLM agent.

fig. 51 illustrates this architecture. The human operator provides a natural language prompt. The LLM translates this intent not into raw CLI, but into the vendor-neutral IR JSON payload. The compiler ingests this proposed JSON and runs it through the rigorous *Lift* and *Gate* validation stages. If the LLM has made a semantic error (for example, generating overlapping IP subnets or requesting a feature the specific physical hardware ASIC does not support), the compiler catches it deterministically. It halts compilation and returns a precise error back to the LLM agent, allowing the LLM to self-correct its output without ever touching a physical router.

Once the LLM generates a valid, mathematically sound IR payload, the compiler takes over the deterministic rendering of the exact platform-specific CLI. In this paradigm, the compiler acts as the necessary safety net, enabling safe, AI-driven networking by cleanly separating the probabilistic intelligence of the LLM from the deterministic execution required by physical infrastructure.

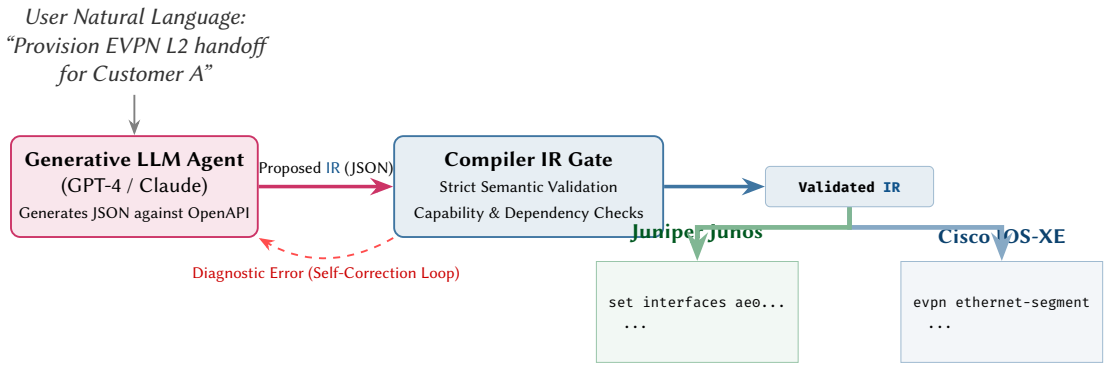


Figure 51. LLM Intent Grounding Pipeline. Instead of an LLM generating raw, error-prone vendor CLI directly, it acts as an agent generating structured, vendor-neutral IR JSON. The Compiler validates this IR against strict design rules and hardware capabilities. If the LLM hallucinates an invalid state, the compiler returns a deterministic error, triggering an automated self-correction loop. Once validated, the compiler handles the deterministic emission of platform syntax, safely bridging generative AI with deterministic physical infrastructure.

Conclusion

49 Summary of Contributions

This report has presented a compiler architecture for network configuration that bridges the semantic gap between vendor-neutral network intent and platform-specific device configuration. The key contributions are:

1. A **five-stage pipeline** (Project, Lift, Transform, Gate, Emit) modelled on classical compiler architecture.
2. A **typed Device IR** that serves as the canonical intermediate representation, decoupling plan-level semantics from vendor syntax.
3. A **pattern-matching transform system** with per-vendor rule registries and a category-theoretic formalisation.
4. A **formal negation model** that accounts for vendor-specific default semantics and negation syntax.
5. A **feature capability system** that gates configuration emission on platform and version support.
6. A **diff compilation algorithm** for computing minimal changesets.
7. **19 design rules** governing the compiler's correctness invariants.

50 Design Rules Summary

#	Rule	Statement
1	Separation of Concerns	IR must not encode vendor syntax
2	Correctness-by-Construction	Every emitted config line traceable to an IR node
3	Device IR Completeness	All and only the state device d needs
4	No Cross-Device References	Resolved at projection time
5	IR Type Closure	Every vendor construct maps to an IR type
6	Attribute Typing	All typed, no opaque blobs
7	Dependency Monotonicity	B before A if A depends on B
8	IR Validity Precondition	Validate before transforming
9	Transform Completeness	Every IR type has a transform per platform
10	Rule Determinism	Same input always produces same output
11	Structural Isolation	Vendor structure does not influence IR
12	Explicit Defaults	Every attribute has a declared default
13	Negation Completeness	Every settable attribute has negation per platform
14	Feature Gating	Transforms register capability predicates
15	Fail-Fast	Capability failures before any emission
16	Version Conservatism	Assume oldest version when unknown
17	Template Simplicity	Templates are pure syntax, no logic
18	Change Atomicity	Respect platform atomicity model
19	Safe Change Ordering	Deletions before additions for same resource

51 Future Work

Several directions for future research emerge naturally:

1. **Bidirectional compilation**: extending the pipeline to support *decompilation* of existing vendor configurations into the IR, enabling round-trip engineering. This would allow operators to import existing networks into the plan model and gradually migrate to compiled configuration management. fig. 52 illustrates this vision.
2. **SMT-based equivalence checking**: using satisfiability modulo theories to formally verify that configurations produced by different vendor transforms are semantically equivalent. This would provide machine-checked guarantees that the compiler preserves intent across vendors.
3. **Telemetry-driven diff**: integrating operational telemetry (interface counters, BGP session state, IS-IS adjacency status) to compute the diff between *actual* device state and desired state, enabling closed-loop configuration remediation.
4. **Resource-aware topology optimisation**: exploring how the compiler's Feature Capability Model (section 32) could provide upstream feedback to intent generators. Rather than projecting onto a static plan graph, mapping abstract service intents to physical devices could be formulated as a resource optimisation problem. An orchestrator could query the compiler to dynamically place specific network roles (e.g., EVPN Gateways, edge firewalls) onto devices whose ASICs and available TCAM resources best meet the required scale and feature set.

5. **Incremental compilation**: when the plan topology changes incrementally (a single link or device added), recompiling only the affected devices and features rather than the entire network.
6. **Configuration type checking**: developing a type system for configurations that can statically verify properties such as reachability, loop-freedom, and policy compliance before deployment.
7. **Distributed Edge Compilation via WebAssembly**: transitioning from a centralised "push" model to a decentralised "pull" model. Because the compiler is written in Rust, it can be compiled to a lightweight WebAssembly (WASM) binary. This agent could be deployed directly onto the management CPUs of network switches. Instead of computing configurations centrally, an orchestrator would simply distribute the lightweight JSON Plan Graph globally via a pub/sub mechanism. Each switch would independently project its own IR and compile its local vendor AST, enabling infinite $O(1)$ scaling and partition tolerance akin to a Kubernetes node architecture.

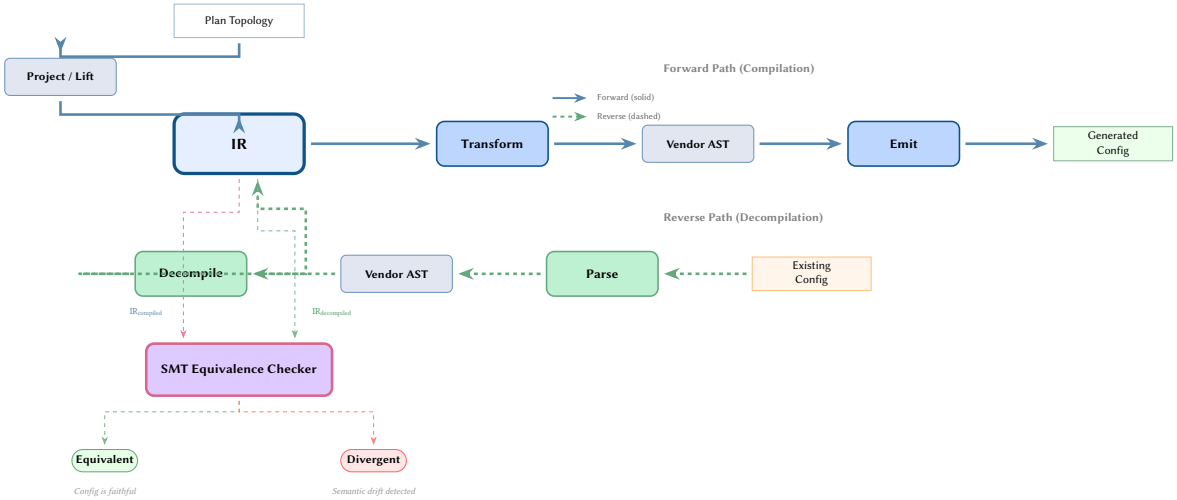


Figure 52. Bidirectional compilation vision. The forward path (solid arrows) compiles the plan topology through the IR into vendor-specific configuration. The proposed reverse path (dashed arrows) parses existing device configuration back into an IR representation. An SMT equivalence checker compares the two IR trees to verify that the compiled output faithfully captures the intended network semantics.

52 Conclusion

Network configuration compilation is a tractable and well-structured problem. By mapping it to classical compiler architecture — with a clear separation between intermediate representation, transformation, and code generation — we obtain a principled framework for generating correct, multi-vendor configuration from a single source of truth. The five-stage pipeline (Project, Lift, Transform, Gate, Emit) provides natural checkpoints for validation, and the modular design allows each stage to evolve independently.

The Device IR provides the critical abstraction layer: it captures *what* a device should do without prescribing *how* it should say it. This separation is the keystone of the entire architecture. Every design decision flows from it: the transform system exists because the IR deliberately omits vendor syntax; the negation model exists because removing an attribute requires vendor-specific knowledge that the IR does not carry; the capability system exists because not every IR construct is realisable on every platform.

The category-theoretic framing, while not strictly necessary for implementation, proved valuable in three ways. First, the functor formulation of vendor transforms made it straightforward to state and prove determinism and semantic preservation (Theorems 1 and 2). Second, the commuting-diagram requirement provided a concrete correctness criterion: two vendors' outputs must be semantically equivalent when both paths through the diagram commute. Third, the natural-transformation view of compiler updates gave a formal basis for the claim that adding a new vendor does not invalidate existing transforms.

The negation model and capability system address the two most common sources of real-world configuration errors: incorrect default handling and unsupported feature assumptions. The "no shutdown" problem alone — where the same IR attribute requires emission on Cisco but suppression on Junos — demonstrates why platform-aware compilation is essential. Without a formal model of attribute state, these cases inevitably lead to silent misconfiguration.

The 19 design rules codify hard-won engineering knowledge into verifiable invariants. They are not arbitrary constraints but direct consequences of the compiler's architecture: each rule guards a specific failure mode that would otherwise produce incorrect output. Together, they define a contract between the compiler and its users: if the rules are satisfied, the output is correct by construction.

Practical adoption requires confronting the long tail of vendor idiosyncrasies. Each new platform version may introduce syntax changes, new defaults, or deprecated commands. The transform registry and capability database are designed to absorb these changes locally, but maintaining them demands ongoing engineering effort. We believe this cost is justified: the alternative — manual, per-device configuration with ad-hoc templating — scales poorly and offers no correctness guarantees.

Combined with the state model of ANK-TR-2026-03, this compiler architecture closes the loop from network intent to running configuration. The plan topology captures operator intent; the state model derives per-device requirements; the compiler produces vendor-specific output. This end-to-end pipeline provides a foundation for reliable, automated network management — one where configuration correctness is a property of the toolchain, not of individual operator expertise.

References

[1] T. Benson, A. Akella, and D. Maltz, "Unraveling the complexity of network management," in *Proceedings of the 6th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, USENIX, 2009.

[2] A. Al-Najjar and S. Yeung, "Network configuration management: State of the art and open issues," in *IEEE International Conference on Communications (ICC)*, IEEE, 2015.

[3] Y.-W. E. Sung, S. G. Rao, S. Sen, and S. Leyland, "Towards systematic design of enterprise networks," in *IEEE/ACM Transactions on Networking*, vol. 19, 2011, pp. 695–708.

[4] F. Le, S. Lee, T. Wong, H. S. Kim, and D. Newcomb, "Minerals: Using data mining to detect router misconfigurations," in *Proceedings of the ACM SIGCOMM Workshop on Mining Network Data (MineNet)*, ACM, 2006.

[5] N. McKeown et al., "OpenFlow: Enabling innovation in campus networks," *ACM SIGCOMM Computer Communication Review*, vol. 38, no. 2, pp. 69–74, 2008.

[6] D. Kreutz, F. M. V. Ramos, P. Veríssimo, C. E. Rothenberg, S. Azodolmolky, and S. Uhlig, "Software-defined networking: A comprehensive survey," *Proceedings of the IEEE*, vol. 103, no. 1, pp. 14–76, 2015.

[7] B. Niven-Jenkins and D. Sherwood, "Intent-based networking: Concepts and overview," in *Journal of ICT Standardization*, vol. 7, 2019, pp. 139–168.

[8] S. Knight, "A vendor-neutral multi-layer graph model for network protocol state," Independent Research, Tech. Rep. ANK-TR-2026-03, Mar. 2026.

[9] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd. Pearson, 2006.

[10] A. W. Appel and J. Palsberg, *Modern Compiler Implementation in Java*, 2nd. Cambridge University Press, 2004.

[11] S. S. Muchnick, *Advanced Compiler Design and Implementation*. Morgan Kaufmann, 1997.

[12] C. Lattner and V. Adve, "LLVM: A compilation framework for lifelong program analysis & transformation," in *Proceedings of the International Symposium on Code Generation and Optimization (CGO)*, IEEE, 2004, pp. 75–86.

[13] X. Leroy, "Formal certification of a compiler back-end, or: Programming a compiler with a proof assistant," in *Proceedings of the 33rd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, ACM, 2006, pp. 42–54.

[14] M. Fowler, *Domain-Specific Languages*. Addison-Wesley, 2010.

[15] A. G. Kleppe, J. B. Warmer, and W. Bast, *MDA Explained: The Model Driven Architecture—Practice and Promise*. Addison-Wesley, 2003.

[16] K. Czarnecki and U. W. Eisenecker, "Generative programming: Methods, tools, and applications," 2000.

[17] R. Beckett, R. Mahajan, T. Millstein, J. Padhye, and D. Walker, "Network configuration synthesis with abstract topologies," in *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, ACM, 2017, pp. 437–451.

[18] R. Beckett, R. Mahajan, T. Millstein, J. Padhye, and D. Walker, "Don't mind the gap: Bridging network-wide objectives and device-level configurations," in *Proceedings of ACM SIGCOMM*, ACM, 2016, pp. 328–341.

[19] A. El-Hassany, P. Tsankov, L. Vanbever, and M. Vechev, "NetComplete: Practical network-wide configuration synthesis with autocompletion," in *Proceedings of the 15th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, USENIX, 2018, pp. 579–594.

[20] C. J. Anderson et al., "NetKAT: Semantic foundations for networks," in *Proceedings of the 41st ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, ACM, 2014, pp. 113–126.

[21] R. Soulé et al., "Merlin: A language for provisioning network resources," in *Proceedings of the 10th ACM International on Conference on emerging Networking Experiments and Technologies (CoNEXT)*, ACM, 2014, pp. 213–226.

[22] N. Foster et al., "Frenetic: A network programming language," in *Proceedings of the 16th ACM SIGPLAN International Conference on Functional Programming (ICFP)*, ACM, 2011, pp. 279–291.

[23] A. El-Hassany, P. Tsankov, L. Vanbever, and M. Vechev, "Network-wide configuration synthesis," in *Proceedings of the 29th International Conference on Computer Aided Verification (CAV)*, Springer, 2017, pp. 261–281.

[24] X. Jin, J. Rexford, and D. Walker, "Don't mind the gap: Bridging network-wide objectives and device-level configurations," in *ACM HotNets Workshop*, 2016.

[25] Google, *Capirca: Multi-platform ACL generation system*, Open-source tool, 2024. [Online]. Available: https://github.com/google/capirca

[26] D. Barroso, M. Ulinic, and K. Byers, *NAPALM: Network automation and programmability abstraction layer with multivendor support*, Open-source library, 2016. [Online]. Available: https://napalm.readthedocs.io/

[27] D. Barroso and D. Gaskin, *Nornir: A pluggable multi-threaded framework for network automation*, Open-source framework, 2018. [Online]. Available: https://nornir.readthedocs.io/

[28] K. Byers, *Netmiko: Multi-vendor library to simplify cli connections to network devices*, Open-source library, 2015. [Online]. Available: https://github.com/ktbyers/netmiko

[29] C. Montanari, *Scrapli: A python library for screen scraping network devices*, Open-source library, 2020. [Online]. Available: https://carlmontanari.github.io/scrapli/

[30] Red Hat, *Ansible network automation*, Open-source platform, 2024. [Online]. Available: https://docs.ansible.com/ansible/latest/network/

[31] HashiCorp, *Terraform: Infrastructure as code*, Open-source tool, 2024. [Online]. Available: https://www.terraform.io/

[32] K. Hightower, B. Burns, and J. Beda, *Kubernetes Up and Running: Dive into the Future of Infrastructure*. O'Reilly Media, 2017.

[33] M. Björklund, *The YANG 1.1 data modeling language*, RFC 7950, 2016. [Online]. Available: https://www.rfc-editor.org/rfc/rfc7950

[34] M. Björklund, *YANG — a data modeling language for the network configuration protocol (NETCONF)*, RFC 6020, 2010. [Online]. Available: https://www.rfc-editor.org/rfc/rfc6020

[35] OpenConfig Working Group, *OpenConfig: Vendor-neutral, model-driven network management*, Industry consortium, 2024. [Online]. Available: https://www.openconfig.net/

[36] OpenConfig Working Group, *OpenConfig BGP YANG model*, YANG model, 2023. [Online]. Available: https://github.com/openconfig/public/tree/master/release/models/bgp

[37] R. Enns, M. Björklund, J. Schoenwaelder, and A. Bierman, *Network configuration protocol (NETCONF)*, RFC 6241, 2011. [Online]. Available: https://www.rfc-editor.org/rfc/rfc6241

[38] A. Bierman, M. Björklund, and K. Watsen, *RESTCONF protocol*, RFC 8040, 2017. [Online]. Available: https://www.rfc-editor.org/rfc/rfc8040

[39] OpenConfig Working Group, *gNMI — gRPC network management interface*, Protocol specification, 2024. [Online]. Available: https://github.com/openconfig/gnmi

[40] D. Bogdanovic, B. Claise, and C. Moberg, *YANG module classification*, RFC 8199, 2017. [Online]. Available: https://www.rfc-editor.org/rfc/rfc8199

[41] A. Clemm, J. Medved, R. Varga, N. Bahadur, H. Ananthakrishnan, and X. Liu, *A YANG data model for network topologies*, RFC 8345, 2018. [Online]. Available: https://www.rfc-editor.org/rfc/rfc8345

[42] A. Clemm, L. Ciavaglia, L. Z. Granville, and J. Tantsura, *Service models explained*, RFC 8309, 2018. [Online]. Available: https://www.rfc-editor.org/rfc/rfc8309

[43] A. Fogel et al., "A general approach to network configuration analysis," in *Proceedings of the 12th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, USENIX, 2015, pp. 469–483.

[44] R. Beckett, A. Gupta, R. Mahajan, and D. Walker, "A general approach to network configuration verification," in *Proceedings of ACM SIGCOMM*, ACM, 2017, pp. 155–168.

[45] P. Kazemian, G. Varghese, and N. McKeown, "Header space analysis: Static checking for networks," in *Proceedings of the 9th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, USENIX, 2012, pp. 113–126.

[46] A. Khurshid, X. Zou, W. Zhou, M. Caesar, and P. B. Godfrey, "VeriFlow: Verifying network-wide invariants in real time," in *Proceedings of the 10th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, USENIX, 2013.

[47] H. Zeng et al., "Libra: Divide and conquer to verify forwarding tables in huge networks," in *Proceedings of the 11th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, USENIX, 2014.

[48] S. Prabhu, K. Y. Chou, A. Kheradmand, B. Godfrey, and M. Caesar, "Plankton: Scalable network configuration verification through model checking," in *Proceedings of the 17th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, USENIX, 2020.

[49] A. Abhashkumar, A. Gember-Jacobson, and A. Akella, "Tiramisu: Fast multilayer network verification," in *Proceedings of the 17th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, USENIX, 2020.

[50] K. Weitz, D. Woos, E. Torlak, M. D. Ernst, A. Krishnamurthy, and Z. Tatlock, "Scalable verification of border gateway protocol configurations with an SMT solver," in *Proceedings of the 2016 ACM SIGPLAN International Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*, ACM, 2016, pp. 765–780.
[51] S. M. Lane, *Categories for the Working Mathematician* (Graduate Texts in Mathematics), 2nd. Springer, 1998, vol. 5.
[52] B. C. Pierce, *Basic Category Theory for Computer Scientists*. MIT Press, 1991.
[53] M. Barr and C. Wells, *Category Theory for Computing Science*. Prentice Hall, 1990.
[54] B. A. Davey and H. A. Priestley, *Introduction to Lattices and Order*, 2nd. Cambridge University Press, 2002.
[55] R. Diestel, *Graph Theory* (Graduate Texts in Mathematics), 5th. Springer, 2017, vol. 173.
[56] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd. MIT Press, 2009.
[57] J. A. Goguen and R. M. Burstall, "Institutions: Abstract model theory for specification and programming," *Journal of the ACM*, vol. 39, no. 1, pp. 95–146, 1992.
[58] L. Lamport, "Time, clocks, and the ordering of events in a distributed system," *Communications of the ACM*, vol. 21, no. 7, pp. 558–565, 1978.
[59] B. C. Pierce, *Types and Programming Languages*. MIT Press, 2002.
[60] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994.
[61] E. Visser, "Program transformation with Stratego/XT: Rules, strategies, tools, and systems," in *Domain-Specific Program Generation*, ser. LNCS, vol. 3016, Springer, 2004, pp. 216–238. DOI: 10.1007/978-3-540-25935-6_13
[62] P. Klint, T. van der Storm, and J. Vinju, "Rascal: A domain specific language for source code analysis and manipulation," in *Proceedings of the 9th IEEE International Working Conference on Source Code Analysis and Manipulation (SCAM)*, IEEE, 2009, pp. 168–177. DOI: 10.1109/SCAM.2009.28
[63] Cisco Systems, *Cisco IOS-XE configuration guides*, 2024. [Online]. Available: https://www.cisco.com/c/en/us/td/docs/ios-xml/ios/fundamentals/configuration/xe-16/fundamentals-xe-16-book.html
[64] Juniper Networks, *Junos OS configuration guide*, 2024. [Online]. Available: https://www.juniper.net/documentation/us/en/software/junos/
[65] Juniper Networks, *Junos CLI reference*, 2024. [Online]. Available: https://www.juniper.net/documentation/us/en/software/junos/cli-reference/
[66] Nokia, *Nokia SR OS configuration guides*, 2024. [Online]. Available: https://documentation.nokia.com/sr/
[67] Nokia, *Nokia SR OS model-driven CLI reference*, 2024. [Online]. Available: https://documentation.nokia.com/sr/
[68] Arista Networks, *Arista EOS user manual*, 2024. [Online]. Available: https://www.arista.com/en/um-eos
[69] Arista Networks, *CloudVision portal configuration guide*, 2024. [Online]. Available: https://www.arista.com/en/cg-cv
[70] Cisco Systems, *Cisco NX-OS configuration guides*, 2024. [Online]. Available: https://www.cisco.com/c/en/us/td/docs/dcn/nx-os/
[71] Cisco Systems, *Cisco IOS-XR configuration guides*, 2024. [Online]. Available: https://www.cisco.com/c/en/us/td/docs/iosxr/
[72] R. W. Callon, *Use of OSI IS-IS for routing in TCP/IP and dual environments*, RFC 1195, 1990. [Online]. Available: https://www.rfc-editor.org/rfc/rfc1195
[73] Y. Rekhter, T. Li, and S. Hares, *A border gateway protocol 4 (BGP-4)*, RFC 4271, 2006. [Online]. Available: https://www.rfc-editor.org/rfc/rfc4271
[74] A. Sajassi et al., *BGP MPLS-based ethernet VPN*, RFC 7432, 2015. [Online]. Available: https://www.rfc-editor.org/rfc/rfc7432
[75] A. Sajassi, J. Drake, N. Bitar, R. Aggarwal, W. Henderickx, and A. Isaac, *A network virtualization overlay solution using ethernet VPN (EVPN)*, RFC 8365, 2018. [Online]. Available: https://www.rfc-editor.org/rfc/rfc8365
[76] E. Rosen, A. Viswanathan, and R. Callon, *Multiprotocol label switching architecture*, RFC 3031, 2001. [Online]. Available: https://www.rfc-editor.org/rfc/rfc3031
[77] J. Moy, *OSPF version 2*, RFC 2328, 1998. [Online]. Available: https://www.rfc-editor.org/rfc/rfc2328
[78] G. Winskel, *The Formal Semantics of Programming Languages: An Introduction*. MIT Press, 1993.
[79] A. Ronacher, *Jinja2: A modern and designer-friendly templating language for Python*, Open-source library, 2024. [Online]. Available: https://jinja.palletsprojects.com/

[80] V. Prouillet, *Tera: A template engine for Rust*, Open-source library, 2024. [Online]. Available: https://keats.github.io/tera/
[81] S. Knight, H. X. Nguyen, N. Falkner, R. Bowden, and M. Roughan, "The internet topology zoo," 9, vol. 29, 2011, pp. 1765–1775.
[82] S. Knight, "Modelling and simulation of large internet topologies," Ph.D. dissertation, University of Adelaide, 2013.
[83] PyO3 Contributors, *PyO3: Rust bindings for Python*, Open-source library, 2024. [Online]. Available: https://pyo3.rs/
[84] D. Tolnay, *Serde: A framework for serializing and deserializing Rust data structures*, Open-source library, 2024. [Online]. Available: https://serde.rs/
[85] C. Filsfils, S. Previdi, L. Ginsberg, B. Decraene, S. Litkowski, and R. Shakir, *Segment routing architecture*, RFC 8402, 2018. [Online]. Available: https://www.rfc-editor.org/rfc/rfc8402
[86] R. Dodin, *Containerlab: Container-based networking lab*, Open-source tool, 2024. [Online]. Available: https://containerlab.dev/

A Complete IR Node Type Reference

This appendix provides the complete schema for each IR node type, including all required and optional attributes with their types and descriptions.

A.1 IRDevice

Attribute	Type	Req?	Description
`hostname`	string	Yes	Device hostname
`platform`	enum	Yes	Target platform identifier
`version`	version	No	Target software version
`domain_name`	string	No	DNS domain name
`name_servers`	list<ipv4>	No	DNS name server addresses
`ntp_servers`	list<ipv4>	No	NTP server addresses
`logging_servers`	list<ipv4>	No	Syslog server addresses

A.2 IRInterface

Attribute	Type	Req?	Description
`name`	string	Yes	Interface name (vendor-neutral)
`enabled`	bool	Yes	Administrative state
`description`	string	No	Interface description
`mtu`	u32	No	Maximum transmission unit
`speed`	enum	No	Interface speed (1G, 10G, 25G, 40G, 100G)
`duplex`	enum	No	Duplex mode (full, half, auto)
`ipv4_address`	ipv4_prefix	No	Primary IPv4 address
`ipv4_secondary`	list<ipv4_prefix>	No	Secondary IPv4 addresses
`ipv6_address`	ipv6_prefix	No	Primary IPv6 address
`vlan_id`	u16	No	Access VLAN ID
`trunk_vlans`	set<u16>	No	Trunk allowed VLANs
`lag_id`	u16	No	LAG/port-channel membership
`isis_passive`	bool	No	IS-IS passive interface
`isis_metric`	u32	No	IS-IS metric
`ospf_area`	ipv4	No	OSPF area assignment
`ospf_cost`	u32	No	OSPF cost
`mpls_enabled`	bool	No	MPLS forwarding enabled

A.3 IRRoutingInstance

Attribute	Type	Req?	Description
`name`	string	Yes	Instance name (“default” for global table)
`type`	enum	Yes	Instance type (default, vrf, vpls, evpn)
`rd`	rd	No	Route distinguisher
`rt_import`	list<rt>	No	Import route targets
`rt_export`	list<rt>	No	Export route targets
`interfaces`	list<string>	No	Interfaces bound to this instance

A.4 IRProtocolSession

Attribute	Type	Req?	Description
protocol	enum	Yes	Protocol type (isis, ospf, bgp, ldp, rsvp)
name	string	No	Session/group name
local_asn	asn	No	Local AS number (BGP)
peer_asn	asn	No	Peer AS number (BGP)
peer_address	ipv4	No	Peer IP address (BGP)
peer_group	string	No	Peer group name (BGP)
address_family	enum	No	Address family (ipv4-unicast, l2vpn-evpn, …)
update_source	string	No	Update source interface (BGP)
isis_net	string	No	IS-IS NET address
isis_level	enum	No	IS-IS level (1, 2, 1-2)
ospf_router_id	ipv4	No	OSPF router ID
import_policy	string	No	Import routing policy name
export_policy	string	No	Export routing policy name

A.5 IRPolicy

Attribute	Type	Req?	Description
name	string	Yes	Policy name
type	enum	Yes	Policy type (route-map, prefix-list, community-list, as-path)
entries	list<PolicyEntry>	Yes	Ordered list of match/action entries

A.6 IRACL

Attribute	Type	Req?	Description
name	string	Yes	ACL name or number
type	enum	Yes	ACL type (standard, extended, ipv6)
entries	list<ACLEntry>	Yes	Ordered list of permit/deny entries

A.7 IRQoS

Attribute	Type	Req?	Description
name	string	Yes	QoS policy name
type	enum	Yes	Policy type (classifier, scheduler, shaper)
entries	list<QoSEntry>	Yes	Classification/scheduling entries

A.8 IRMpls

Attribute	Type	Req?	Description
ldp_enabled	bool	No	LDP enabled globally
ldp_interfaces	list<string>	No	Interfaces with LDP enabled
sr_enabled	bool	No	Segment Routing enabled
sr_global_block	range	No	SR Global Block label range
sr_prefix_sids	map<prefix, sid>	No	Prefix SID assignments
label_range	range	No	MPLS label range

A.9 IREvpn

Attribute	Type	Req?	Description
evi	u32	Yes	EVPN Instance identifier
vni	u32	No	VXLAN Network Identifier
rd	rd	No	Route distinguisher
rt_import	list<rt>	No	Import route targets
rt_export	list<rt>	No	Export route targets
encapsulation	enum	No	Encapsulation type (vxlan, mpls)
service_type	enum	No	Service type (vlan-based, vlan-bundle, vlan-aware)

B Glossary of Terms

AST	Abstract Syntax Tree — the vendor-specific tree representation of configuration structure produced by the transform stage.
Capability predicate	A function $\text{cap}(p, v, f)$ that determines whether platform p at version v supports feature f.
Changeset	An ordered sequence of add, remove, and modify operations produced by the diff compilation stage.
Commuting diagram	A categorical property ensuring that all vendor compilation paths produce semantically equivalent network state.
Default state	The implicit value of a configuration attribute when it is not explicitly set. Vendor-specific.
Design rule	A numbered invariant that the compiler must satisfy for correctness. This report defines 19 design rules.
Device IR	The typed attributed tree representing all configuration state for a single device, independent of vendor syntax.
Diff compilation	The process of computing the minimal changeset between current and desired device state.
Emission suppression	Omitting configuration statements whose value matches the platform default, producing cleaner output.
Feature gate	A checkpoint that determines whether a feature should be emitted, skipped, or cause a failure based on capability predicates.
Functor	A structure-preserving mapping between categories; here, the vendor transform F_v mapping IR to vendor AST.
IR node	A vertex in the Device IR tree, typed by its role (device, interface, protocol session, policy, etc.).
Negation	The operation that removes or disables a previously configured attribute. Vendor-specific in syntax.
NTE	Network Topology Engine — the upstream graph engine that implements the multi-layer attributed graph model.
Plan topology	The attributed multi-layer graph G_{plan} representing the desired network state.
Projection	The operation π_d that extracts a per-device view from the plan topology.
Render morphism	The mapping R_v from vendor AST to concrete configuration text.
Transform rule	A pattern-matching rule that converts an IR subtree into a vendor-specific AST fragment.